密　　级：公开

加密论文编号：

论文题目：面向反无人机场景的无人机识别与定位系统设计与实现

学　　号：M202640106

作　　者：梁辰

专 业 名 称：软件工程

2026 年 11 月 20 日

面向反无人机场景的无人机识别与定位系统设计与实现

Design and Implementation of a Drone Recognition and Localization System for Counter-UAV Scenarios

研究生姓名：梁辰
指导教师姓名：段世红
北京科技大学数理学院
北京 100083，中国

Master Degree Candidate：Liang Chen
Supervisor：Shihong Duan
School of Mathematics and Physics
University of Science and Technology Beijing
30 Xueyuan Road，Haidian District
Beijing 100083，P.R.CHINA

中图分类号： TP391 学校代码: 10008
U D C ： 004.8 密 级: 公开

北京科技大学硕士学位论文

论文题目：

面向反无人机场景的无人机识别与定位系统设计与实现

作者： 梁辰

指 导 教 师： 段世红 **单位：** 北京科技大学 **职称：** 副教授

指 导 小 组 成 员： _____ **单位：** _____ **职称：** _____

_____ **单位：** _____ **职称：** _____

论文提交日期： 2026 年 11 月 20 日

学位授予单位： 北 京 科 技 大 学

摘 要

随着低空经济的快速发展，微型无人机在救援、航拍、物流、巡检等领域广泛应用，但“黑飞”“乱飞”等违规飞行事件随之增多，对机场、军区等保密单位构成威胁。这一问题已成为反无人机领域亟待解决的关键问题。雷达、声学、光学、视觉等探测方案中，可见光视觉具有成本低、部署灵活、可直观确认目标等优势，但也面临两个突出问题：一是较远距离的目标在图像中成像过小，往往只有几十个像素，属于典型的小目标检测问题；二是真实场景中树木、建筑等障碍物随处可见，检测数据频繁中断，难以形成连续、持久、可用的航迹。

针对以上问题，本文设计并实现了一个面向反无人机场景的无人机识别与定位系统。在这个系统中，单目可见光相机是唯一的视觉传感器，通过“检测—跟踪—定位—显示”这一流程展开，主要工作包括以下五个方面。

（1）面向小目标检测搭建模型。该模型以 YOLOv8n 为基础，在特征金字塔中增加 stride=4 的 P2 高分辨率检测头，并且在 C2f 模块中嵌入 EMA 高效多尺度注意力机制，结合成 YOLOv8n-P2-EMA 模型。随后将 Det-Fly 数据集划分为训练集与验证集，避免同一序列帧同时出现在训练和验证中，造成数据泄露。四组消融实验表明：YOLOv8n、+P2、+EMA、+P2+EMA 的 mAP50 分别为 81.3%、82.3%、76.2% 与 83.5%，AP_small 分别为 6.1%、8.3%、4.1% 与 9.8%，推理速度分别为 123、90、101 与 98 fps。本文模型以 2.94 M 参数量取得最优综合指标，AP_small 相对基线提升 60.7%，并可满足 30 fps 实时处理需求。

（2）揭示了仿真环境下检测死区的形成机理。在 PX4+jMAVSim 测试中发现，未微调模型在 3–6 m 近距离段检出率接近 0，在 9 m 以上远距离段亦无法稳定锁定，有效工作窗口仅 7–9 m。通过对 2228 帧录像与真值几何的逐帧审计，排除了“俯视视角导致失效”的假设，确认死区的决定因素是目标在图像中的尺度：近距离大尺度样本超出 Det-Fly 的远距小目标分布，属于分布外（OOD）输入；远距离样本则低于初锁门限。

（3）提出了仿真域微调方法并完成自动标注数据构建。利用

jMAVSim 护航相机始终注视目标机的特性，将目标框中心约束在光轴附近，再由真值距离与等效尺寸反算框高，实现零人工标注的 450 帧仿真数据采集；按距离分层覆盖 3–6 m、6–7 m、7–9 m 与 9–12 m 四个区间。以 20 轮、学习率 0.001、关闭 Mosaic 的方式微调后，3–10 m 全距离段检出率提升至 100%，置信度中位数 0.867，检测死区被完全消除。

（4）设计了三态跟踪状态机与单目定位方法。状态机包含 SEARCH、LOCKED 与 COAST 三种状态，采用“0.25 维持门限 + 0.55 初锁门限”的双门限策略抑制误锁，采用 140 px 空间门限的最近邻匹配抑制身份跳变，采用 1.0 s 卡尔曼预测维持抑制短时遮挡中断。定位环节由检测框中心直接解算方位角与俯仰角，由框高与等效尺寸解算距离，并以六维状态的自适应卡尔曼滤波（Sage-Husa）平滑三维位置与速度。仿真闭环实验表明，微调后单帧测距 RMSE 为 0.26 m、AKF 稳态中位误差 0.13 m、1 s 距离预测 ADE 中位 0.26 m、方位角误差中位 0.05°；实拍视频中 AKF 相对单帧测距 RMSE 降低 32%，抖动降低 33%。

（5）完成了系统集成与自动化验证。系统采用“算法端—显示端”双进程架构，算法进程通过 TCP 接收 720p@30fps 视频帧、通过 UDP 接收 50 Hz 真值，并以 UDP/JSON 向显示端输出结构化结果；基于 PySide6 构建三页签监视大屏，包含卫星地图态势、光电画面、距离与方位仪表、威胁分级、CSV 记录与参数持久化等功能。设计并执行了 14 个自动化用例，覆盖基线检测、真实场景、门限扫描、尺寸敏感、TCP 遮挡与摄像头输入，13 例完成、6 例判定通过、0 例失败，验证了完整链路的正确性与鲁棒性。

最后，本文将所提方法应用于改造后的 jMAVSim 城市巡游场景，实现了目标机主动飞行、护航相机被动尾随的编队巡游，并通过段内加减速与瞄准点滞后使画面呈现真实的追踪感，为后续外场试验与多传感器融合提供了可复现的实验平台。

关键词：反无人机；小目标检测；YOLOv8；P2 检测头；EMA 注意力；卡尔曼滤波；单目测距；仿真域微调

Design and Implementation of a Drone Recognition and Localization System for Counter-UAV Scenarios

ABSTRACT

With the rapid development of the low-altitude economy, micro unmanned aerial vehicles (UAVs) have been widely used in many fields such as rescue, aerial photography, logistics, and inspection. However, illegal and irregular flight events such as “black flights” and “chaotic flights” have also increased, posing threats to airports, military regions, and other confidential units. This problem has become a key issue to be solved in the counter-UAV field. Radar, acoustic, optical, and vision-based solutions have advantages such as low cost, flexible deployment, and intuitive target confirmation, but they also face two prominent problems. The first problem is that distant targets are imaged too small in images, about tens of pixels, which belongs to small-object detection. The second problem is that in real scenarios, obstacles such as trees and buildings are ubiquitous, and detection data interruptions are frequent, making it difficult to form continuous, persistent, and usable tracks.

To address the above problems, this paper designs and implements a UAV identification and localization system for counter-UAV scenarios. In this system, a monocular visible-light camera is the only visual sensor. Through the pipeline of “detection–tracking–localization–display”, the main work includes the following five aspects.

(1) A model for small-object detection is built. The model is based on YOLOv8n, adds a P2 high-resolution detection head with stride=4 to the feature pyramid, and embeds an Efficient Multi-Scale Attention (EMA) mechanism into the C2f module, forming the YOLOv8n-P2-EMA model. Then the Det-Fly dataset is divided into training and validation sets to avoid the same sequence frames appearing in both training and validation and causing data leakage. Four ablation experiments show that the mAP50 of YOLOv8n, +P2, +EMA, and +P2+EMA are 81.3%, 82.3%, 76.2%, and 83.5%, respectively; AP_small are 6.1%, 8.3%, 4.1%, and 9.8%, respectively; and the inference speeds are 123, 90, 101, and 98 fps, respectively. The proposed model achieves the best overall performance with 2.94 M parame-

ters; AP_small is improved by 60.7% relative to the baseline, and it can meet the 30 fps real-time processing requirement.

(2) The formation mechanism of the detection dead zone in the simulation environment is revealed. In PX4+jMAVSim tests, it is found that the detection rate of the model without fine-tuning is close to 0 in the near range of 3–6 m, and it cannot stably lock in the far range above 9 m; the effective working window is only 7–9 m. Through frame-by-frame audit of 2228 frames of video and ground-truth geometry, the hypothesis that "the top-down view causes failure" is ruled out, and it is confirmed that the decisive factor of the dead zone is the scale of the target in the image: near-range large-scale samples exceed the far-range small-target distribution of Det-Fly and belong to out-of-distribution (OOD) inputs; far-range samples are below the initial locking threshold.

(3) A simulation-domain fine-tuning method is proposed and automatic annotation data construction is completed. Taking advantage of the characteristic that the jMAVSim escort camera always looks at the target UAV, the center of the target box is constrained near the optical axis, and then the box height is back-calculated from the ground-truth distance and equivalent size, realizing zero-manual-annotation collection of 450 frames of simulation data; the data are stratified by distance and cover four intervals: 3–6 m, 6–7 m, 7–9 m, and 9–12 m. After fine-tuning for 20 epochs with a learning rate of 0.001 and Mosaic disabled, the detection rate over the full range of 3–10 m is increased to 100%, the median confidence is 0.867, and the detection dead zone is completely eliminated.

(4) A three-state tracking state machine and a monocular localization method are designed. The state machine contains three states: SEARCH, LOCKED, and COAST. A dual-threshold strategy of "0.25 maintenance threshold + 0.55 initial locking threshold" is used to suppress false locking; nearest-neighbor matching with a 140 px spatial threshold is used to suppress identity switching; and 1.0 s Kalman prediction maintenance is used to suppress short-term occlusion interruptions. In the localization stage, the azimuth and elevation angles are directly solved from the center of the detection box, the distance is solved from the box height and equivalent size, and a six-dimensional state adaptive Kalman filter (Sage-Husa) is used to smooth the 3D position and velocity. Simulation closed-loop experiments show

that after fine-tuning, the single-frame ranging RMSE is 0.26 m, the AKF steady-state median error is 0.13 m, the median ADE of 1 s distance prediction is 0.26 m, and the median azimuth error is 0.05°; in real captured videos, the AKF reduces the RMSE by 32% and the jitter by 33% compared with single-frame ranging.

(5) System integration and automated verification are completed. The system adopts a dual-process architecture of "algorithm side–display side". The algorithm process receives 720p@30 fps video frames via TCP and 50 Hz ground truth via UDP, and outputs structured results to the display side via UDP/JSON; a three-tab monitoring dashboard is built based on PySide6, including satellite map situation, electro-optical imagery, distance and azimuth gauges, threat grading, CSV logging, and parameter persistence. Fourteen automated test cases are designed and executed, covering baseline detection, real scenes, threshold sweeping, size sensitivity, TCP occlusion, and camera input. Thirteen cases are completed, six are judged as passed, and zero cases failed, verifying the correctness and robustness of the complete pipeline.

Finally, the proposed method is applied to a modified jMAVSim urban patrol scene, realizing a formation patrol in which the target UAV flies actively and the escort camera passively tails it. Through intra-segment acceleration/deceleration and aiming-point lag, the image presents a realistic tracking feel, providing a reproducible experimental platform for subsequent field tests and multi-sensor fusion.

Keywords: counter-UAV; small object detection; YOLOv8; P2 detection head; EMA attention; Kalman filter; monocular ranging; simulation-domain fine-tuning

目 录

摘要 I
ABSTRACT III
目录 VII
1 绪论 1
1.1 研究背景 1
1.1.1 低空经济发展与无人机规模化应用 1
1.1.2 反无人机系统的典型环节与技术路线 2
1.1.3 光学视觉方案面临的核心困难 3
1.2 研究意义 3
1.2.1 理论意义 3
1.2.2 工程意义 4
1.3 国内外研究现状 4
1.3.1 通用目标检测算法的发展 4
1.3.2 无人机小目标检测研究现状 5
1.3.3 反无人机系统研究现状 6
1.3.4 单目定位与目标跟踪研究现状 6
1.3.5 仿真验证与域适应研究现状 7
1.4 现有方案的不足与本文切入点 7
1.5 本文主要研究内容 8
1.6 论文组织结构 9
2 相关理论与数据基础 11
2.1 目标检测基础 11
2.1.1 卷积神经网络 11
2.1.2 特征金字塔与多尺度检测 12
2.1.3 检测头的预测形式与损失函数 12
2.1.4 评测指标 13
2.2 YOLOv8 网络结构 14
2.2.1 整体结构 14
2.2.2 C2f 模块 14
2.2.3 SPPF 与解耦头 14
2.3 注意力机制 15
2.3.1 从 SE 到 CBAM 15
2.3.2 EMA 高效多尺度注意力 15

2.4 卡尔曼滤波 16
2.4.1 线性高斯状态空间模型 16
2.4.2 自适应噪声估计 17
2.4.3 滤波的收敛性与可观测性 17
2.5 针孔相机模型与标定 17
2.5.1 坐标系定义 17
2.5.2 投影模型 18
2.5.3 张正友标定法 19
2.6 PX4 与 jMAVSim 仿真框架 19
2.6.1 PX4 Autopilot 与 SITL 19
2.6.2 jMAVSim 架构 20
2.6.3 离屏渲染与帧流 20
2.7 系统开发技术 20
2.7.1 PySide6 与 QtWebEngine 20
2.7.2 OpenCV 与 PyTorch 21
2.7.3 TCP 与 UDP 通信 21
2.8 数据集构建与数据工程 21
2.8.1 Det-Fly 数据集 22
2.8.2 自建 AprilTag 数据集 24
2.8.3 仿真域自动标注数据集 25
2.8.4 评测口径与可复现性 27
2.8.5 本章小结 28
3 小目标检测模型与仿真域微调 29
3.1 基础模型选型 29
3.1.1 规模选择 29
3.1.2 标准结构的局限 30
3.2 P2 小目标检测头 30
3.2.1 结构设计 30
3.2.2 检测头配置 31
3.2.3 计算量与参数分析 31
3.2.4 正负样本分配 31
3.3 EMA 注意力机制集成 32
3.3.1 插入位置 32
3.3.2 模块结构 32
3.3.3 计算开销 32

3.4 训练配置 33
3.4.1 数据集与划分 33
3.4.2 超参数 33
3.4.3 训练过程 33
3.5 消融实验 34
3.5.1 实验设置 34
3.5.2 实验结果 34
3.5.3 结果分析 34
3.5.4 混淆矩阵与 P-R 曲线 35
3.5.5 检测效果可视化 37
3.6 与非 YOLO 方法的横向讨论 38
3.7 有效工作范围与失效模式 38
3.7.1 置信度与目标尺度的关系 38
3.7.2 典型失效模式 38
3.7.3 检测死区的发现 39
3.7.4 仿真域微调 41
3.8 本章小结 43
4 三态跟踪与单目定位 45
4.1 问题描述 45
4.1.1 误检 45
4.1.2 身份跳变 46
4.1.3 遮挡中断 46
4.2 状态定义与总体流程 46
4.2.1 状态定义 46
4.2.2 总体流程 47
4.3 SEARCH 状态 47
4.3.1 双门限策略 47
4.3.2 初锁判定 48
4.3.3 先验空间约束 49
4.3.4 首锁时延 49
4.4 LOCKED 状态 49
4.4.1 候选框收集 49
4.4.2 空间门限关联 49
4.4.3 卡尔曼滤波更新 50
4.4.4 丢失判定 50
4.4.5 框尺寸的平滑 51

4.5 COAST 状态 51
4.5.1 预测步 51
4.5.2 超时与恢复 51
4.5.3 预测质量 52
4.6 算法描述与复杂度 52
4.6.1 伪代码 52
4.6.2 复杂度分析 52
4.6.3 参数汇总 52
4.7 实验验证 54
4.7.1 门限敏感性 54
4.7.2 遮挡场景 54
4.7.3 真实场景录像 55
4.8 设计讨论 55
4.8.1 为什么不用多目标跟踪框架 55
4.8.2 为什么用状态机而不是有限状态自动机的形式化方法 55
4.8.3 失效边界 56
4.9 单目定位与自适应卡尔曼滤波 56
4.9.1 坐标系与符号约定 56
4.9.2 方位角与俯仰角 57
4.9.3 距离解算 58
4.9.4 自适应卡尔曼滤波器设计 60
4.9.5 定位精度评估 62
4.9.6 误差来源与改进方向 64
4.10 本章小结 65
5 仿真闭环与系统平台 67
5.1 仿真闭环总体架构 67
5.1.1 进程与数据流 67
5.1.2 lockstep 同步 68
5.1.3 启动流程 68
5.2 jMAVSim 改造 69
5.2.1 改造需求 69
5.2.2 脚本目标机 69
5.2.3 护航相机 70
5.2.4 真值广播 70
5.2.5 离屏渲染与帧流 70
5.2.6 场景构建 71

5.3 编队巡游与尾随相机 71
5.3.1 设计目标 71
5.3.2 目标机航迹与加减速 72
5.3.3 尾随相机动力学 72
5.3.4 瞄准点滞后 73
5.3.5 验证结果 73
5.3.6 设计约束 74
5.4 系统设计与实现 74
5.4.1 需求分析 75
5.4.2 总体架构 76
5.4.3 算法端实现 77
5.4.4 显示端实现 79
5.4.5 通信协议 82
5.4.6 配置与持久化 83
5.4.7 回放与自动化测试 83
5.4.8 部署与运行维护 84
5.5 实验与结果分析 85
5.5.1 实验环境与评测口径 85
5.5.2 检测模型性能 86
5.5.3 仿真闭环定位精度 87
5.5.4 实拍视频定位精度 88
5.5.5 退化条件下的鲁棒性 89
5.5.6 14 用例自动化回归测试 91
5.5.7 运行性能 93
5.5.8 讨论 93
5.6 本章小结 95
6 总结与展望 97
6.1 全文总结 97
6.2 主要创新点 98
6.3 不足与展望 99
6.3.1 存在的不足 99
6.3.2 未来工作 99
6.4 结语 100
参考文献 101
致谢 105

独创性说明 107
关于论文使用授权的说明 109
学位论文数据集 111

1 绪论

1.1 研究背景

1.1.1 低空经济发展与无人机规模化应用

过去十年，微机电系统、飞控算法、电池与无线链路逐项成熟，多旋翼无人机于是从航拍的专业工具变成了柜台上常见的消费品。DJI、道通、零度等厂商把整机价格压到数千元，起飞重量不足250 g的微型机甚至不用登记就能买到[1]。低空里的活动随之密集起来：航拍测绘、电力巡检、农林植保、快递物流、应急救援、灯光表演，应用清单一年比一年长，城市上空逐渐长出一种新的经济形态，也就是人们常说的“低空经济”。

这个体量有多大，几组公开数字可以说明。中国民航局《2025年民航行业发展统计公报》记到2025年底，全行业注册无人机共328.7万架，比上年底增长51.0%，而2023年底还只有126.7万架；无人机拥有者注册用户237.8万个，全年累计飞行4530.29万小时，同比增长69.89%；全国已建成的低空飞行服务站46个，覆盖23个省（自治区、直辖市）[2]。同一时期，我国低空经济市场规模约1.5万亿元，民用无人机整机总产值1761亿元，同比增长约20%[3]。产业界的估算方向也差不多：全球反无人机市场2025年约44.8亿美元，2030年预计达到145.1亿美元，年复合增长率约26.5%[4]。

繁荣的另一面是治理压力。无人机的合法运行要报飞行计划、申请空域、实名登记、接受实时监管，而这类飞行器买起来太容易、操控太简单、体积又太小，“黑飞”（未经许可的飞行）、“乱飞”（违反禁飞区或高度限制的飞行）以及恶意入侵，很难靠传统手段拦住。近几年机场净空区被闯入导致航班大面积延误、军事设施与敏感区域被近距离拍摄、危险载荷飞越人群之类的事件，国内外都出过。这类事件有个共同点：起势快、处置窗口短，等值班人员反应过来，目标往往已经飞出视野，事后追查又缺少可用的影像证据。2025年以来中央空管办组织的专项整治中，天津、重庆、沈阳、成都等地都查处过“黑飞”扰航案件，涉事人员被行政拘留；有个别放飞高度突破民航巡航层的案子，已经按以

危险方法危害公共安全罪立案[5-6]。低空安全从纸面风险变成了现实威胁，反无人机（Counter-UAV，C-UAV）技术也随之进入学术界、产业界与监管部门的视野。

1.1.2 反无人机系统的典型环节与技术路线

一套完整的反无人机系统，一般拆成“探测—识别—跟踪—定位—处置”五段[7-8]。探测管的是空域里有没有可疑目标；识别判断它到底属于哪一类（无人机、鸟类、风筝还是飞机）；跟踪要在时间轴上把同一个目标串起来；定位给出目标相对观测平台或地理坐标系的位置；处置则按威胁等级选择告警、干扰、诱骗或者拦截。前几个环节——尤其是探测、跟踪与定位——撑起了整套系统的信息底座，它们做不好，后面的处置就无从谈起。

按所用物理媒介划分，现有探测手段大致有雷达、射频、声学、光学和多传感器融合五类，表 1-1 将它们放在一起对照。

表 1-1: 典型反无人机探测手段对比

手段	作用距离	优势	局限	适用场景
雷达	数百米至数公里	全天候、测距测速直接	小型目标雷达散射截面积小、设备昂贵、需频率许可	机场、边境、要地远程预警
射频侦测	数百米至数公里	被动工作、可识别型号	目标静默时失效、无法定位非合作目标	城市频谱监测
声学探测	数十米至数百米	成本低、可被动布设	易受环境噪声干扰、作用距离短	楼宇近程补盲
光学视觉	数十米至数百米	成本低、部署灵活、可直观确认	受光照与天气影响、小目标检出难、遮挡敏感	近程确认、跟踪与取证
多传感器融合	视配置而定	互补性强、可靠性高	系统复杂、时空高配准与数据融合难度大	等级防护系统

从表里能看出各自的长短。雷达看得远，适合预警，但低慢小目标的回波太弱，设备价格与频率许可门槛都不低[7]；射频侦测被动工作，目标一旦静默便失去线索[9]；声学方案成本低，作用距离却往往只有百米量级，城市噪声升高后虚警率难以控制[10]；多传感器融合性能最好，代价是异构传感器之间的时空配准、数据关联与决策融合，每一项都是

工程难题[8,11]。光学视觉只需要一台普通可见光相机加一台计算设备，便宜、好部署，还能留下目标的外观图像供操作员复核。近程反无人机系统里，这一路很难被替代[12]。

1.1.3 光学视觉方案面临的核心困难

低成本方案也有自身的局限。真实的反无人机场景中，光学视觉至少要应对两处困难。

第一处困难来自小目标。反无人机讲究“早发现”，可发现得越早，目标越远，成像越小。1280×720 的画面中，100 m 外的微型无人机常常只占十几个像素，换上大倍率长焦镜头也未必超过 50 像素。通用检测模型多在 COCO 这类以中大型目标为主的数据集上训练，特征金字塔最深层是 32 倍下采样，一个十几像素的目标落到那层特征图上还不到一个像素，响应几乎为零。Det-Fly 等无人机检测数据集的统计也指向同一件事：目标框面积中位数不到画面面积的 0.06%，99% 以上的框小于画面的 1%。既要不漏，又要跟得上帧率，这是检测环节的固有矛盾。

第二处困难来自遮挡。低空里树木、建筑、电线、广告牌随处可见，目标在飞行中频繁被短时遮挡。如果系统把“每帧检测成功”当成输出前提，遮挡一发生，输出立刻消失，航迹断裂、告警闪烁，操作员分不清目标是被暂时挡住还是已经飞离。检测框本身还有像素级抖动，直接拿框高反算距离，米级跳变很常见。检测缺失时如何维持航迹，检测存在时如何平滑输出，这是跟踪与定位环节要回答的问题。

还有第三处困难，出现在工程落地阶段：仿真与真实之间的域差异。受成本与空域限制，算法通常先在仿真里跑通闭环，再搬到真机。可仿真渲染的光照、纹理与边缘与真实相机存在差异，在真实数据上训练出的模型，到了仿真画面里可能出现大面积失效。本文在实验初期即观察到这一现象，后来把它单独立成一项研究内容。

1.2 研究意义

1.2.1 理论意义

现有工作大多从“增加检测层”“引入注意力”“改进损失函数”入手提升指标，至于改进为什么有效、在什么条件下失效，往往缺乏一

致解释。本文用四组消融实验把 P2 检测头的作用落到“保留浅层空间分辨率”上，把 EMA 的作用落到“在通道与空间两个维度重加权”上；+EMA 单独使用时并不奏效、+P2+EMA 联合才见效这一现象，说明二者是“看清”与“认准”的分工。这个解释对小目标检测的网络设计有一般性的参考价值。

仿真域检测死区的机理分析，方法论上的意义更大。模型在仿真中失效，最容易想到的解释是“视角不合适”或者“渲染质量差”。本文没有停在猜测上，而是用真值几何审计与距离分层统计把视角假设排除掉，将原因锁定在目标尺度分布偏移，再以仿真域微调实验回证。这种“现象—假设—证伪—验证”的分析路径，比单纯报告一个指标更能说明域偏移究竟发生在哪里。

三态状态机、单目测距与自适应滤波被整合为一条可运行的完整链路，检测、跟踪与定位的耦合性能在统一时间基准下评估。对反无人机视觉系统来说，这是一套可以照着复现的工程范式。

1.2.2 工程意义

工程上的价值更直接。整套系统只依赖一台单目可见光相机和一台带 GPU 的普通计算机，激光雷达、毫米波雷达、高精度惯导都不需要，园区、楼宇、临时活动现场均可快速部署。输出也不止一个检测框：距离、方位角、俯仰角、速度、威胁等级以及地理坐标一并给出，可以直接接进上级指挥系统。为了让它经得起反复修改，本文设计了 14 个自动化用例与统一的 CSV 记录格式，可在无人值守条件下完成回归测试，每次迭代都有量化基线可对。更换硬件或场地，只要配置照旧，系统行为就应当一致。仿真环境、数据采集脚本、自动标注流程与训练配置均已固化，研究者和工程师按同样的步骤就能复现文中的全部实验。

1.3 国内外研究现状

1.3.1 通用目标检测算法的发展

目标检测的演进大体可以分成三段。最早是手工特征加分类器的组合，HOG+SVM 与可变形部件模型（DPM）是代表，精度受制于人工设计的特征，换个场景便未必适用。深度学习进来以后，卷积网络成了

特征提取器，路线分叉为两阶段与单阶段：两阶段以 Faster R-CNN[13] 为代表，先由区域建议网络（RPN）给出候选框，再逐个分类回归，精度较高但速度较慢；单阶段以 SSD[14] 与 YOLO 系列[15-17] 为代表，直接在特征图上预测类别与边框，速度优势明显。再往后是 Transformer 架构，RT-DETR[18] 用编码器—解码器结构与集合预测损失摆脱了锚框与非极大值抑制，精度追平甚至超过同期 YOLO，代价是计算量与部署复杂度都上去了。

落地到工程里，YOLO 系列靠着“精度—速度—易用性”的平衡占据了主流地位。YOLOv5[16] 定下了 CSPDarknet 骨干、PANet 颈部、解耦头部的格局；YOLOv8[17] 换上 C2f 模块与更灵活的任务对齐分配策略，并给出 n/s/m/l/x 五档规模。本文选 YOLOv8n 作基础模型，考虑有三：参数量约 3 M，便于与桌面应用集成；在 RTX 4060 级别的 GPU 上批量推理可达 100 fps 以上；Ultralytics 的训练、验证与导出接口成熟，便于复现实验。

1.3.2 无人机小目标检测研究现状

围绕无人机小目标，近年的工作大致沿四条路走。

一条路是增加高分辨率检测层。标准 YOLO 最浅的检测层为 P3（stride=8），对付十几个像素的目标仍然吃力。SOD-YOLOv8[19] 在 YOLOv8 里加了更高分辨率的检测分支，配合特征融合，把交通场景中小目标的召回率提了上去。本文把这套思路搬到无人机场景，增加 stride=4 的 P2 检测头，40 px 的目标在特征图上因此占据 10 个网格单元，而不是 5 个。

另一条路是注意力机制。CBAM[20] 把通道注意力与空间注意力串联起来，强化目标区域的响应；EMA[21] 采用分组并行与跨空间学习，在较低计算开销下同时建模通道与空间关系。YOLOv8-MPEB[22] 将骨干替换为 MobileNetV3 并引入高效多尺度注意力，做轻量化无人机检测。MSDF-Net[23] 结合可变形卷积与高分辨率特征增强，在林业病害检测中验证了跨版本特征增强确实有效。

也有研究把力气花在损失函数与标签分配上。小目标对定位误差格外敏感，IoU 系列损失在目标框很小时梯度不稳定。SIoU[24] 引入角度代价，让回归方向更明确；Wise-IoU 通过动态聚焦机制压低低质量样本的梯度权重。本文没有在这条路上继续加码，训练沿用 YOLOv8 默认的 TaskAlignedAssigner 与 CIoU 损失，为的是与基线模型保持可比。

剩下一条路是数据集与评测协议。Det-Fly[25] 是目前公开的较大规模无人机检测数据集之一，背景、机型与飞行状态都比较丰富。UAVDT[26] 面向城市交通场景中的无人机跟踪。Kurmashev 等人[27] 系统比较了不同 YOLO 版本在无人机检测任务上的表现，结论是模型规模、输入分辨率与数据分布对结果的影响，远大于网络结构上的细微差别。这提醒我们，报告指标时必须把数据划分与评测口径写清楚，否则结论很容易被误读。

1.3.3 反无人机系统研究现状

整套系统的研究则集中在三块。

一块是多传感器融合。文献[11] 提出射频、光学与雷达的多模态框架，用 Remote ID 广播信号做身份识别，用视觉做确认，用雷达测距；文献[8] 对现有反无人机方法与基准做了系统综述，指出任何单一传感器在复杂环境下都有盲区，融合几乎是提高可靠性的必由之路。

另一块是射频与声学探测。文献[9] 借助无人机图传与遥控链路的射频特征做检测与跟踪，能在目标静默之前给出较远距离的预警；文献[10] 用 MEMS 麦克风阵列配合机器学习做被动声学定位，成本极低，作用距离却受限。

视觉检测与跟踪也是一块。文献[12] 比较了 YOLO 与 Faster R-CNN 在监控视频中检测微型无人机的表现，单阶段模型速度占优，两阶段模型在极小目标上略好一些。把这些工作放在一起看，多数仍聚焦于单个算法环节的指标，面向完整链路、能实际跑起来、还带可视化界面的系统级开源实现并不多见，这正是本文想补的位置。从工程角度看，一条能跑通的链路，往往比多零点几个点的指标更能说明问题。

1.3.4 单目定位与目标跟踪研究现状

单目视觉测距常见三条路。一条靠已知目标尺寸，用相似三角形从框高或框宽反算距离，实现简单，但对尺寸先验依赖很重；一条假设目标位于地面，借相机高度与像平面几何关系解算落点，只适用于地面目标；还有一条靠多帧运动，由特征点跟踪与三角化恢复深度，计算量大且吃纹理。本文要测的是空中的无人机，地面平面约束用不上，外观尺寸又相对稳定，所以走第一条路，并通过仿真域标定拿到等效尺寸。

跟踪方面，SORT[28]用卡尔曼滤波预测目标位置、用匈牙利算法关联检测框，多目标跟踪的基本框架由此奠定；DeepSORT[29]加入外观特征，把身份切换压了下去。本文做的是单目标跟踪，不需要外观重识别，但“预测—关联—更新”的思路照搬了过来，并把关联失败时的处理显式建成 COAST 状态。卡尔曼滤波[30]是最优线性估计的经典解法，前提是量测噪声协方差已知。Sage-Husa 自适应滤波[31]用残差在线估计噪声统计特性，量测质量一变，滤波增益就跟着调整。本文沿着这个思路搭了自适应卡尔曼滤波器（AKF）。

1.3.5 仿真验证与域适应研究现状

空域管制与成本摆在那里，无人机算法的早期验证基本离不开仿真。PX4[32]是用得最广的开源飞控固件，支持 SITL（Software In The Loop）模式；jMAVSim[33]是它自带的轻量级三维仿真器，基于 Java3D 构建，资源开销小，能模拟多旋翼动力学与相机视图。Gazebo 与 AirSim 渲染更逼真，对显卡与算力的要求也更高。本文选 jMAVSim，看重的是启动快、可改性好——改 Java 源码就能注入目标机、相机与真值广播，这一点比渲染质量更关键。

仿真到真实的域差异（sim-to-real gap）这几年讨论很多，常见做法有风格迁移、域随机化和目标域微调。风格迁移用生成对抗网络把仿真图像染成真实风格，伪影是常被诟病的问题；域随机化在训练时随机改变光照、纹理与噪声，鲁棒性上去了，训练资源也上去了；目标域微调直接拿少量目标域数据调整模型参数，简单，往往也最有效。本文走的是微调这条路，并针对仿真环境设计了零人工标注的采集流程，用较低成本把检测死区消掉。

1.4 现有方案的不足与本文切入点

把上面的分析收一收，现有方案大致有四处欠缺。

（1）**小目标改进缺乏机理层面的解释**。多数工作只报告 mAP 涨了多少，不说改进在哪个尺度、哪类背景下起作用，也很少讨论单独上一个模块会不会反而拖后腿。本文用消融实验与特征尺度分析，给出 P2 与 EMA 互补的解释。

（2）**仿真验证中的性能退化被忽视**。不少工作在仿真环境里直接套

用真实数据训出的模型，性能一掉就归因于“渲染不真实”，定量分析欠奉。本文逐帧审计真值来定位死区成因，再用微调实验闭环验证。

（3）**跟踪与定位耦合不足**。检测、跟踪、测距常常各评各的，缺少统一时间基准下的端到端指标。本文在仿真闭环中以 50 Hz 真值为基准，把检出率、测距误差、角度误差与预测误差放在一起看。

（4）**系统级实现与可复现性不足**。许多研究停在脚本与 notebooks 层面，界面、参数管理、日志记录与自动化测试都缺。这些细节在论文里往往看不出来，真到现场部署时却处处掣肘。本文把桌面系统与测试框架完整做出来，并将全部配置固化。

基于以上分析，本文的切入点可以概括为：**以单目可见光相机为传感器，以改进的 YOLOv8n 为检测前端，以三态状态机为跟踪骨架，以自适应卡尔曼滤波为定位核心，以仿真闭环为验证平台，构建一套端到端可运行、可测试、可复现的反无人机识别与定位系统**。

1.5 本文主要研究内容

围绕上述不足，本文的工作分五个方面展开，前后几项互为输入，最终汇到同一套系统里。

（1）**小目标检测模型改进与消融分析**。在 YOLOv8n 上叠加 P2 检测头与 EMA 注意力，组合出四种模型配置，按序列划分 Det-Fly 的训练/验证集，跑完消融实验，再从特征分辨率与注意力重加权两个角度解释结果。

（2）**数据集构建与数据工程**。完成 Det-Fly 的格式转换、序列级划分与统计分析；采集自建 AprilTag 标定数据，用于相机内参标定与定位精度评估；搭建 450 帧仿真自动标注数据集，供仿真域微调使用。

（3）**三态跟踪状态机设计**。围绕误检、跳变与遮挡三个问题，设计 SEARCH/LOCKED/COAST 状态机，交代双门限、空间门限与超时参数的选取依据，并用遮挡用例检验 COAST 的航迹保持能力。

（4）**单目定位与自适应滤波**。建立针孔相机模型，推导方位角、俯仰角与距离的解算公式，分析测距误差如何随距离与框高误差传播；设计六维状态自适应卡尔曼滤波器，给出预测步与更新步的完整推导，再用仿真与实拍数据评估平滑与预测性能。

（5）**仿真闭环与系统实现**。改造 PX4+jMAVSim 闭环，注入脚本目

标机、护航相机、真值广播与离屏帧流；查明并消除检测死区；基于PySide6实现三页签监视大屏；以14个自动化用例完成回归测试；进一步做出目标机主动飞行、相机被动尾随的编队巡游与加减速行为。

1.6 论文组织结构

本文共分六章，各章内容安排如下。

第一章为绪论。说明研究背景与反无人机系统的技术路线，指出光学视觉方案面临的小目标检测与遮挡中断两处困难，综述国内外研究现状，归纳现有方案的不足，并给出本文的研究内容与组织结构。

第二章为相关理论与数据基础。前半部分介绍卷积神经网络、特征金字塔、YOLOv8网络结构、EMA注意力、卡尔曼滤波、针孔相机模型、PX4与jMAVSim仿真框架以及PySide6与网络通信；后半部分交代Det-Fly数据集的来源、格式与序列级划分，自建AprilTag标定数据的采集流程，450帧仿真自动标注数据的构建方法，以及数据增强与评测口径。

第三章为小目标检测模型与仿真域微调。阐述P2检测头与EMA注意力的设计，给出结构、计算量与正负样本分配的分析；介绍训练配置与四组消融实验，从尺度分布、注意力响应与参数效率解释结果；随后分析仿真闭环中检测死区的发现过程与尺度成因，给出零人工标注的微调数据采集、分层采样、微调配置与前后对比，并讨论模型的有效工作范围与失效模式。

第四章为三态跟踪与单目定位。针对误检、身份跳变与遮挡设计SEARCH、LOCKED、COAST状态机，说明双门限、空间门限与超时参数的选择依据，给出伪代码、复杂度与遮挡场景验证；在此基础上建立相机坐标系与像素坐标系的映射，推导方位角、俯仰角与距离的解算公式及误差传播，设计六维状态的自适应卡尔曼滤波器，并用仿真闭环与实拍视频评估定位与预测精度。

第五章为仿真闭环与系统平台。介绍PX4 SITL与jMAVSim的组合架构、lockstep同步与启动流程；说明对jMAVSim的改造，包括脚本目标机、护航相机、真值广播、离屏帧流、城市场景与编队巡游；随后给出系统的需求分析、双进程架构、算法端与显示端的实现、部署运维；最后汇总检测、定位、退化鲁棒性、自动化回归与运行性能等实验结果，

并讨论能力边界与失败案例。

第六章为总结与展望。总结全文工作与创新点，分析存在的不足，展望后续研究方向。

2 相关理论与数据基础

一套视觉反无人机系统要从论文走到能运行的代码，所涉及的内容可分为两类：一类回答“为什么这样做”，是方法与模型的理论依据；另一类回答“拿什么来训练和验证”，是数据及其工程处理。本章把这两样放在一处交代，读起来会更连贯。理论上，本文只挑与后续设计直接相关的那几支：卷积网络与特征金字塔、YOLOv8 的骨干与头部、EMA 注意力、卡尔曼滤波与 Sage-Husa 噪声估计、针孔成像与张正友标定，再往外延伸到 PX4、jMAVSim 这套仿真工具链，以及 PySide6、TCP/UDP 这些实现手段；数据上则是 Det-Fly、自建 AprilTag 与 450 帧仿真自动标注三套材料，连同划分、增强与评测口径。这样安排，是为了让后面每一处设计选择都有出处可查，而非凭空而来。

2.1 目标检测基础

2.1.1 卷积神经网络

卷积网络之所以能替代手工特征，靠的是三个很朴素的约束：局部连接、权值共享与空间池化。它们把图像的平移不变性先验写进了结构里，特征因此可以一层层从边缘、纹理堆到部件与整体。设第 l 层的输入特征图为 $\mathbf{X}^{l-1} \in \mathbb{R}^{H_{l-1}\times W_{l-1}\times C_{l-1}}$，卷积核为 $\mathbf{W}^{l} \in \mathbb{R}^{k\times k\times C_{l-1}\times C_{l}}$，则第 l 层的输出为

$$\mathbf{X}^{l}_{i,j,c} = \sigma\left(\sum_{u=0}^{k-1}\sum_{v=0}^{k-1}\sum_{c'=0}^{C_{l-1}-1} \mathbf{W}^{l}_{u,v,c',c}\,\mathbf{X}^{l-1}_{i+u,\,j+v,\,c'} + b^{l}_{c}\right), \tag{2-1}$$

其中 k 为卷积核尺寸，b^{l}_{c} 为偏置，$\sigma(\cdot)$ 为非线性激活函数。本文所依托的检测网络一律采用 SiLU（Swish），而不是更常见的 ReLU：

$$\mathrm{SiLU}(x) = x\cdot\mathrm{sigmoid}(x) = \frac{x}{1+e^{-x}}. \tag{2-2}$$

选它的理由在负半轴。ReLU 在 $x<0$ 处梯度恒为零，小目标本来响应就弱，一旦落进去就再也拉不回来；SiLU 在负半轴留了一小段非零梯度，小目标即便暂时落进负区，也还有机会被拉回来。这一点在后面的小目标实验里体现得很直接。

批归一化（Batch Normalization，BN）按小批量统计量对每个通道做标准化，各层输入分布因此稳定下来，学习率可以放大，收敛也快：

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta. \tag{2-3}$$

式中 $\mu_{\mathcal{B}}$ 与 $\sigma_{\mathcal{B}}^2$ 为当前批次的均值与方差，γ、β 为可学习的缩放与平移，ϵ 用来防止除零。

2.1.2 特征金字塔与多尺度检测

目标的尺度跨度是检测器最头疼的事：同一张图里可能既有几十像素的飞机，也有上千像素的近景目标，单一尺度的特征图顾此失彼。FPN[34] 的办法是用自顶向下的通路把深层语义送到浅层，让每一层都兼有语义与分辨率；PANet[35] 再补一条自底向上的通路，缩短浅层特征到检测头的距离。本文的颈部直接沿用 YOLOv8 的 PAN-FPN，没有另起炉灶——原因在 P2 一节还会提到：多接一条高分辨率支路，比重新设计整个颈部划算得多。

设骨干网络输出的特征图为 C_3, C_4, C_5，分别对应 stride 8、16、32；FPN 的融合过程可写为

$$P_l = \mathrm{Conv}\left(\mathrm{Upsample}(P_{l+1}) \oplus \mathrm{Conv}(C_l)\right), \tag{2-4}$$

其中 $\oplus$ 表示逐元素相加，Conv 通常为 1×1 卷积（用于通道对齐）加 3×3 卷积（用于消除上采样混叠）。PAN 路径则从 P_3 向 P_5 做下采样融合。最终检测头分别接在 P_3, P_4, P_5 上：下采样倍数越大，感受野越大，越适合大目标；倍数越小，空间分辨率越高，越适合小目标。也正因为这条规律，想再往下够一层，就只能自己补一个 P2。

2.1.3 检测头的预测形式与损失函数

YOLOv8 的头部不带锚框，每个网格位置直接预测目标中心相对网格的偏移量、宽高以及类别概率，锚框聚类与按 IoU 匹配正负样本那一套都省掉了。分类分支使用二元交叉熵（BCE）损失：

$$L_{cls} = -\frac{1}{N_{pos}} \sum_{i} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \tag{2-5}$$

式中 N_{pos} 为正样本数，y_i 为类别标签，p_i 为预测概率；除以 N_{pos} 是为了让不同批量、不同目标数下的损失量级可比。

边框回归这边，YOLOv8 把分布焦点损失（Distribution Focal Loss，DFL）与 CIoU 损失拼在一起用。DFL 将边框回归建模为离散概率分布，网络学到的就不只是一个数，还有边界的不确定程度：

$$L_{DFL} = -\sum_{i} \left((y_{i+1} - y) \log p_i + (y - y_i) \log p_{i+1} \right). \tag{2-6}$$

CIoU 则同时考虑重叠面积、中心点距离与宽高比一致性：

$$L_{CIoU} = 1 - \mathrm{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v, \tag{2-7}$$

其中 $\rho(\cdot)$ 为欧氏距离，c 为最小外接矩形对角线长度，v 衡量宽高比的一致性，α 为权重系数。本文没有动这套损失，目的仍是与基线保持可比。

2.1.4 评测指标

检测结果好不好，需要一套大家都认的尺子。最基础的是交并比（IoU），即预测框与真值框的交集面积除以并集面积：

$$\mathrm{IoU} = \frac{|\mathcal{B}_{pred} \cap \mathcal{B}_{gt}|}{|\mathcal{B}_{pred} \cup \mathcal{B}_{gt}|}. \tag{2-8}$$

在某个 IoU 阈值下把预测按置信度降序排开，逐点计算 Precision 与 Recall，可得 P-R 曲线，曲线下面积就是该类别在该阈值下的平均精度 AP；对所有类别再取平均，便是 mAP。COCO 评测协议把 IoU 从 0.50 到 0.95 按步长 0.05 切成十档取平均，记作 mAP50-95；mAP50 只看 IoU=0.50 这一档。

COCO 还按目标框面积把目标分成小、中、大三档：面积小于 32^2 像素为 small，介于 32^2 与 96^2 之间为 medium，大于 96^2 为 large。反无人机任务里，small 子集上的 AP（本文记作 AP_small）最值得看，因为远距离目标大多落在这个区间。这里有个容易混淆的地方：框面积与目标的物理尺寸并不是一回事，同一架无人机飞远飞近，会在三个桶之间来回跳，所以 AP_small 量的是“模型在多远处仍能检出”，而不是“模型认不认得出小东西”。

速度指标通常以 FPS（Frames Per Second）衡量。不同实现的预处理、后处理与显存拷贝开销差异很大，不注明条件的 FPS 没有比较意义。本文统一采用 batch=1、帧预载内存、包含预处理与后处理的口径，保证模型之间横向可比。

2.2 YOLOv8 网络结构

2.2.1 整体结构

YOLOv8 由骨干（Backbone）、颈部（Neck）与头部（Head）三部分组成。骨干是改进的 CSPDarknet，靠 C2f 模块在保持梯度流动的同时把计算量压下来；颈部是 PAN-FPN，做多尺度特征的双向融合；头部走解耦路线，分类与回归各用一条分支，免得两个任务的特征相互干扰。YOLOv8n 是五种规模里最小的一种，参数量约 3.0 M，在 640×640 输入下的计算量约 8.7 GFLOPs。

2.2.2 C2f 模块

C2f（CSP Bottleneck with 2 convolutions and f fused outputs）模块是 YOLOv8 的核心组件，结构可以这么理解：输入先用 1×1 卷积降维，随后分成两路，一路直接短路，另一路串若干 Bottleneck 后逐级拼接，再由 1×1 卷积融合输出。与 YOLOv5 的 C3 模块相比，C2f 留出的梯度流分支更多，浅层特征能更直接地送到检测头——对小目标来说，这一点比省下那点算量更重要。

设输入为 $\mathbf{X}$，C2f 的输出可写为

$$\mathbf{Y}=\mathrm{Conv}_{1\times1}\left(\mathrm{Concat}\left[\mathbf{X}_0,\mathcal{B}_1(\mathbf{X}_1),\mathcal{B}_2(\mathbf{X}_2),\ldots,\mathcal{B}_n(\mathbf{X}_n)\right]\right), \tag{2-9}$$

其中 $\mathcal{B}_i$ 表示第 i 个 Bottleneck，$\mathbf{X}_i$ 为前一层输出。每个 Bottleneck 的输出都参与最终拼接，网络实际得到的是多种感受野的组合，而不是单一尺度的特征。

2.2.3 SPPF 与解耦头

SPPF（Spatial Pyramid Pooling - Fast）把几个不同尺寸的最大池化层串联起来，感受野扩大了，计算量却没怎么涨，模型对目标整体结构的

感知因此更强。解耦头把分类与回归分到两条独立的卷积分支上：分类支输出 C 个类别的概率，回归支输出 4 个边框参数与 16 个分布参数。两件事分开做，任务间的负迁移少了，这也是 YOLOv8 相对前代比较实在的一处改动。

2.3 注意力机制

2.3.1 从 SE 到 CBAM

注意力模块的家族不算小，本文只取其中一支，这里把来龙去脉交代清楚。SE 模块先用全局平均池化拿到通道描述子，再经两层全连接网络生成通道权重：

$$\mathbf{s} = \sigma\left(\mathbf{W}_2\,\delta\left(\mathbf{W}_1\,\mathrm{GAP}(\mathbf{X})\right)\right), \qquad \mathbf{X}' = \mathbf{s} \odot \mathbf{X}. \tag{2-10}$$

CBAM[20] 在通道注意力之后又串联了一级空间注意力，空间分支对通道维度做最大池化与平均池化，再经 7×7 卷积生成空间权重图。SE 与 CBAM 的思路都是“先压缩、再激励”，但两者都只在一个维度上建模，空间分支的感受野也有限。本文没有采用 CBAM，倒不是它无效，而是其通道与空间两级串联在 P2 这种高分辨率特征图上开销偏大，且两个维度分开计算，对细窄条带目标的响应不够集中；相比之下，EMA 在这一点上更合算。

2.3.2 EMA 高效多尺度注意力

EMA（Efficient Multi-scale Attention）[21] 的着眼点是**分组并行 + 跨空间学习**。设输入特征图为 $\mathbf{X} \in \mathbb{R}^{C\times H\times W}$，EMA 先沿通道维度把它切成 G 组，每组各自走一遍下面的流程：

（1）组内特征分别沿水平与垂直方向做一维池化，得到两组带方向信息的描述子；

（2）对描述子做一维卷积，建模局部的跨通道交互；

（3）两组描述子经 sigmoid 归一化后与原始特征逐元素相乘，得到方向注意力增强后的特征；

（4）对增强特征做全局平均池化，再通过 softmax 生成跨空间的注意力权重；

（5）两组权重与原始输入相乘后相加，输出最终特征。

与 CBAM 摆在一起看，EMA 省在两个地方。分组并行把大通道数下的计算量摊薄了；一维方向卷积用极小的参数量就建起了长程依赖；跨空间学习又让组与组之间能够交换信息，分组带来的特征割裂被补了回来。小目标在复杂背景里常常只是一条细窄的响应带，这种“方向感知 + 跨组交互”恰好对路。

2.4 卡尔曼滤波

2.4.1 线性高斯状态空间模型

卡尔曼滤波[30] 解决的是线性高斯系统的最优状态估计问题。设离散时间系统的状态方程与量测方程为

$$\mathbf{x}_k = \mathbf{F}_k\mathbf{x}_{k-1} + \mathbf{B}_k\mathbf{u}_k + \mathbf{w}_k, \qquad \mathbf{z}_k = \mathbf{H}_k\mathbf{x}_k + \mathbf{v}_k, \tag{2-11}$$

其中 $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$ 为过程噪声，$\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k)$ 为量测噪声，二者相互独立。整个滤波只有两拍：预测与更新。

预测步：

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k\hat{\mathbf{x}}_{k-1|k-1}, \tag{2-12}$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k\mathbf{P}_{k-1|k-1}\mathbf{F}_k^T + \mathbf{Q}_k. \tag{2-13}$$

更新步：

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}_k^T\left(\mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^T + \mathbf{R}_k\right)^{-1}, \tag{2-14}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k\left(\mathbf{z}_k - \mathbf{H}_k\hat{\mathbf{x}}_{k|k-1}\right), \tag{2-15}$$

$$\mathbf{P}_{k|k} = \left(\mathbf{I} - \mathbf{K}_k\mathbf{H}_k\right)\mathbf{P}_{k|k-1}. \tag{2-16}$$

增益矩阵 $\mathbf{K}_k$ 说的是“预测与量测之间该信谁”：量测噪声 $\mathbf{R}$ 大，增益就小，滤波器偏向预测；预测不确定度 $\mathbf{P}$ 大，增益就大，滤波器偏向量测。后面自适应滤波要动的，正是这个 $\mathbf{R}$。

2.4.2 自适应噪声估计

标准卡尔曼滤波要求 **Q** 与 **R** 已知且恒定。视觉跟踪里这条假设很难成立：检测框抖多少，跟着目标距离、光照与背景一起变，固定的 **R** 顾此失彼。Sage-Husa 自适应滤波[31] 利用新息（innovation）序列在线估计噪声统计特性。新息定义为

$$\boldsymbol{\nu}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1}. \tag{2-17}$$

若噪声统计与假设一致，新息应为零均值白噪声，其协方差为

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k. \tag{2-18}$$

对新息协方差做滑动平均，可以反解出 $\mathbf{R}_k$ 的估计：

$$\hat{\mathbf{R}}_k = \frac{1}{M} \sum_{i=k-M+1}^{k} \left(\boldsymbol{\nu}_i \boldsymbol{\nu}_i^T - \mathbf{H}_i \mathbf{P}_{i|i-1} \mathbf{H}_i^T \right), \tag{2-19}$$

其中 M 为滑动窗口长度。实际实现中还得给 $\hat{\mathbf{R}}_k$ 加下界约束并做对角化处理，否则数值容易发散。本文的具体实现放在第四章。

2.4.3 滤波的收敛性与可观测性

卡尔曼滤波的估计误差协方差，在系统一致可观测的条件下才会收敛。本文的状态是六维（位置与速度），量测只给位置，速度只能通过位置差分间接估计。目标长时间匀速直线运动时，速度的可观测性就弱，滤波收敛慢；一旦出现机动，位置变化反而提供了额外的速度信息。工程上常用的办法是把过程噪声 **Q** 中的速度项调大，收敛快一些，代价是稳态抖动变大。本文在参数整定时对两者取折中，没有一味求快。

2.5 针孔相机模型与标定

2.5.1 坐标系定义

本文涉及四个坐标系：世界坐标系、相机坐标系、图像坐标系与像素坐标系。世界坐标系采用北—东—地（North-East-Down，NED）约定，x 轴指向北，y 轴指向东，z 轴指向地心，地面上 $z = 0$，高度越大 z 越

小。相机坐标系以光心为原点，z_c 轴沿光轴指向景物，x_c 轴向右，y_c 轴向下。图像坐标系以主点为原点、以物理尺寸为单位；像素坐标系以图像左上角为原点、以像素为单位。四者之间的换算关系必须一次定死，后面测距与角度解算全靠它。

设世界坐标系到相机坐标系的旋转矩阵为 **R**、平移向量为 **t**，则点 P_w 在相机坐标系下的坐标为

$$\mathbf{P}_c = \mathbf{R}\,\mathbf{P}_w + \mathbf{t}. \tag{2-20}$$

2.5.2 投影模型

针孔模型先把三维点投影到归一化像平面，再经内参矩阵映射到像素坐标：

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}\begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \qquad \mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{2-21}$$

其中 $s = Z_c$ 为深度，f_x, f_y 为以像素为单位的焦距，(c_x, c_y) 为主点。若只考虑相机坐标系下的点，式子可以简化成

$$u = f_x\frac{X_c}{Z_c} + c_x, \qquad v = f_y\frac{Y_c}{Z_c} + c_y. \tag{2-22}$$

实际镜头存在径向畸变与切向畸变，常用的 Brown-Conrady 模型为

$$\begin{aligned} x_d &= x(1 + k_1r^2 + k_2r^4 + k_3r^6) + 2p_1xy + p_2(r^2 + 2x^2), \\ y_d &= y(1 + k_1r^2 + k_2r^4 + k_3r^6) + p_1(r^2 + 2y^2) + 2p_2xy, \end{aligned} \tag{2-23}$$

其中 (x, y) 为归一化无畸变坐标，$r^2 = x^2 + y^2$，k_1, k_2, k_3 为径向畸变系数，p_1, p_2 为切向畸变系数。本文的仿真相机不带畸变，实拍相机则在标定后按上述模型去畸变——这一步省不得，否则画面边角处的目标会被系统性地拉偏。

2.5.3 张正友标定法

内参从哪里来？棋盘格标定是成本最低的一条路，本文用的是张正友标定法[36]：拿平面棋盘格在不同位姿下拍若干幅图像，靠单应性矩阵的约束解内参。设棋盘格平面为 $Z_w = 0$，则式 (2-21) 退化为单应变换：

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix}. \tag{2-24}$$

利用旋转向量正交约束 $\mathbf{r}_1^T \mathbf{r}_2 = 0$、$\|\mathbf{r}_1\| = \|\mathbf{r}_2\| = 1$，可以构造出关于 $\mathbf{K}$ 的线性方程组，至少三幅不同位姿的图像即可求解。随后用最大似然估计优化重投影误差：

$$\min_{\mathbf{K},\mathbf{k},\{\mathbf{R}_i,\mathbf{t}_i\}} \sum_i \sum_j \left\| \mathbf{p}_{ij} - \hat{\mathbf{p}}(\mathbf{K}, \mathbf{k}, \mathbf{R}_i, \mathbf{t}_i, \mathbf{P}_j) \right\|^2. \tag{2-25}$$

本文在自建数据采集环节使用 9×6、方格边长 25 mm 的棋盘格完成内参标定，标定结果直接用于评估单目定位的绝对精度。

2.6 PX4 与 jMAVSim 仿真框架

2.6.1 PX4 Autopilot 与 SITL

PX4[32] 是一款开源飞控固件，多旋翼、固定翼、垂直起降都撑得住。其软件在环（SITL）模式把飞控固件编译成可在普通 PC 上运行的进程，通过 MAVLink[37] 协议与仿真器交换传感器数据与执行器指令。选 SITL 的理由很直接：不用真实飞控硬件就能验证控制逻辑、航点任务与失效保护；可以跟 jMAVSim、Gazebo、AirSim 等多种仿真器组合；支持 lockstep 模式，仿真时间与飞控时间严格同步，实验才谈得上可复现。

在 lockstep 模式下，仿真器每推进一步都要等待飞控完成相应的控制计算，二者按同一个仿真时间前进。“仿真 60 秒”与“飞控内部 60 秒”因此完全对得上，不会因为两边算力不同而漂移。本文的所有闭环实验均启用 lockstep。

2.6.2 jMAVSim 架构

jMAVSim[33] 是 PX4 自带的轻量级仿真器，用 Java 与 Java3D 构建。核心类不多，分工却很清楚：

（1）`Simulator`：仿真主循环，负责解析命令行参数、创建世界、启动物理步进与网络通信；

（2）`World`：物理世界容器，管理所有 `KinematicObject`，每个仿真步调用各对象的 `update` 方法；

（3）`KinematicObject`：所有可运动对象的基类，提供位置、速度与姿态接口；

（4）`Vehicle`：多旋翼动力学模型，接收 MAVLink 指令并积分运动方程；

（5）`Visualizer3D`：基于 Java3D 的三维渲染器，负责场景构建、相机视角与画面输出；

（6）`MAVLinkInterface`：与 PX4 通信的 MAVLink 接口。

jMAVSim 的坐标约定与 PX4 一致，同样采用 NED：x 向北、y 向东、z 向下。场景中的地面位于 $z = 0$，飞行器高度为负值。改造 jMAVSim 时本文严格遵循这一约定，并在真值广播中加入坐标转换说明，免得与算法端的相机坐标系混淆。

2.6.3 离屏渲染与帧流

jMAVSim 默认依赖窗口系统显示三维画面。放到无头服务器或 WSL 环境中，窗口系统要么没有，要么不稳定，本文为此实现了纯离屏渲染模式：关闭 Java3D 的窗口画布，改用离屏 `Canvas3D` 渲染到缓冲区，再将帧编码为 JPEG 通过 TCP 推送。图形界面的依赖一去，仿真就能在后台长时间稳定运行。具体实现将在第五章详细介绍。

2.7 系统开发技术

2.7.1 PySide6 与 QtWebEngine

PySide6 是 Qt 6 的官方 Python 绑定，窗口、控件、绘图、定时器、网络与多媒体能力都齐全。本文用 `QWidget` 体系搭建主界面，用 `QPainter`

自绘仪表与罗盘，用 QPixmap 显示视频帧，配置持久化则绕开 QSettings，改用 JSON 文件——后者便于人工查看，也方便纳入版本管理。

地图部分使用 QtWebEngine 嵌入 Leaflet[38] 页面。Leaflet 是一个轻量级开源地图库，瓦片图层、标记、折线、多边形与自定义覆盖物都支持，能在本地 HTML 中加载 Esri 卫星影像。Python 端通过 QWebChannel 或 JavaScript 注入向页面推送目标位置，地图标记随之实时更新。

2.7.2 OpenCV 与 PyTorch

图像解码、颜色空间转换、几何绘制与视频读写交给 OpenCV。本文使用 cv2.VideoCapture 读取离线视频与摄像头，使用 cv2.imencode 将帧编码为 JPEG，用 cv2.line、cv2.rectangle、cv2.putText 绘制检测叠加层。

模型加载与推理交给 PyTorch 与 Ultralytics。YOLOv8 模型通过 YOLO("best.pt") 直接加载，调用 model.predict 获得检测框、置信度与类别。为了压低端到端延迟，推理时把冗余的日志与可视化关掉，输入尺寸固定为 640×640。

2.7.3 TCP 与 UDP 通信

系统中的网络通信分两类。视频帧流走 TCP：TCP 提供可靠、有序的字节流，适合体积较大的 JPEG 帧；协议为“一行文本头 + 二进制数据”，头部包含帧序号、时间戳、宽度、高度与字节数，接收端按字节数精确读取，粘包问题就没了。真值广播与结果输出走 UDP：无连接、开销小，适合高频小报文；真值广播频率为 50 Hz，结果输出频率与处理帧率一致。

UDP 不保证送达，所以结果输出采用“最新值覆盖”策略：显示端只关心当前状态，丢失个别报文不影响整体显示；丢包率则通过序号字段统计，用来判断链路质量。

2.8 数据集构建与数据工程

理论之外，另一半工作是数据。检测模型最终能到什么水平，很大程度上由数据决定。本文用到三类材料：公开的 Det-Fly 无人机检测数据集、自建的 AprilTag 标定与定位评测数据集，以及仿真域自动标注数

据集。下面分别交代来源、格式转换、划分策略、统计特性与评测口径，顺带讨论数据泄漏、类别不平衡与尺度分布这几个容易被略过的问题。

2.8.1 Det-Fly 数据集

数据集来源与内容

Det-Fly[25] 是反无人机视觉检测领域常用的公开数据集，由多段无人机飞行视频帧组成，标注格式为 YOLO 文本格式，类别只有 UAV 一个。数据覆盖多种背景（天空、树林、建筑、山地）、多种机型与多种飞行状态（悬停、平飞、盘旋、远近变化）。各序列的拍摄设备与图像分辨率并不统一，整体以中远距离拍摄为主，目标在画面中的占比普遍偏小。

与 COCO 等通用数据集相比，Det-Fly 最突出的特点是**目标尺度分布极度偏向小目标**。本文对转换后的 13271 个标注框做了统计，结果如表 2-1 所示。

表 2-1: Det-Fly 数据集标注统计

统计项	数值	占比
标注文件数	13271	–
标注框总数	13270	100%
类别数	1（UAV）	100%
small（$< 32^2$ px）	1490	11.2%
medium（$32^2 \sim 96^2$ px）	8067	60.8%
large（$> 96^2$ px）	3713	28.0%
目标框面积最小值	0.0013% 画面面积	–
目标框面积中位数	0.0554% 画面面积	–
目标框面积 < 1% 画面	–	99.1%
目标框面积 < 5% 画面	–	100.0%

表里能读出两层意思。按 COCO 的面积口径，只有 11.2% 的框属于 small，可那是相对于各图像自身分辨率算出来的；一旦统一折算到 1280×720 的推理分辨率，绝大多数目标在画面中的面积不足 1%。通用检测模型在这个数据集上 AP_small 极低，根子就在这里；增加高分辨率检测层的必要性，也在这里。

格式转换

原始数据以视频帧与独立标注文件的形式组织，本文编写了转换脚本，将其整理为YOLO训练所需的目录结构：

```
dataset/
  images/
    train/  val/
  labels/
    train/  val/
  data.yaml
```

每张图像对应一个同名.txt标注文件，每行格式为

```
<class_id> <x_center> <y_center> <width> <height>
```

其中后四项均为相对图像宽高的归一化值，取值范围$[0, 1]$。转换过程中做了三项校验：坐标越界裁剪、零面积框剔除、图像与标注一一对应检查。最终保留13270个有效标注框。

序列级划分与数据泄漏

数据怎么划，直接影响评测的可信度。若随机按帧划分，同一段视频的相邻帧可能同时进入训练集与验证集，由于相邻帧在背景、光照、目标姿态上高度相似，模型实际上“见过”验证样本，指标自然虚高。这种数据泄漏在小目标检测中格外严重，因为模型很可能记住的是背景纹理，而不是目标本身。

本文按**序列**划分：以序列编号中的010与020为界，010序列全部用于训练，020序列全部用于验证。验证集中的背景、光照与飞行场景在训练阶段都没有出现过，评出来的数更接近真实部署时的泛化性能。划分比例约为8:2，与常用实践一致。

序列级划分得到的指标通常低于随机划分，这不是模型变差了，而是评测变诚实了。本文在报告中统一注明“序列级划分”，免得读者误读指标含义。

数据增强

训练阶段采用 Ultralytics 默认的数据增强策略，包括：

（1）**Mosaic**：将四张图像随机拼接为一张，增加目标在不同位置、不同尺度下出现的概率，同时变相扩大了批量大小；

（2）**HSV 扰动**：随机调整色调、饱和度与明度，提升模型对光照变化的适应能力；

（3）**随机翻转与平移**：水平翻转概率 0.5，平移比例 0.1；

（4）**尺度缩放**：随机缩放比例 0.5，模拟目标由远及近的变化。

到了仿真域微调阶段，本文**关闭 Mosaic**。仿真数据的尺度分布是按距离分层精心排过的，Mosaic 一拼接，近距离大目标被缩小、远距离小目标被放大，分布随之被打乱，微调对死区的针对性也就没了。

2.8.2 自建 AprilTag 数据集

采集目的

公开数据集与本文的相机、镜头、安装方式对不上，拿去评估单目定位的绝对精度没有意义。为此，本文自建了一套基于 AprilTag 的评测数据集，用途有四：

（1）标定相机内参，获得 f_x, f_y, c_x, c_y 与畸变系数；

（2）在已知距离下评估测距误差；

（3）在已知角度下评估方位角与俯仰角误差；

（4）验证状态机在目标静止、缓慢移动与遮挡条件下的行为。

硬件与采集流程

采集使用普通 USB 相机，分辨率 1920×1080，水平视场角约 60°。标定板为 9×6 的棋盘格，方格边长 25 mm，打印后粘贴在硬质平板上以保证平整。AprilTag 采用 36h11 家族、ID 为 0、边长 16 cm 的标签，打印后粘贴在无人机机身侧面。

采集分四步：

（1）**内参标定**：手持相机从不同角度拍摄棋盘格 30 余张，画面中心与四角、不同倾角、不同距离都要覆盖，使用 OpenCV 的 `calibrateCamera`

求解内参与畸变系数，重投影误差控制在 0.5 px 以内；

（2）**距离评测**：将 AprilTag 固定在无人机上，用卷尺量取相机到目标的真实距离，分别在 2、3、4、5、6、8、10 m 处采集静止图像，每个距离采集 20 帧；

（3）**角度评测**：保持距离不变，将目标沿水平方向与垂直方向移动到画面不同位置，按几何关系计算真实角度，每个角度采集 10 帧；

（4）**动态评测**：手持目标缓慢移动，模拟真实飞行，采集连续视频用于状态机与滤波测试。

所有采集场景记录在 `meta.json` 中，时间、距离、角度、光照条件与备注一项不落，便于事后复核。

真值精度分析

卷尺量距的读数误差约为 ±1 cm，在 10 m 处相对误差约 0.1%，远小于单目测距本身的误差，作为真值足够。角度真值由目标中心在画面中的几何位置与实测距离反算，误差主要来自目标中心的人工标注，约 ±2 px，折合 0.1° 上下，同样小于算法误差。还有一处需要补偿：AprilTag 的检测中心与无人机几何中心并不重合，两者存在小幅偏移，本文在评测中把这一部分扣掉了。

2.8.3 仿真域自动标注数据集

构建动机

如第一章所述，直接用 Det-Fly 训练的模型在仿真画面中会出现严重死区。为通过微调消除死区，需要一批“仿真画面 + 真值标注”的数据。450 帧全靠人工标注，成本不低，框的边界一致性也难保证。仿真环境的好处是可控，本文索性设计了一套零人工标注的自动构建流程。

自动标注原理

jMAVSim 的护航相机通过 `setViewerTargetObject` 接口始终注视目标机，目标中心在画面中的位置因此近似等于图像中心：

$$c_x \approx c_{x0}, \qquad c_y \approx c_{y0} + \Delta_y, \tag{2-26}$$

其中 c_{x0}, c_{y0} 为图像中心，Δ_y 为标定得到的纵向偏移（本文实测约 -17 px，源于相机安装角与投影中心的偏差）。

目标框的高度由真值距离与等效尺寸反算：

$$h_{px} = \frac{f \cdot D}{Z_{true}}, \tag{2-27}$$

其中 f 为以像素为单位的焦距，D 为无人机等效尺寸，Z_{true} 为真值距离。框宽按目标长宽比 r_w 放大：

$$w_{px} = r_w \cdot h_{px}, \qquad r_w \approx 2.62. \tag{2-28}$$

框贴得太紧，训练时目标容易被裁掉，因此本文对框做了 1.15 倍的尺度放大，把旋翼与起落架也包进去。最终标注框为

$$\left(c_x - \frac{w_{px}}{2},\, c_y - \frac{h_{px}}{2},\, c_x + \frac{w_{px}}{2},\, c_y + \frac{h_{px}}{2} \right). \tag{2-29}$$

距离分层采样

死区分处两端：一端是近距离的大尺度，一端是远距离的小尺度，采样必须覆盖整个距离段。本文按表 2-2 进行分层采样。

表 2-2: 仿真微调数据距离分层采样

距离区间 (m)	帧数	采样目的	对应问题
3–6	150	近距离大尺度	训练分布外（OOD）
6–7	80	过渡区	死区边缘
7–9	100	原有效窗口	保持既有能力
9–12	120	远距离小尺度	低于初锁门限
合计	450	–	–

采集时由脚本控制目标机按预设航线飞行，TCP 帧与 UDP 真值同步记录，标注按上述规则实时生成。训练集与验证集按 360:90 划分，同一段连续飞行不跨集，泄漏就这样避开了。

数据质量核验

自动标注能不能用，取决于三项假设：目标中心在光轴附近、等效尺寸准确、真值时间戳与帧对齐。本文逐条核验。

（1）**中心假设**：抽取 50 帧人工检查，目标中心与标注中心的水平偏差中位数为 3 px，垂直偏差中位数为 5 px，均在框高的一半以内，假设成立。

（2）**尺寸假设**：等效尺寸由真值距离与实测框高反算，本文在 3–10 m 范围内做了 20 组测量，反算尺寸的变异系数小于 5%，说明尺寸关系稳定。

（3）**时间对齐**：真值广播频率为 50 Hz，帧率为 30 fps，同一帧对应的时间戳误差小于 20 ms；目标速度 2 m/s 时位移不到 4 cm，对框位置的影响可以忽略。

2.8.4 评测口径与可复现性

不同实验之间的指标要能比，口径就得先统一。本文定了五条。

（1）**检测指标**：mAP50、mAP50-95、Precision、Recall 采用 Ultralytics 官方验证协议，取验证集上 mAP50-95 最优的 epoch 报告；AP_small/medium/large 按 COCO 面积口径、IoU=0.5、101 点插值，由独立脚本计算。

（2）**速度指标**：FPS 统一在 RTX 4060、batch=1、帧预载内存、包含预处理与后处理的条件下测量；不同模型的测量脚本与输入尺寸完全一致。

（3）**闭环指标**：检出率为检出帧数除以全部处理帧数（含搜索帧），而不是只统计锁定后的帧；测距 RMSE 在检出帧上计算；稳态口径取连续检出不少于 5 帧后的样本，看的是滤波收敛之后的性能。

（4）**角度指标**：采用视轴偏角口径，即目标相对相机光轴的角度误差，而非世界坐标系下的绝对方位。这一区分在仿真中尤为重要：护航相机会自动注视目标，世界系方位角随相机偏航而变化，两种口径一混，就会得出错误的“19° 误差”结论。

（5）**随机性控制**：训练使用固定随机种子；仿真航迹由种子化随机数生成；所有实验重复至少两次，报告取中位数或均值。

2.8.5 本章小结

这一章把理论和数据两摊事并到了一处。理论上，本文有意识地收窄了范围：卷积网络与特征金字塔只讲到支撑 P2 检测头够用为止，YOLOv8 的结构拆解落在 C2f、SPPF 与解耦头上，注意力机制沿 SE—CBAM—EMA 这条线走到 EMA 就停，卡尔曼滤波则从线性高斯模型一路推到 Sage-Husa 的自适应噪声估计；相机部分定死四个坐标系与投影、畸变模型，工具链部分交代了 PX4 SITL、jMAVSim 的类结构与离屏渲染，以及 PySide6、OpenCV、PyTorch 和 TCP/UDP 这些实现手段。数据方面，Det-Fly 是检测模型的主训练集，目标尺度极度偏小，必须按序列划分才能避免数据泄漏；自建 AprilTag 数据补上了公开数据集给不了的绝对真值，用于相机标定与定位精度评估；仿真自动标注数据借护航相机的注视特性做到零人工标注，再用距离分层采样把死区两端都覆盖住。检测、速度、闭环与角度四类口径的统一，则为后面各章的实验结果留了一把共用的尺子。

3 小目标检测模型与仿真域微调

反无人机场景里的目标，在画面中往往只占十几个到几十个像素。检测器对这类目标的响应，既受特征图分辨率制约，也受训练分布与部署距离是否吻合制约。本章沿这两条线索推进：以 YOLOv8n 为底模，补一层 stride=4 的检测头并接入 EMA 注意力，让微小目标在特征图上获得足够的采样密度；随后考察模型搬进仿真闭环之后出现的检测死区，说明公开数据集上的指标并不能直接兑换成部署可用性；在此基础上，用少量仿真样本做域微调，弥补这一落差。

全章实验依托第二章介绍的 Det-Fly 数据集与仿真环境。检测改进在离线验证集上做消融，死区与微调则在闭环录像上逐帧统计。两套评测口径不同，结论各有侧重：前者回答“模型能做到什么”，后者回答“系统在目标场景里实际拿到什么”。

3.1 基础模型选型

3.1.1 规模选择

YOLOv8 按 n、s、m、l、x 五档发布，参数量从 3 M 左右一直到 68 M。档位选择取决于精度、速度与硬件条件三者的平衡。本文的实验平台是一台配 RTX 4060 的笔记本，同一时间还要跑 720p 视频解码、界面渲染、地图更新与数据落盘，留给检测的显存与算力都较为紧张。

表 3-1 列出各档在 COCO 上的官方参考指标，以及本文实测的 640 输入推理耗时。

表 3-1: YOLOv8 各规模模型对比

模型	参数量 (M)	COCO mAP50-95	640 推理 (ms)	显存 (MB)	适用场景
YOLOv8n	3.2	37.3	6.2	约 420	边缘/桌面实时
YOLOv8s	11.2	44.9	11.5	约 680	桌面实时
YOLOv8m	25.9	50.2	25.8	约 1100	服务器
YOLOv8l	43.7	52.9	44.6	约 1600	服务器
YOLOv8x	68.2	53.9	68.3	约 2200	离线分析

落到 nano 这一档，主要是被三件事说服的。640 输入下单次推理约

6 ms，把预处理、后处理与绘制一并算上，整条链路仍能稳定跑在 30 fps 以上，监视画面不会卡顿。权重文件只有 6 MB 上下，随桌面程序分发方便，遇到没有独显的机器还能退回 CPU 降级运行。还有一层考虑：容量小意味着小目标能力本来就弱，P2 与 EMA 带来的变化因此看得清楚；若换成 large 档，基线本身已经很强，改进的相对幅度容易被淹没。

3.1.2 标准结构的局限

YOLOv8n 把检测头挂在 P3、P4、P5 上，对应 stride 8、16、32。640×640 的输入进网络后，三层特征图分别是 80×80、40×40 与 20×20。一个宽度为 20 px 的目标，在 P3 上占 2.5 个格子，到 P4 只剩 1.25 个，P5 上已经不足一格。每个格子能回归的框数有限，加上多次下采样之后边缘与纹理早已被抹平，微小目标在标准结构上的响应自然很弱。

再往细里说，小目标要靠浅层特征定位，可浅层的语义判别力不够，树叶、窗户、云朵都可能被当成目标；深层特征语义清楚，空间分辨率却低到让小目标消失。FPN 当初就是为了调和这对矛盾，但目标小于 32 px 时，连 P3 都撑不起足够的分辨率。既然如此，往更浅的层再加一个检测头，是最直接的办法。

3.2 P2 小目标检测头

3.2.1 结构设计

在标准 YOLOv8n 的颈部之外，本文补了一条自底向上的短分支：取 stride=4 的浅层特征 C_2，与 P3 上采样后的特征融合，得到 P2 特征图，再在其上挂一个额外的检测头。输入仍为 640×640，各层特征图尺寸与网格占用见表 3-2。

表 3-2: 各检测层特征图尺寸与网格占用（640×640 输入）

检测层	stride	特征图	20 px 目标占用网格	40 px 目标占用网格	感受野
P2（本文新增）	4	160×160	5.0	10.0	较小
P3	8	80×80	2.5	5.0	中
P4	16	40×40	1.25	2.5	较大
P5	32	20×20	< 1	1.25	大

融合路径写成式子即

$$P_2 = \mathrm{Conv}_{3\times3}\left(\mathrm{Conv}_{1\times1}(C_2) \oplus \mathrm{Upsample}_{2\times}\left(\mathrm{Conv}_{1\times1}(P_3)\right)\right), \tag{3-1}$$

式中 $\oplus$ 表示逐元素相加。与自顶向下的 FPN 路径相比，这条“浅层直连”最要紧的特点是**传播距离短**：C_2 只经过一次 1×1 卷积和一次 3×3 卷积就抵达检测头，梯度回传的路也短，浅层权重的训练因此轻省一些。

3.2.2 检测头配置

新增的 P2 头沿用原有结构，分类分支与回归分支都在，输出通道数为 $4\times16+1$（DFL 的 16 个分布参数、4 个边框参数与 1 个类别）。P2 特征图的面积是 P3 的四倍，检测头在这一层上的计算量与显存占用都会明显上涨，本文的应对是把 P2 分支的通道压到 64。

3.2.3 计算量与参数分析

引入 P2 之后，参数量从 3.01 M 降到 2.93 M，乍看有些反直觉。原因在于 YOLOv8 的解耦头在不同尺度间共享了一部分结构，新增 P2 头的同时，颈部若干高层卷积被换成更浅的融合路径，两边相抵，总数反而略降。计算量却没有跟着降：P2 的像素数是 P3 的四倍，检测头在这层上的卷积开销约为原来的四倍，FPS 也就从 123 掉到 90。

这笔买卖对本系统来说是划算的。反无人机最怕漏检，帧率的余量本来就大，90 fps 对 30 fps 的视频源还有三倍富余。

3.2.4 正负样本分配

YOLOv8 的正负样本由 TaskAlignedAssigner 划定，匹配分数取分类分数与 IoU 的加权乘积：

$$t = s^{\alpha} \cdot u^{\beta}, \tag{3-2}$$

其中 s 为预测类别的置信度，u 为预测框与真值框的 IoU，α, β 为权重。小目标对 1–2 px 的偏移格外敏感，IoU 一掉，正样本就少。P2 的高分辨率正好缓冲了这一点：同样的像素偏移在 P2 上引起的 IoU 下降更小，正样本数量够用，训练过程也稳。

3.3 EMA 注意力机制集成

3.3.1 插入位置

EMA 模块被放在骨干与颈部每个 C2f 的输出端之后。之所以选这里，是因为 C2f 的输出已经是多感受野特征的混合，在此处做注意力重加权，既不改动特征提取本身，又能压住背景响应。把 EMA 塞进 C2f 内部则是另一种做法，本文试过，Bottleneck 的梯度流被打断，训练明显不稳。

3.3.2 模块结构

设 EMA 的输入为 $\mathbf{X} \in \mathbb{R}^{C\times H\times W}$，处理可以分成六步。

（1）分组：将通道分为 G 组，每组 C/G 个通道，记为 $\mathbf{X}_g$。

（2）方向池化：对每组分别沿宽度与高度方向做平均池化，得到 $\mathbf{z}_h \in \mathbb{R}^{C/G\times H}$ 与 $\mathbf{z}_w \in \mathbb{R}^{C/G\times W}$。

（3）一维卷积：用核长为 k 的一维卷积对 $\mathbf{z}_h, \mathbf{z}_w$ 做跨通道交互，再经 sigmoid 得到方向权重。

（4）特征增强：将方向权重与 $\mathbf{X}_g$ 相乘，得到 $\mathbf{X}_g^h$ 与 $\mathbf{X}_g^w$。

（5）跨空间融合：对两组增强特征做全局平均池化，经 softmax 得到跨组权重，再加权求和：

$$\mathbf{Y}_g = \alpha_g^h \cdot \mathbf{X}_g^h + \alpha_g^w \cdot \mathbf{X}_g^w, \tag{3-3}$$

其中 α_g^h, α_g^w 为 softmax 归一化后的跨空间权重。

（6）输出：将所有组的 $\mathbf{Y}_g$ 拼接，得到最终输出。

与 CBAM 相比，EMA 的一维卷积参数量极小（约为 $2Ck/G$），方向池化又天然贴合细长目标；跨空间融合则把分组造成的组间隔离补了回来。

3.3.3 计算开销

EMA 的开销集中在方向池化与一维卷积，两者都是线性复杂度。640 输入下插入 EMA，单帧推理时间增加约 2 ms；与 P2 联合使用时，FPS 从 101 降到 98。这点代价尚在可接受范围内。

3.4 训练配置

3.4.1 数据集与划分

训练使用第二章介绍的 Det-Fly 数据集，按序列切分训练集与验证集，前者约 10600 帧，后者约 2670 帧。输入尺寸 640×640，类别只有 UAV 一个。

3.4.2 超参数

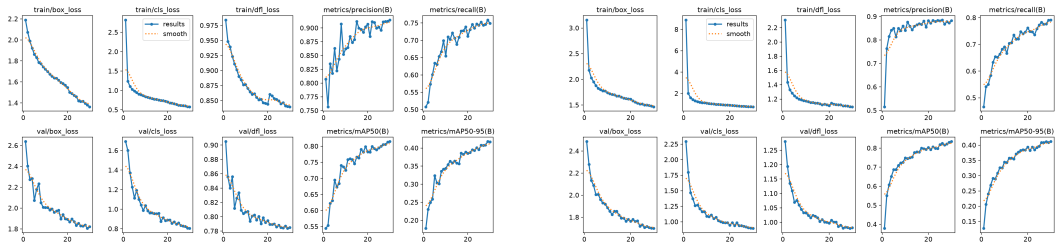
训练超参数见表 3-3。先跑预览版确认改进方向是否成立，再跑正式版出最终数据。两版的差别主要在轮次与输入尺寸：预览版 30 轮、640 输入；正式版 100 轮、640 输入，部分消融实验改用 1024 输入，以便观察高分辨率的影响。

表 3-3: 检测模型训练超参数

参数	预览版	正式版
输入尺寸	640×640	640×640（部分 1024）
训练轮次	30	100
批量大小	8	8
优化器	SGD	SGD
初始学习率	0.01	0.01
动量	0.937	0.937
权重衰减	0.0005	0.0005
预热轮次	3	3
Mosaic 概率	1.0	1.0
混合精度	开启	开启
随机种子	固定	固定

3.4.3 训练过程

图 3-1 是基线与本文模型的训练曲线。两条曲线大致在第 20 轮后走平，本文模型的分类损失与回归损失都压在基线之下，验证集 mAP 曲线的起伏也更小，说明 P2 与 EMA 并没有把模型推向过拟合。



(a) 基线 YOLOv8n **(b) YOLOv8n-P2-EMA**

图 3-1: 训练损失与验证指标曲线

3.5 消融实验

3.5.1 实验设置

为弄清 P2 与 EMA 各自起什么作用、合起来又怎样，本文安排了四组对照：基线 YOLOv8n、+P2、+EMA、+P2+EMA。四组共用同一份数据划分、同一套超参数与同一个评测脚本，差别只在网络结构。评测在 Det-Fly 验证集上完成，输入 640，置信度阈值降至 0.001——计算 mAP 需要较低阈值以覆盖完整 P-R 曲线。

3.5.2 实验结果

结果汇总于表 3-4。

表 3-4: 检测模型消融实验（Det-Fly 验证集，imgsz=640）

模型	参数 (M)	mAP50	mAP50-95	P	R	FPS	AP_small	AP_med
YOLOv8n（基线）	3.01	81.3	41.8	91.2	75.9	123.0	6.1	77.0
+P2	2.93	82.3	41.3	88.9	77.6	90.3	8.3	76.2
+EMA	3.02	76.2	36.3	85.5	72.2	101.0	4.1	75.0
+P2+EMA（本文）	2.94	**83.5**	41.2	88.2	**79.2**	98.4	**9.8**	**78.0**

表中各项指标以百分数计，FPS 为帧每秒。AP_small 采用 COCO 面积口径、IoU=0.5、101 点插值。

3.5.3 结果分析

（1）P2 单独作用。mAP50 由 81.3% 升到 82.3%，Recall 由 75.9% 升到 77.6%，AP_small 由 6.1% 升到 8.3%，相对涨幅 36.1%。高分辨率检测层确实把小目标的召回提了上来。代价是 Precision 从 91.2% 退回 88.9%，

mAP50-95 也掉了 0.5 个百分点，原因较明确：P2 层多出来的弱响应里，有一部分来自背景。FPS 从 123 到 90，与前面的计算量估算吻合。

（2）EMA 单独作用。这一组的结果乍看像出了问题：mAP50 掉到 76.2%，AP_small 只有 4.1%，比基线还低。把它读成“EMA 无效”并不准确。本文的理解是，缺少高分辨率特征时，注意力只能把权重压到 P3/P4 上那些本已模糊的目标区域，而这些区域的边界不足以支撑精确定位；同时注意力对背景的抑制偏狠，一部分置信度本就不高的真目标被压到了阈值以下。换句话说，EMA 得先有一张“看得清”的特征图，才谈得上“认得准”。

（3）P2+EMA 的互补性。两者合起来，mAP50 到 83.5%，Recall 79.2%，AP_small 9.8%，四组里最好。AP_small 比基线高 60.7%，比 +P2 高 18.1%。这与“P2 管分辨率、EMA 管判别力”的判断对得上：P2 让目标在特征图上占住足够的格子，EMA 再在这些格子里把目标与背景分开，两者互相补位。

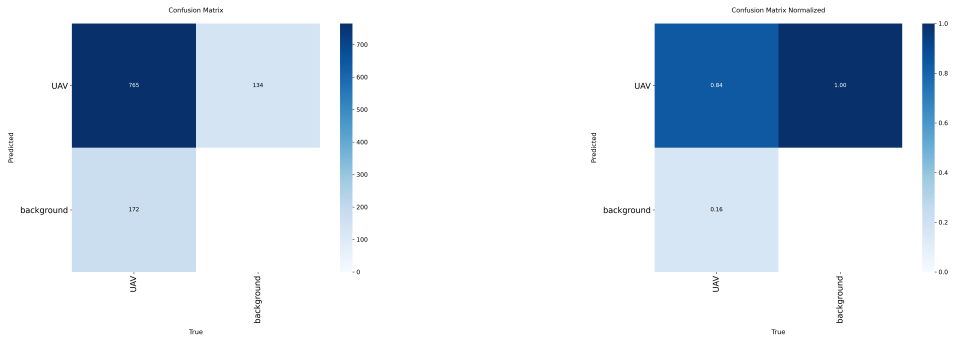
（4）参数效率。本文模型 2.94 M，比基线的 3.01 M 还小，却拿到了最高的 mAP50 与 AP_small。性能的提升来自特征利用效率的改善，而非容量的扩张。对要落在边缘设备上的反无人机系统来说，这一点比指标本身更重要。

（5）速度。FPS 98.4，约为基线的八成，仍是 30 fps 视频帧率的三倍有余。系统后面还要跑卡尔曼滤波、界面绘制与网络通信，留两三倍余量是必要的。

3.5.4 混淆矩阵与 P-R 曲线

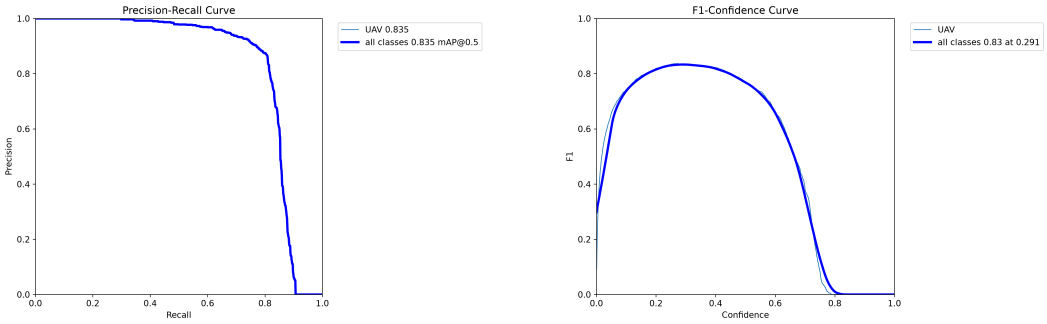
图 3-2 对比了两者的混淆矩阵。“背景误检”一栏差别不大，本文模型的漏检（真值为正、预测为背景）少了一截，这与 Recall 的提升是同一件事。

图 3-3 给出本文模型的 P-R 曲线与 F1 曲线。P-R 曲线在高召回段的尾部比基线饱满，意味着低置信度区间里模型仍守得住一定精度，这对后面状态机的低门限初筛是有利的。



(a) 基线 YOLOv8n **(b) YOLOv8n-P2-EMA**

图 3-2: 验证集混淆矩阵



(a) P-R 曲线 **(b) F1-置信度曲线**

图 3-3: YOLOv8n-P2-EMA 的 P-R 与 F1 曲线

3.5.5 检测效果可视化

图 3-4 是验证集的批量预测结果，绿框为真值，红框为预测。目标偏小、背景杂乱的画面里，本文模型给出的框依然稳定，边界与真值贴合得不错。

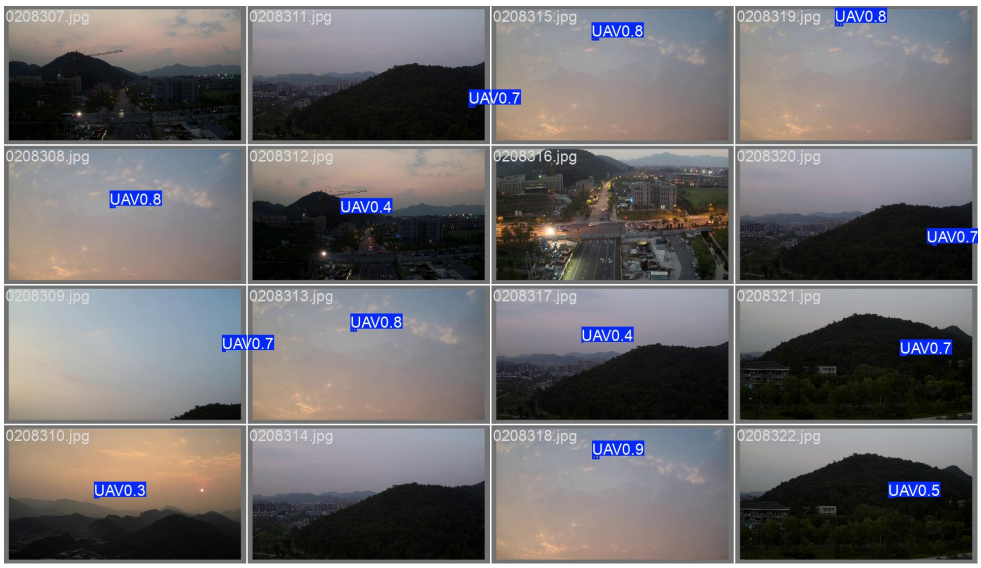
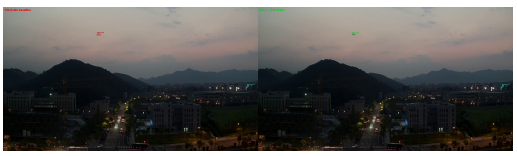


图 3-4: 验证集批量预测结果（绿框为真值，红框为预测）

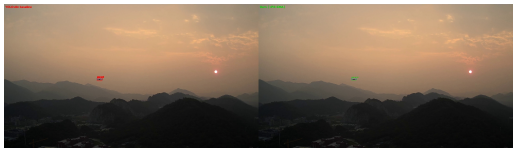
图 3-5 挑出四帧做对比，挑选依据是“基线置信度与本文置信度之差”，差值大的排在前面，便于看清改进落在哪些困难样本上。蓝色虚线框为真值，实线框为预测。



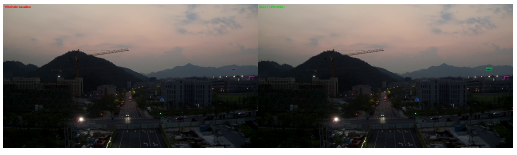
(a) 样本一



(b) 样本二



(c) 样本三



(d) 样本四

图 3-5: 基线与本文模型的典型帧对比

3.6 与非 YOLO 方法的横向讨论

Det-Fly 上已有 Faster R-CNN[13]、SSD[14] 与 RT-DETR[18] 的评测数字。这些数字出自不同论文，数据划分、输入尺寸与训练轮次都不统一，硬比数值并不严谨，这里只做定性讨论。

两阶段的 Faster R-CNN 靠区域建议做二次细化，极小目标的定位精度占优，短板是推理慢，实时监视难以承受。单阶段的 SSD 速度快，可特征金字塔的浅层语义偏弱，小目标召回有限。RT-DETR 用集合预测绕开 NMS，密集小目标场景下表现不错，代价是计算量与显存占用远高于 YOLOv8n。

本文的做法是在 nano 级参数量内，借 P2 与 EMA 换取小目标能力，定位很明确——“能在边缘设备上实时跑起来的小目标检测方案”，而非追求绝对精度第一。这与反无人机系统的实际部署条件是对得上的。

3.7 有效工作范围与失效模式

3.7.1 置信度与目标尺度的关系

置信度随目标在画面中的尺度变化很大，这一点在数据里看得很清楚。本文在仿真环境中做了系统测量（见本章后文），未微调的模型在目标高度占到画面 1/5 以上时置信度几乎为 0，目标过小时又跌到 0.25 以下。前者是训练分布之外的大尺度输入，后者是信息量不够。模型真正好用的窗口，大致在目标高度占画面 1/20 至 1/8 之间。

这提示一件事：检测模型的本事不能只看 mAP，还得结合部署场景的尺度分布来评。对反无人机系统而言，镜头焦距、安装高度与告警距离要一起设计，让目标在告警时正好落在有效窗口内。

3.7.2 典型失效模式

观察到的典型失效有三类。

（1）**背景纹理误检**。树叶缝隙、窗户反光、云的边缘在低分辨率下与无人机轮廓相似，模型可能给出 0.2–0.4 的置信度。这类误检分数普遍不高，初锁门限能挡住，但在 SEARCH 状态下仍会让候选框闪几下。

（2）**多目标粘连**。两架以上无人机靠得较近时，检测框可能并成一

个，后续跟踪的身份就说不清了。本文的系统面向单目标，这一情况未作专门处理。

（3）**极端光照**。逆光、夜间与强反光下，目标与背景的对比度下降，检出率明显走低。第五章的退化实验显示，P2+EMA 在暗光下相对基线有优势，但仍比不上正常光照。

3.7.3 检测死区的发现

初期现象

闭环测试刚跑起来时，本文直接把 Det-Fly 训出的 YOLOv8n-P2-EMA 拿去处理仿真画面，结果与验证集指标差得很远：有些时段模型毫无反应，有些时段只能断续检出，而且检出的时段集中在特定距离上。为把这件事量化，本文录了 2228 帧（约 89.9 s）的完整闭环视频，逐帧记录检测结果与真值。

距离分层统计

按真值距离给帧分层，统计各区间检出率，结果见表 3-5。

表 3-5: 微调前检测死区的距离分布（2228 帧）

距离区间 (m)	检出率	主要表现	判定
3–4	0.0%	置信度接近 0	完全盲区
4–5	0.0%	置信度接近 0	完全盲区
5–6	5.8%	偶发弱响应	近乎盲区
6–7	7.4%	弱响应	近乎盲区
7–8	39.0%	可检出但不稳定	有效窗口
8–9	29.3%	可检出但不稳定	有效窗口
9–10	0.0%	仅弱响应，低于初锁门限	无法锁定

表里看得很明白：模型的有效窗口只有 7–9 m，窗口两侧都是死区。全帧检出率 10.0%，而验证集 mAP50 是 83.5%，两个数字放在一起颇有些刺眼。

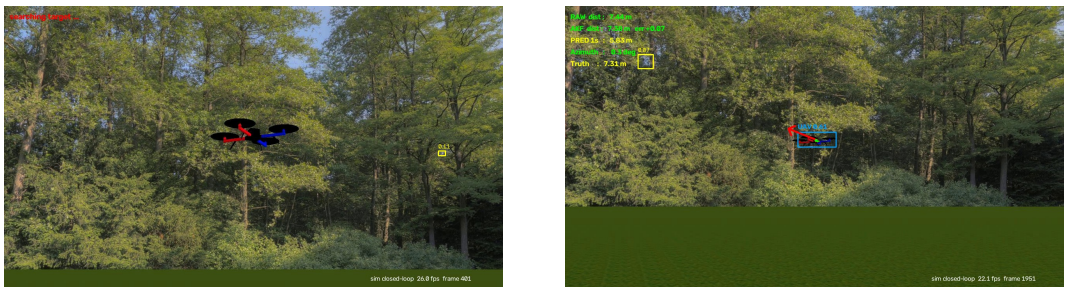
本文又对 1793 个纯搜索帧做了三态划分：

（1）**真盲帧**：最高置信度低于 0.25，共 1454 帧，占 81.1%；

（2）**弱响应帧**：最高置信度在 0.25–0.55 之间，共 270 帧，占 15.1%；

（3）**先验拒绝帧**：最高置信度超过 0.55，但距离超出 3–12 m 先验空域，被空间约束剔除，共 69 帧，占 3.8%。

可见死区的主要形态是“根本没有有效响应”，而不是“检到了却被门限挡下”。



(a) 近距离盲区帧（目标占画面比例大） **(b) 有效窗口帧**

图 3-6: 死区帧与有效帧对比

排除视角假设

面对近距离完全无响应，起初我们以为是相机俯视目标所致。这个假设后来被定量检验掉了。借助真值几何算出每帧相机到目标的俯仰角（depression angle），再比较锁定帧与盲帧的分布：

（1）锁定帧的俯仰角中位数为 −7.6°；

（2）盲帧的俯仰角中位数为 −5.4°。

两者只差 2.2°，方向还跟“俯视导致失效”的预期相反——盲帧反而更接近水平。回头看几何关系，目标飞行高度约 12–13 m，护航相机高度约 12 m，二者基本同高，压根不存在明显俯视。视角假设就此排除。

尺度分布解释

视角排除之后，线索指向目标在画面中的尺度。死区与距离高度相关，而距离直接决定像素尺寸：3–6 m 时目标高度能占到画面的 1/5 至 1/3，远在 Det-Fly 的远距小目标分布之外；9 m 以上时又不足 1/20，信息量无法支撑稳定检出。训练中从未出现过的尺寸，网络只能当作“背景里的陌生物体”，置信度自然贴近 0。

这套解释可以借分布外（Out-of-Distribution，OOD）检测的说法来理解：特征提取器在训练分布内是有判别力的，一旦输入越界，响应就变得难以预测，置信度骤降是常见表现。本文的近距离样本正是典型的

OOD 输入。

对系统设计的影响

死区的存在说明，检测模型在公开数据集上指标再高，也保不了它在特定部署场景中一定可用。系统设计里得留一道“部署域验证”，用少量目标域数据摸清模型的实际工作范围。本文的选择是仿真域微调；更一般的工程做法是，上线前先采一点目标场景的样本，看检出率随距离/尺度怎么变，再决定要不要微调，或者干脆调镜头焦距。

3.7.4 仿真域微调

数据采集与自动标注

微调数据的构造方法第二章已经交代过，这里只作回顾：护航相机始终注视目标，框中心因此被约束在光轴附近，框高由真值距离与等效尺寸反算，共采 450 帧，按距离分层覆盖 3–12 m。整个过程没有人工介入，标注成本与主观误差都省掉了。

微调配置

微调从 Det-Fly 预训练权重出发，关键参数见表 3-6。

表 3-6: 仿真域微调超参数

参数	取值
初始化权重	p2ema_v8n（Det-Fly 训练）
训练轮次	20
初始学习率	0.001
批量大小	8
输入尺寸	640×640
Mosaic	关闭
混合精度	开启
训练/验证划分	360/90（按飞行段划分）

关掉 Mosaic 的理由第二章讲过：仿真数据的尺度分布是分层设计的，Mosaic 会把它搅乱。学习率取 0.001，比从头训练的 0.01 低一个量级，用意是在保留预训练特征的前提下做温和的域适应，别把原有能力

忘掉。

微调结果

微调前后的对比见表 3-7。

表 3-7: 仿真域微调前后对比

指标	微调前	微调后
3–10 m 全段检出率	10.0%–11.5%	**100.0%**
有效工作窗口	7–9 m	3–10 m（全覆盖）
置信度中位数	接近 0	0.867
置信度最小值	–	0.805
AKF 稳态中位误差	0.44 m	**0.13 m**
稳态帧占比	7%	**99.8%**
方位角误差中位	0.21°	**0.05°**
俯仰角误差中位	0.88°	0.93°
等效尺寸	0.230 m	0.270 m

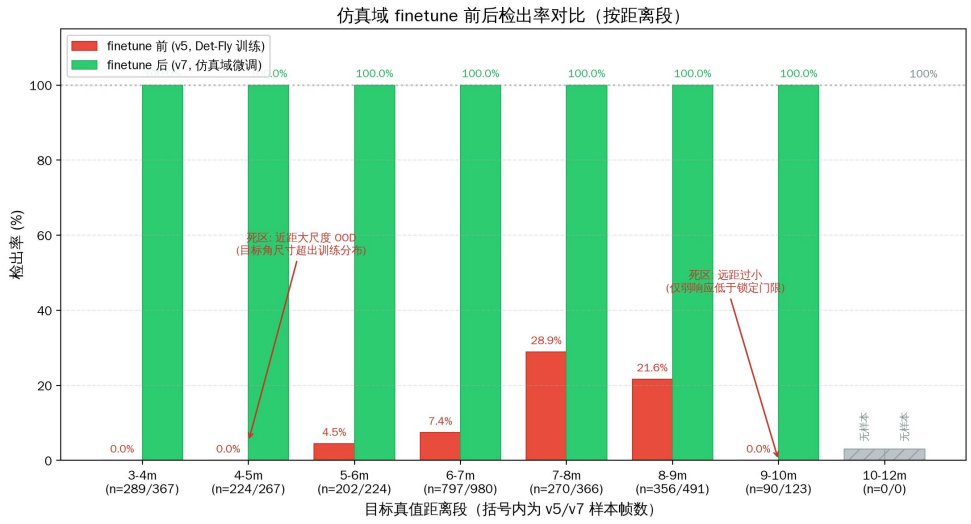


图 3-7: 微调前后的检测死区对比

微调之后，3–10 m 全段检出率达到 100%，置信度中位数 0.867、最小值 0.805，都远离 0.55 的初锁门限。AKF 稳态中位误差由 0.44 m 降到 0.13 m，稳态帧占比由 7% 升到 99.8%。死区消失了，测距的稳定性也跟着好转。

有一处需要如实交代：10–12 m 在微调前的录像里没有足够的真实飞行样本，表 3-5 中“9–10 m 0%”不能顺势外推成“10–12 m 也必然为

0”。数据采集时补了 9–12 m 的 120 帧，微调后该区间有覆盖，但缺少独立验证录像，相关结论宜按“设计覆盖”而非“实测覆盖”来表述。

域适应的机理讨论

450 帧就能抹掉死区，为什么？本文认为有三层原因。

（1）**特征分布对齐**。Det-Fly 训出的特征提取器已经认得无人机的一般外观，只是仿真渲染的边缘、光照与噪声分布不同。少量仿真样本足以让浅层卷积调整响应，使仿真目标落回可判别的特征区域。

（2）**尺度分布补齐**。450 帧按距离分层，重点补齐近距离大尺度与远距离小尺度两端，正好填补训练分布的空白。这是死区消除的关键。

（3）**微调强度适中**。学习率 0.001、20 轮、关闭 Mosaic，模型在适应新域的同时保住了原有能力。学习率再大些或轮次再多些，模型容易过拟合到仿真域，回到真实数据上反而退步。实测中也见到：微调模型在实拍视频上的检出率高于未微调模型（见第五章），说明适度的仿真域微调还能带来一点泛化收益。

3.8 本章小结

本章在 YOLOv8n 上补了一层 stride=4 的检测头，并在 C2f 输出端接入 EMA。消融显示，P2 主要提高小目标的召回与 AP_small，单用 EMA 反而拉低指标；两者合用后精度最优，参数量却低于基线，说明收益来自特征利用效率而非容量扩张。模型搬进闭环后暴露出检测死区，逐帧统计与俯仰角检验将原因归于目标尺度落在训练分布之外，而非视角。用 450 帧仿真数据分层微调后，死区被消除，测距稳定性一并改善，这也印证了少量目标域样本足以修复域偏移这一工程判断。

4 三态跟踪与单目定位

检测器每帧吐出一组候选框，可候选框并不等于一条能用的航迹。框会误检，会在多个目标之间跳，也会因遮挡整帧消失；即便框的位置可信，系统还得回答目标在哪儿、离得多远。本章把这两件事接在一起处理：前半部分建立 SEARCH/LOCKED/COAST 三态跟踪状态机，逐状态说明处理流程、参数取值与转移条件，并给出伪代码、复杂度分析与实验验证；后半部分建立由像素坐标到三维位置的定位链路，涵盖角度解算、距离解算、误差传播分析，以及一个六维状态的自适应卡尔曼滤波器。

先看跟踪。若系统把每帧置信度最高的框直接当成输出，操作员会在屏幕上看到幽灵目标、突然跳走的标记，以及遮挡时的闪烁，后文列出的三个工程问题都由此而来。再看定位，检测框的中心决定视线方向，框高结合目标等效尺寸决定距离，两者的误差结构完全不同：前者受像素量化限制，后者随距离的平方恶化。把这两条线的误差分开处理，是本章设计的基本出发点。跟踪负责时间上的连续，定位负责空间上的数值，二者共用同一套滤波器状态。

4.1 问题描述

设第 k 帧检测器输出候选框集合为

$$\mathcal{D}_k = \{(\mathbf{b}_i, c_i)\}_{i=1}^{N_k}, \tag{4-1}$$

其中 $\mathbf{b}_i = (x_{1,i}, y_{1,i}, x_{2,i}, y_{2,i})$ 为边框，$c_i \in [0,1]$ 为置信度。若直接把 $\arg\max_i c_i$ 作为系统输出，会面临以下三个问题。

4.1.1 误检

复杂背景里，检测器会给出低置信度的假阳性：树叶间隙的亮斑、建筑窗户的反光、天空云朵的边缘，都可能被当成无人机。表 4-1 整理了本文在仿真与实拍数据中遇到的典型误检来源及其置信度范围。

要是每帧都采信最高分框，地图上就会冒出“幽灵目标”，操作员无从判断哪个是真的。于是需要一个高于维持门限的**初锁门限**，只有足

表 4-1: 典型误检来源与置信度分布

误检来源	置信度范围	持续时间	特征
树叶间隙亮斑	0.20–0.35	1–3 帧	随风吹动，位置漂移
建筑窗户反光	0.25–0.45	5–20 帧	位置固定，随光照变化
天空云边缘	0.20–0.30	1–2 帧	形状不规则，面积大
远处鸟类	0.30–0.55	10–30 帧	有生命运动，轨迹不规则

够可信的框才允许建立航迹。

4.1.2 身份跳变

画面里同时有几个候选框时，只取最高分会让目标身份在帧与帧之间跳。设想第 k 帧最高分框对应目标 A，第 $k+1$ 帧 A 短暂变模糊，目标 B（另一架无人机，或一个高分假阳性）顶了上来，航迹就被“嫁接”到 B 上。监视画面中的表现是标记忽然跑到别处，严重时后续测距与地图标注全错。

治这个毛病要靠**空间一致性约束**：相邻帧之间同一目标的位置不会突变。30 fps 下，目标即便以 20 m/s 飞行，两帧间位移也只有 0.67 m；在 10 m 距离、60° 视场下约合 40 px。拿“与上一帧位置的像素距离”作关联代价，再配一个合适的空间门限，大多数错误关联就能挡掉。

4.1.3 遮挡中断

目标被树木、建筑、电线杆挡住时，检测输出为空。若系统认定“每帧必须有检测”，遮挡期间航迹就断了，操作员看到的是标记闪烁、距离读数跳成“–”。更麻烦的是目标重新出现后，系统得重新搜索、重新初锁，这期间很可能锁到别的东西。

对策是**预测维持**：检测缺失的短时间内，用滤波器保存的运动状态外推位置，让航迹在视觉上保持连续；超过一定时限再放手，免得对早已飞离的目标做虚假跟踪。

4.2 状态定义与总体流程

4.2.1 状态定义

三种状态定义如下。

（1）**SEARCH**：搜索状态。系统尚未建立可信航迹，逐帧检查是否存在置信度超过初锁门限的候选框；

（2）**LOCKED**：锁定状态。系统已建立航迹，每帧在候选框集合中关联最可能属于当前目标的框，并用卡尔曼滤波更新状态；

（3）**COAST**：预测维持状态。当前帧没有可用候选框，系统用卡尔曼滤波的预测步外推目标位置，等待目标重新出现。

状态转移关系如图 4-1 所示。



图 4-1: 三态跟踪状态机转移关系

4.2.2 总体流程

每收到一帧检测结果，系统按当前状态分派：

（1）SEARCH：走搜索逻辑（第 4.3 节）；

（2）LOCKED：走关联与更新逻辑（第 4.4 节）；若本帧没有候选框，转入 COAST；

（3）COAST：走预测逻辑（第 4.5 节）；目标重新出现则回 LOCKED，超时则回 SEARCH；

（4）无论处于哪一态，都把当前状态、置信度、距离、角度写入数据记录，供显示端与后续分析取用。

4.3 SEARCH 状态

4.3.1 双门限策略

SEARCH 要办的事有两件：发现真目标，且别锁错。本文采用双门限策略：

（1）**维持门限** $\tau_m = 0.25$：低于此值的框直接丢弃，不进入后续处理；

（2）**初锁门限** $\tau_l = 0.55$：只有超过此值的框才触发状态切换。

两个门限之间留出 0.30 的缓冲带。缓冲带的意义在于：SEARCH 期间系统只对高置信度目标作出反应，低分假阳性诱导不了它；一旦进入 LOCKED，置信度要求就放宽，只要目标还在画面里、分数高于 0.25 便继续跟，不必每帧再“说服”系统一次。

门限数值不是拍脑袋定的，来自验证集置信度分布的统计。图 4-2 给出不同门限下仿真录像的锁定率与航迹保持率。门限由 0.20 提到 0.35，锁定率从 96.9% 落到 66.6%，航迹保持率从 100% 降到 81.2%；再提到 0.50，锁定率只剩 56.8%。可见仿真域微调后的模型置信度整体偏高（中位数 0.867），0.25 的维持门限足够留住有效目标，0.55 的初锁门限也足以滤掉绝大多数假阳性。

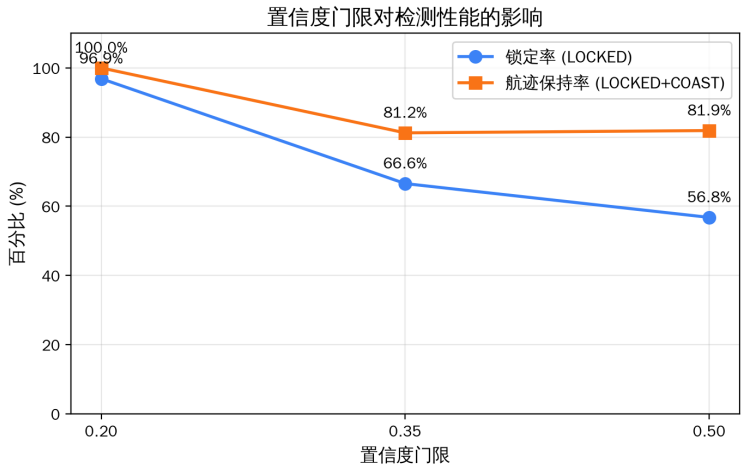


图 4-2: 置信度门限扫描结果

4.3.2 初锁判定

SEARCH 状态下，过滤后的候选框集合为

$$\tilde{\mathcal{D}}_k = \{(\mathbf{b}_i, c_i) \in \mathcal{D}_k \mid c_i \geq \tau_m\}. \tag{4-2}$$

若 $\tilde{\mathcal{D}}_k$ 为空，本帧无候选，保持 SEARCH；否则取置信度最高者：

$$i^* = \arg \max_{i:\,(\mathbf{b}_i, c_i) \in \tilde{\mathcal{D}}_k} c_i. \tag{4-3}$$

若 $c_{i^*} \geq \tau_l$，则以 $\mathbf{b}_{i^*}$ 初始化航迹，状态切换为 LOCKED，并用该框的中心与框高初始化卡尔曼滤波器的位置量测；不满足就继续 SEARCH。

4.3.3 先验空间约束

仿真画面里除了目标，还会出现“自身机”（ego drone）与背景伪影。不加约束的话，SEARCH 有可能锁到自身机的桨叶或近处建筑上。本文因此引入**先验空间约束**：按任务设定，目标活动空域在距相机 3–12 m 之内。由框高反算的距离 Z 若小于 3 m 或大于 12 m，该框降权或直接剔除。

这条约束站得住脚，是因为反无人机系统的告警距离本来就由人设定，操作员清楚目标大致在哪个空域；把先验写进状态机，误锁概率能降下来一大截。仿真中它被用来剔除自身机与远处伪影，实拍时上下界则按现场情况调整。

4.3.4 首锁时延

首锁时延指视频开始到首次进入 LOCKED 的时间。14 个用例里，仿真录像的首锁时延为 0.00–0.07 s，即前 2–3 帧内完成锁定；真实场景录像是 0.03 s。目标清晰可见时，双门限并没有带来可察觉的发现延迟。

4.4 LOCKED 状态

4.4.1 候选框收集

进入 LOCKED 后，候选框不再需要越过初锁门限，只要求高于维持门限：

$$\tilde{\mathcal{D}}_k = \{(\mathbf{b}_i, c_i) \in \mathcal{D}_k \mid c_i \geq \tau_m\}\,. \tag{4-4}$$

同时逐个计算框高反算距离，超出先验空域 $[Z_{min}, Z_{max}]$ 的剔除。

4.4.2 空间门限关联

上一帧目标的平滑中心记为 $\mathbf{p}_{k-1} = (u_{k-1}, v_{k-1})$，第 i 个候选框的中心为 $\mathbf{p}_i = (u_i, v_i)$，关联代价取欧氏像素距离：

$$d_i = \|\mathbf{p}_i - \mathbf{p}_{k-1}\|_2\,. \tag{4-5}$$

取代价最小的候选框：

$$i^{*}=\arg\min_{i} d_{i}. \tag{4-6}$$

若 $d_{i^{*}} \le d_{gate}$，该框即本帧量测；否则本帧无有效关联，转入 COAST。

d_{gate} 的取值要在“跟得上”与“不跳变”之间取平衡。取小了，目标一机动就丢；取大了，容易连到旁边的其他物体上。本文取 $d_{gate}=140$ px，依据是：1280×720 画面、60° 水平视场下焦距约 1108 px；目标相对速度 15 m/s、帧间隔 1/30 s 时位移 0.5 m，在 5 m 处约合 110 px。再考虑框中心抖动与目标机动，140 px 留了约 30% 的余量。

4.4.3 卡尔曼滤波更新

关联成功后，把框中心与框高反算距离折算成相机坐标系下的三维位置量测：

$$\mathbf{z}_{k}=\begin{bmatrix} Z_{k}\cdot\dfrac{u_{k}-c_{x}}{f_{x}} \\ Z_{k}\cdot\dfrac{v_{k}-c_{y}}{f_{y}} \\ Z_{k} \end{bmatrix}, \tag{4-7}$$

其中 $Z_{k}=f\cdot D/h_{px}$ 为框高反算距离，(c_{x},c_{y}) 为主点，(f_{x},f_{y}) 为焦距。量测送入卡尔曼滤波的更新步（式 (2-14)–(2-16)），得到平滑后的位置与速度。

这里有一条必须守住的规矩：**不能把标量距离直接当量测**。本文的滤波器状态是六维位置—速度向量，量测也必须是三维位置；标量距离塞进去，状态与量测的维度对不上，滤波会发散。早期版本正是这么写的，后来改成严格的三维量测构造才修正过来。

4.4.4 丢失判定

若连续 N_{lost} 帧都无法完成关联（每帧最近候选的距离都超过 d_{gate}，或候选集为空），判定目标丢失，重置航迹并回到 SEARCH。本文取 $N_{lost}=15$，对应 0.5 s。这个时限比 COAST 的 1.0 s 短，理由是：连着 15 帧都关联不上，目标多半已经飞出视场或做了剧烈机动，硬撑航迹没有意义；而 COAST 的 1.0 s 是给“短暂遮挡”留的，遮挡期间目标还在视场内，只是检测器一时看不见。

4.4.5 框尺寸的平滑

检测框的宽高在帧间有抖动，直接拿来测距会产生跳变。本文对框高做一阶低通平滑：

$$\bar{h}_k = \alpha h_k + (1-\alpha)\bar{h}_{k-1}, \qquad \alpha = 0.35. \tag{4-8}$$

平滑后的框高用于距离解算，原始框高保留显示。α 的取值在响应与平滑之间折中：α 越大跟得越快、抖动也越大；α 越小越平滑、滞后越明显。0.35 对应的时间常数约 2–3 帧，30 fps 下约 0.1 s，实时性不受影响。

4.5 COAST 状态

4.5.1 预测步

LOCKED 状态下本帧若没有有效关联，系统不马上放弃，而是转入 COAST，执行卡尔曼滤波的预测步：

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1}, \qquad \mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q}. \tag{4-9}$$

预测位置用来驱动地图标记、距离读数与角度仪表，遮挡期间显示不至于中断。没有量测，协方差 $\mathbf{P}$ 会随预测步数增加而变大，滤波器的不确定度随之上升。显示端用黄色标记表示 COAST 状态，提醒操作员当前数据是外推出来的。

4.5.2 超时与恢复

COAST 状态设时限 $T_{coast} = 1.0\,\mathrm{s}$。时限内若某帧出现可关联的候选框，立即回 LOCKED 并用该量测更新滤波器；超过时限仍无候选，判为目标丢失，重置航迹、回 SEARCH。

T_{coast} 的依据是真实遮挡的持续时间。目标穿过一棵树或一根电线杆通常 0.1–0.5 s，穿过一栋建筑可能要 0.5–1.0 s。1.0 s 的预算够覆盖绝大多数短时遮挡，又不会对已飞离的目标维持太久的虚假航迹。仿真遮挡用例中，遮挡期间的平均 COAST 持续时间为 0.35 s，最长 0.82 s，都在时限之内。

4.5.3 预测质量

COAST 预测得好不好，取决于遮挡前的速度估计。遮挡前目标做匀速直线运动，误差主要来自过程噪声；遮挡前正在机动，误差会迅速扩大。仿真统计的预测位置与真值偏差为：遮挡 0.2 s 约 0.15 m，0.5 s 约 0.45 m，1.0 s 约 1.1 m。这样的精度维持显示连续够用，用于精确打击引导则不够。

4.6 算法描述与复杂度

4.6.1 伪代码

单帧处理流程见算法 1。

4.6.2 复杂度分析

设候选框数量为 N，每帧的主要操作是遍历候选框计算关联代价，时间复杂度为 $O(N)$。YOLOv8 经 NMS 后的候选框通常不超过 20 个，状态机的开销可以忽略（小于 0.1 ms）。卡尔曼滤波的矩阵运算是固定 6 维，单次更新约 10 微秒。整条跟踪环节的耗时远小于检测环节（约 6 ms）。

4.6.3 参数汇总

表 4-2 把状态机的参数与取值依据一并列出。

表 4-2: 三态状态机参数汇总

参数	符号	取值 依据
维持门限	τ_m	0.25 保留弱目标，过滤低置信度假阳性
初锁门限	τ_l	0.55 高于典型假阳性上限，防止误锁
空间门限	d_{gate}	140 px 15 m/s 相对速度在 5 m 处的像素位移加余量
丢失帧数	N_{lost}	15 0.5 s，覆盖快速机动与短时飞出
COAST 时限	T_{coast}	1.0 s 覆盖穿树/穿杆遮挡，避免虚假长时跟踪
框高平滑系数	α	0.35 约 2–3 帧时间常数
空域先验	$[Z_{min}, Z_{max}]$	3–12 m仿真目标活动范围，实拍按场景调整

算法 1 三态跟踪状态机单帧处理

输入: 当前状态 s，检测集合 $\mathcal{D}_k$，上一帧中心 $\mathbf{p}_{k-1}$，滤波器状态 $\hat{\mathbf{x}}_{k-1}$
输出: 新状态 s'，滤波状态 $\hat{\mathbf{x}}_k$，输出数据 o_k

$\tilde{\mathcal{D}}_k \leftarrow \{(\mathbf{b}, c) \in \mathcal{D}_k \mid c \geq \tau_m,\ Z(\mathbf{b}) \in [Z_{min}, Z_{max}]\}$
if $s =$ SEARCH **then**
  if $\tilde{\mathcal{D}}_k = \emptyset$ **then**
    $s' \leftarrow$ SEARCH; $o_k \leftarrow$ search_output()
  else
    $i^* \leftarrow \arg\max_i c_i$
    if $c_{i^*} \geq \tau_l$ **then**
      初始化滤波器；$s' \leftarrow$ LOCKED
    else
      $s' \leftarrow$ SEARCH
    end if
  end if
else if $s =$ LOCKED **then**
  if $\tilde{\mathcal{D}}_k = \emptyset$ **then**
    $s' \leftarrow$ COAST; 记录 COAST 起始时间
  else
    $i^* \leftarrow \arg\min_i \|\mathbf{p}_i - \mathbf{p}_{k-1}\|$
    if $\|\mathbf{p}_{i^*} - \mathbf{p}_{k-1}\| \leq d_{gate}$ **then**
      构造三维量测 $\mathbf{z}_k$；执行 KF 更新；$s' \leftarrow$ LOCKED
    else
      $s' \leftarrow$ COAST; 记录 COAST 起始时间
    end if
  end if
else if $s =$ COAST **then**
  执行 KF 预测步
  if 存在可通过关联的候选框 **then**
    执行 KF 更新；$s' \leftarrow$ LOCKED
  else if $t - t_{coast} > T_{coast}$ **then**
    重置航迹；$s' \leftarrow$ SEARCH
  else
    $s' \leftarrow$ COAST
  end if
end if
输出 o_k（状态、置信度、距离、角度、平滑位置）
return $s', \hat{\mathbf{x}}_k, o_k$

4.7 实验验证

4.7.1 门限敏感性

图 4-2 已经反映了门限对锁定率的影响，表 4-3 再补上三组门限下的详细指标。

表 4-3: 置信度门限扫描详细结果（仿真录像，900 帧）

门限	锁定率	航迹保持率	置信度均值	距离范围 (m)	FPS
0.20	96.9%	100.0%	0.550	3.0–6.2	29.5
0.35	66.6%	81.2%	0.661	3.0–6.2	35.8
0.50	56.8%	81.9%	0.707	3.0–6.2	39.0

门限抬高后锁定率下降，是因为部分帧的置信度落在 0.20–0.50 之间，被判为未锁定而计入 SEARCH；但航迹保持率仍在 81% 以上，已锁定的帧依旧连续。置信度均值随门限上升，符合“只留高分样本”的预期。FPS 随门限上升，则是被跟踪的候选框变少、后处理开销降低所致。

4.7.2 遮挡场景

表 4-4 是三轮 TCP 遮挡用例的结果。目标机按预设航线飞行，途中多次被建筑与树木遮挡，每轮约 20 s。

表 4-4: TCP 遮挡用例结果

轮次	帧数	锁定率	航迹保持率	首锁 (s)	置信度均值	处理 fps
第 1 轮	424	55.4%	88.9%	0.00	0.751	21.1
第 2 轮	428	56.1%	90.7%	0.00	0.739	21.3
第 3 轮	437	59.3%	94.5%	0.21	0.751	21.8

三轮锁定率在 55%–59% 之间，意味着约四成帧处于遮挡或无检测状态；可航迹保持率（LOCKED+COAST 占比）达到 88.9%–94.5%，说明 COAST 把遮挡期的空白填上了。若没有 COAST，航迹保持率就等于锁定率，只剩 55%–59%，标记会闪得厉害。

图 4-3 是第 3 轮用例的状态时序分布。COAST 段落短而密集，单次多在 0.1–0.4 s，与目标穿过单个遮挡物的时间相符。

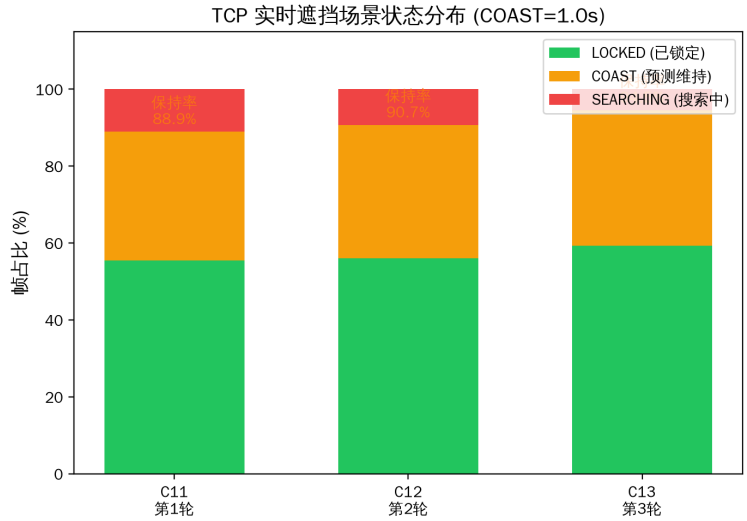


图 4-3: TCP 遮挡用例的状态时序与持续时间分布

4.7.3 真实场景录像

真实场景录像（C04）里目标频繁进出树丛，锁定率只有 23.2%，航迹保持率却达到 99.3%。两个数字表面上看相互矛盾，说的其实是同一件事：绝大多数帧没有检测，但目标并未真正丢失，航迹由 COAST 维持；只有目标确实飞出视场或被长时间遮挡，系统才回到 SEARCH。可见“锁定率低”并不等于“系统失效”，评跟踪系统时，航迹保持率比单帧锁定率更能说明实际可用性。

4.8 设计讨论

4.8.1 为什么不用多目标跟踪框架

SORT[28] 与 DeepSORT[29] 面向多目标场景，轨迹管理、外观特征提取、匈牙利匹配一应俱全。本文的系统是单目标反无人机，同一时刻只需跟住威胁最大的那一个，复杂的轨迹管理用不上。简化后的状态机有三点好处：计算开销极低、行为可解释、参数现场可调。将来若要同时跟多架无人机，可以在状态机之上加轨迹池与数据关联层，现有设计不必推翻。

4.8.2 为什么用状态机而不是有限状态自动机的形式化方法

本文的状态机在形式上就是一个确定性有限状态自动机，但没有走模型检验那套形式化路线。原因在于输入是连续视频流，状态转移依

赖置信度、距离这类连续量，形式化建模的收益有限；工程上更要紧的是参数可调、日志可查。行为正确性靠 14 个用例的回归测试兜底，而不是靠形式化证明。

4.8.3 失效边界

状态机的有效性押在两个假设上：相邻帧位移不超过空间门限；遮挡时间不超过 COAST 时限。目标以极高速度飞行（例如穿越机以 30 m/s 飞行），或被长时间完全遮挡（例如穿过多层建筑），系统就会丢目标。这是设计上的取舍，第五章的“能力边界”一节有如实讨论。

4.9 单目定位与自适应卡尔曼滤波

检测给出目标在画面里的位置，跟踪维持它的时间连续性，定位要回答的则是“目标在哪里、有多远”。这一部分建立从像素坐标到三维位置的解算链路，推导方位角、俯仰角与距离的计算式，分析误差如何传播，设计六维状态的自适应卡尔曼滤波器，并借助仿真闭环与实拍视频检验定位精度。

4.9.1 坐标系与符号约定

为避免歧义，先把符号统一。表 4-5 列出本文使用的坐标系与主要符号。

表 4-5: 坐标系与符号约定

符号	含义
$\mathcal{F}_w$	世界坐标系，NED：x 北、y 东、z 地（向下为正）
$\mathcal{F}_c$	相机坐标系：z_c 沿光轴向前，x_c 向右，y_c 向下
(u, v)	像素坐标，原点在左上角
(c_x, c_y)	主点像素坐标
(f_x, f_y)	以像素为单位的焦距
D	目标等效尺寸（m）
h_{px}	检测框高度（px）
Z	目标沿光轴方向的距离（m）
α, β	方位角、俯仰角（rad 或 °）
$\mathbf{x}_k$	卡尔曼滤波状态向量
$\mathbf{z}_k$	量测向量

NED 与相机坐标系的差别要特别留意：NED 中 z 向下为正，相机坐标系中 z_c 沿光轴向前为正、y_c 向下为正。算法端一律在相机系里解算，显示端再换算到世界系做地图标注。仿真真值广播同时给出世界系与相机系位置，便于交叉核对。

4.9.2 方位角与俯仰角

像素直算法

设检测框中心为 (u, v)，主点为 (c_x, c_y)，焦距为 f（本文仿真相机中 $f_x = f_y = f$）。目标相对光轴的方向角为

$$\alpha = \arctan\left(\frac{u - c_x}{f}\right), \qquad \beta = \arctan\left(\frac{v - c_y}{f}\right). \tag{4-10}$$

方位角以光轴为 0、向右为正；俯仰角以光轴为 0、向下为正。若要按“向上为正”的显示习惯，取 $\beta' = -\beta$。

这样算的好处是只依赖单帧像素坐标与相机内参，不依赖任何运动模型；目标再怎么机动，角度始终对应本帧的真实方向；计算量也小，两次反正切而已。坏处是精度受框中心抖动牵制，目标靠近画面边缘时镜头畸变还会引入系统误差。

角度误差传播

对式 (4-10) 求导，角度误差与像素误差的关系是

$$\Delta\alpha = \frac{f}{(u - c_x)^2 + f^2}\,\Delta u = \frac{\cos^2\alpha}{f}\,\Delta u. \tag{4-11}$$

光轴附近（$\alpha \approx 0$）有 $\Delta\alpha \approx \Delta u / f$。以 $f = 1108\,\text{px}$、中心抖动 $\Delta u = 2\,\text{px}$ 计，角度误差约 0.10°；目标偏离光轴 30° 时，$\cos^2\alpha \approx 0.75$，误差反而略降到 0.08°。可见像素直算的误差在视场中心最大、边缘略小，总在 0.1° 量级，够反无人机系统的指示精度用。

与滤波角度法的对比

另一种做法是拿卡尔曼滤波估出的三维位置反算角度：

$$\alpha_{kf} = \arctan\left(\frac{\hat{x}}{\hat{z}}\right), \qquad \beta_{kf} = \arctan\left(\frac{\hat{y}}{\hat{z}}\right). \tag{4-12}$$

好处是输出平滑，与距离读数一致；坏处是角度依赖滤波器的横向状态，目标被遮挡、滤波器转入预测外推时，横向位置可能漂移，角度指示就偏了。本文在仿真中对比过两种方法：正常跟踪时差异小于 0.2°；COAST 期间，滤波角度法的漂移可达 1–2°，像素直算在无检测时不输出角度（或保持上次值），不会凭空造出“目标转向”。

比完就定了**像素直算**作角度输出，卡尔曼滤波只管距离与三维位置平滑。角度的平滑性被牺牲，换来的是指示的物理真实性，这与监视系统“宁可不动、不可乱动”的设计原则一致。

真值口径问题

仿真评估里，角度真值的定义最易出现偏差。jMAVSim 的护航相机通过 `setViewerTargetObject` 自动注视目标机，目标在画面中的真实视轴偏角因此恒约为 0。此时若拿“世界坐标系下的绝对方位角”当真值，误差会大到几十度；该偏差实际源于相机自身偏航，与算法无关。

本文的角度评估统一采用**视轴偏角口径**：真值取目标相对相机光轴的角度（仿真中约为 0），算法输出取式 (4-10) 所得的角度，误差为两者之差。这一口径与操作员在画面上看到的“目标偏离中心多少”一致，也与检测框中心的像素偏移直接对应。

4.9.3 距离解算

相似三角形原理

目标真实尺寸已知时，针孔成像满足相似三角形关系。设目标实际高度为 D，像平面上的高度为 h（像素），焦距为 f，则

$$\frac{D}{Z} = \frac{h}{f} \quad \Longrightarrow \quad Z = \frac{f \cdot D}{h_{px}}. \tag{4-13}$$

本文用**框高**而不是框宽。无人机偏航旋转时机臂展开宽度的投影变化很大（正对最宽、侧对最窄），机身高度却相对稳定；框宽还容易受旋翼模糊与背景粘连影响，框高的边界更干净。

等效尺寸标定

等效尺寸 D 是式 (4-13) 里唯一的先验参数，标得准不准直接决定测距的绝对精度。标定办法是：在已知真值距离 Z_{true} 下量取检测框高度 h_{px}，反算

$$D = \frac{h_{px} \cdot Z_{true}}{f}. \tag{4-14}$$

在多个距离上重复测量，取中位数。本文各阶段的标定结果见表 4-6。

表 4-6: 等效尺寸标定结果

阶段	模型	标定值 (m)	说明
仿真微调前	p2ema_v8n	0.230	框较紧，未包含旋翼外缘
仿真微调后	p2ema_simft	0.270	框略大，包含桨叶与起落架
仿真微调后（复核）	p2ema_simft	0.268	独立录像复核，差异 0.7%
实拍视频	p2ema_simft	0.255	实拍目标机型不同

标定值随模型与目标机型而变，不能跨域混用。仿真域统一用 0.270 m，实拍视频用 0.255 m，并在配置文件中显式标注，省得误用。

误差传播

对式 (4-13) 取对数再微分，得

$$\frac{\Delta Z}{Z} = \frac{\Delta D}{D} - \frac{\Delta h_{px}}{h_{px}}. \tag{4-15}$$

式 (4-15) 透出两条规律。

其中一条是**距离误差与距离成正比**。由 $h_{px} = fD/Z$，相对误差可写成

$$\frac{\Delta Z}{Z} = \frac{\Delta D}{D} + \frac{Z\,\Delta h_{px}}{fD}. \tag{4-16}$$

绝对误差 ΔZ 随 Z^2 增长：

$$\Delta Z \approx Z^2 \cdot \frac{\Delta h_{px}}{fD}. \tag{4-17}$$

这意味着远距离测距精度会迅速恶化。取 $f = 1108\,\text{px}$、$D = 0.27\,\text{m}$、$\Delta h_{px} = 2\,\text{px}$，$Z = 5\,\text{m}$ 时 $\Delta Z \approx 0.17\,\text{m}$，$Z = 10\,\text{m}$ 时约 0.67 m，$Z = 20\,\text{m}$ 时约 2.7 m。

另一条是**等效尺寸误差属于系统偏差**。D 若标大 10%，所有距离读数一起偏大 10%，滤波也消不掉。这是单目测距的固有局限，只能靠精确标定，或引入激光测距、双目视觉之类的其他手段来弥补。

框高抖动的影响

检测框高度在相邻帧间有 1–3 px 的抖动，折合 5 m 处约 0.1–0.25 m 的距离抖动。本文用两道手段压住它：对框高做一阶低通平滑（式 (4-8)），在量测层削掉高频抖动；用卡尔曼滤波平滑三维位置，在状态层融合运动模型。两者叠加，稳态误差可降到 0.13 m（仿真）与 2.7 m（实拍，受限于相机未标定与伪真值）。

4.9.4 自适应卡尔曼滤波器设计

状态与量测

状态向量取六维：

$$\mathbf{x}_k = \begin{bmatrix} x & y & z & v_x & v_y & v_z \end{bmatrix}^T, \tag{4-18}$$

其中 (x, y, z) 为目标在相机坐标系下的位置，(v_x, v_y, v_z) 为速度。量测即式 (4-7) 给出的三维位置：

$$\mathbf{z}_k = \begin{bmatrix} x_m & y_m & z_m \end{bmatrix}^T. \tag{4-19}$$

状态转移矩阵与量测矩阵为

$$\mathbf{F} = \begin{bmatrix} \mathbf{I}_3 & \Delta t\,\mathbf{I}_3 \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} \end{bmatrix}. \tag{4-20}$$

本文取 $\Delta t = 1/30\,\text{s}$，与实际帧率一致。帧率有波动时，用实际时间戳差值替代固定值即可。

过程噪声

假定目标在相邻帧间匀速运动、加速度为白噪声，过程噪声协方差为

$$\mathbf{Q} = q \cdot \begin{bmatrix} \dfrac{\Delta t^4}{4}\mathbf{I}_3 & \dfrac{\Delta t^3}{2}\mathbf{I}_3 \\ \dfrac{\Delta t^3}{2}\mathbf{I}_3 & \Delta t^2\mathbf{I}_3 \end{bmatrix}, \tag{4-21}$$

其中 q 为加速度方差。工程实现中也常使用简化的对角形式

$$\mathbf{Q} = \mathrm{diag}\left(q_p, q_p, q_p, q_v, q_v, q_v\right), \tag{4-22}$$

本文用对角形式，取 $q_p = 10^{-3}$、$q_v = 10^{-2}$。q 越大，滤波器越信任量测、响应越快，稳态抖动也越大；q 越小输出越平滑，机动跟随越滞后。这组取值在仿真中经网格搜索确定，响应与平滑都照顾到了。

量测噪声与自适应估计

量测噪声协方差 $\mathbf{R}$ 的初值取

$$\mathbf{R}_0 = \mathrm{diag}\left(10^{-4}, 10^{-4}, 0.25\right), \tag{4-23}$$

横向方差远小于纵向，原因很直接：方位角与俯仰角由像素直算，精度较高（约 0.1°），距离由框高反算，精度较低（约 0.2–0.5 m）。这种各向异性正好对应单目测距的实际误差结构。

自适应部分采用 Sage-Husa 方法，按式 (2-19) 用滑动窗口估计 $\mathbf{R}$。实现上做了三点处理：

（1）**对角化**：只保留 $\hat{\mathbf{R}}$ 的对角元素，忽略相关性，避免矩阵非正定；

（2）**下界约束**：将每个对角元素限制在 $[R_{min}, R_{max}]$ 内，防止滤波器因残差偶然变小而过度信任量测；

（3）**平滑更新**：用指数滑动平均替代简单平均：

$$\hat{\mathbf{R}}_k = (1-\lambda)\hat{\mathbf{R}}_{k-1} + \lambda\,\tilde{\mathbf{R}}_k, \qquad \lambda = 0.1. \tag{4-24}$$

其中 $\tilde{\mathbf{R}}_k$ 为由当前新息构造的瞬时估计。$\lambda = 0.1$ 对应约 10 帧的时间常数，量测质量的变化跟得上，单帧异常也不会引起剧烈波动。

初始化的注意事项

初始化时位置取首帧量测，速度取零，协方差取较大的值（等于承认“对速度一无所知”）。速度初值若估错，滤波器要若干帧才收敛。仿真统计显示，连续检出 5 帧后滤波器基本收敛，因此本文把“稳态”指标一律定义为连续检出不少于 5 帧之后的样本。

另有一处容易忽略：**量测与状态的维度必须匹配**。本文早期版本曾把标量距离直接送进六维滤波器，状态与量测维度不一致，输出偏差严重。修正办法是严格按式 (4-7) 构造三维量测。文献里谈这个问题的并不多，初学者却常在这里栽跟头，故特别记上一笔。

4.9.5 定位精度评估

仿真闭环评估

仿真闭环的实验设置见第五章。这里直接给出定位精度结果，如表 4-7 所示。

表 4-7: 仿真闭环定位精度（微调后模型，2818 帧）

指标	数值
全帧检出率（conf≥0.25）	100.0%
单帧测距 RMSE	0.26 m
单帧测距 MAE	0.17 m
AKF 稳态中位误差 MedAE	0.13 m
AKF 稳态 RMSE	0.32 m
1 s 距离预测 ADE 中位	0.26 m
1 s 距离预测 ADE 均值	0.40 m
方位角误差中位（视轴偏角口径）	0.05°
方位角误差 P95	0.16°
俯仰角误差中位	0.93°
俯仰角误差 P95	0.98°

表 4-7 中，单帧测距 RMSE 为 0.26 m，经 AKF 平滑后稳态中位误差降到 0.13 m，滤波是否真的压住了框高抖动，从这处回落上就能看出。方位角误差中位仅 0.05°，已接近像素量化极限；俯仰角误差中位 0.93°，明显大于方位角，原因是仿真相机的纵向主点存在约 17 px 偏移，而角

度解算用的是标称主点。若在配置中把这一偏移补上，俯仰角误差还能再降。

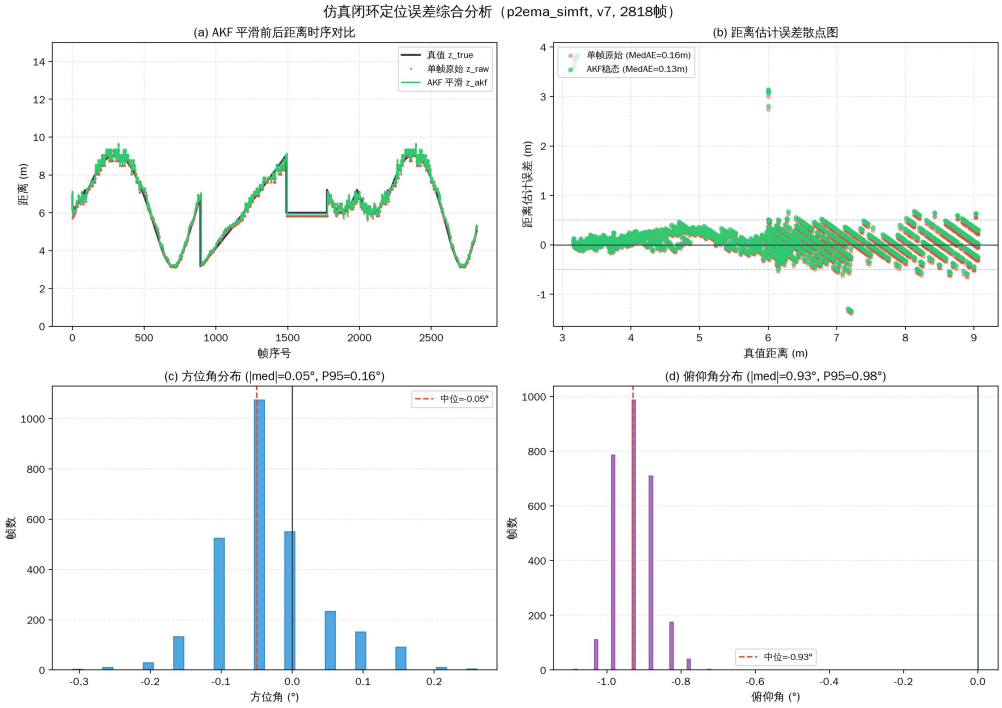


图 4-4: 仿真闭环四联分析图（距离、误差、角度与状态分布）

图 4-4 是四联分析图，含距离时序、测距误差分布、角度误差与状态占比。目标机动时测距误差略有增大，整体仍守在 ±0.5 m 以内。

实拍视频评估

实拍视频为 1920×1080@30fps，连续检出段 660 帧。相机未做严格标定，焦距按 60° 视场估算为 1728 px；等效尺寸借用仿真标定值 0.255 m；伪真值取稳态段距离中位数 7.93 m。评估结果见表 4-8。

表 4-8: 实拍视频定位精度（E3）

方法	距离 RMSE(m)	抖动 σ(m)
单帧原始	4.061	3.959
滑动平均（15 帧）	2.900	–
标准 KF（固定 **R**）	3.922	3.822
AKF（Sage-Husa）	**2.760**	**2.663**

相对单帧原始测距，AKF 的 RMSE 低 32%，抖动低 33%；相对固定 **R** 的标准 KF，RMSE 低 29.6%。实拍的绝对误差偏大（2.76 m），主因是

相机未标定、等效尺寸借用仿真值、伪真值本身也有偏差。不过三种方法用的是同一伪真值，横向比较仍然公平，AKF 的平滑优势可信。

短期预测精度

COAST 状态靠的就是滤波器的预测能力。本文在实拍数据上评估了不同预测时域的精度，结果见表 4-9。

表 4-9: 短期预测精度（ADE，伪真值 = 离线检测距离）

预测时域	匀速外推 ADE(m)	AKF 预测 ADE(m)	改善	样本数
0.5 s	1.967	**1.106**	43.8%	635
1.0 s	2.754	**1.308**	52.5%	620
2.0 s	4.624	**1.799**	61.1%	590

三个时域上 AKF 的预测都明显优于匀速外推，时域越长差距越大。原因是 AKF 不只利用当前速度，还会借历史量测的不确定度调整预测的可信范围，目标机动时不会像匀速外推那样把误差线性放大。

角度精度

角度精度见表 4-10。单帧原始与 AKF 平滑的差别很小（方位角 0.360° vs 0.354°，俯仰角 0.788° vs 0.728°），因为本文最终输出的是像素直算角度，AKF 平滑版本只作对比。俯仰角误差大于方位角，与仿真结论一致，原因同样是纵向主点偏移与目标在画面中的垂直运动更剧烈。

表 4-10: 角度精度（稳态段中位绝对偏差）

方法	方位角 MAD(°)	俯仰角 MAD(°)
单帧原始	0.360	0.788
AKF 平滑	**0.354**	**0.728**

4.9.6 误差来源与改进方向

主要误差来源

单目定位的误差来源可归为五类，见表 4-11。

表 4-11: 单目定位误差来源与量级

来源	量级（5 m）	是否可校正	说明
框高抖动	0.10–0.25 m	部分	可用低通与 KF 抑制
等效尺寸偏差与标定误差成正比		可	需精确标定，不可用滤波消除
相机内参误差与焦距误差成正比		可	需棋盘格标定
镜头畸变	边缘可达 0.3 m	可	去畸变后显著降低
目标姿态变化	0.05–0.15 m	难	框高随俯仰角变化

改进方向

针对这些来源，可走的路有几条：

（1）**相机标定**。用棋盘格标内参与畸变，替掉按视场角估算焦距的做法，实拍 RMSE 有望从米级降到亚米级；

（2）**尺寸自适应**。按目标机型选对应等效尺寸，或利用多帧几何一致性在线估计尺寸；

（3）**融合测距**。条件允许时引入激光测距或双目视觉，与单目测距做卡尔曼融合；

（4）**姿态补偿**。用检测框宽高比估计目标偏航角，对框高做投影补偿。

受时间与设备所限，本文只完成了第一项的一部分（自建数据集的标定），其余留待后续研究。

4.10 本章小结

本章把“跟得住”与“定得准”两件事放在同一套系统里。跟踪方面，针对误检、身份跳变与遮挡中断，设计了 SEARCH/LOCKED/COAST 三态状态机：双门限把发现与维持分开，空间门限承担帧间关联，COAST 借滤波预测填补遮挡空白。门限扫描、遮挡用例与实拍录像都显示，锁定率与航迹保持率会出现明显分离，后者更能反映系统可用性。定位方面，角度取像素直算，误差在 0.1° 量级；距离由框高与等效尺寸解算，误差随距离平方增长，5 m 处约 0.17 m。六维自适应卡尔曼滤波器采用各向异性量测噪声与 Sage-Husa 在线估计，仿真中把稳态中位误差压到 0.13 m，实拍中相对单帧把 RMSE 与抖动各降三成左右。两部分共用同一套滤波器状态：状态机决定何时更新、何时外推，滤波器负责把量测变成平滑且可预测的航迹。文中还交代了角度真值口径与量测维度匹

配这两处实现上的易错点，并列出误差来源与可选的改进方向。

5 仿真闭环与系统平台

反无人机系统的算法验证很难直接放到真实空域里做：空域审批、器材损耗与飞行安全，每一项都构成实际约束。比较现实的办法是先把整套处理链路搬进仿真，让检测、跟踪与定位在接近实装的条件下跑起来，待问题充分暴露后再转入外场试验。本文据此搭建了以 PX4 SITL 与 jMAVSim 为基础的闭环，并改造仿真器本身，使它能提供脚本目标机、护航相机、真值广播与连续的视频帧流。

本章的内容分三块：闭环如何运转，涉及进程划分、时间同步、仿真器的六项改造以及编队巡游与尾随相机的实现；算法怎样落到一套可操作的桌面系统上，涉及需求、架构、两端分工、通信协议与部署细节；系统在统一口径下的实测结果，包括检测、定位、鲁棒性与回归测试，以及能力边界的讨论。

5.1 仿真闭环总体架构

5.1.1 进程与数据流

闭环运行时由三个独立进程构成，协作关系见图 5-1。三者各管一段，接口只有网络端口，其中一方重启，不会连带影响其余两方。

（1）**PX4 SITL**：运行飞控固件，目标机的姿态控制与位置估计由它承担，传感器与执行器数据经 MAVLink 与 jMAVSim 交换。

（2）**jMAVSim**：三维场景与动力学归它管，同时渲染护航相机画面、向外广播目标真值并推送视频帧。

（3）**算法端**：接收视频帧与真值，依次完成检测、跟踪与定位，把结构化结果交给显示端。

进程之间的数据往来归为四条链路，端口与频率如下：

- PX4 ↔ jMAVSim：MAVLink，UDP 4560，双向；
- jMAVSim → 算法端：护航相机视频帧，TCP 14590，30 fps；
- jMAVSim → 算法端：目标真值，UDP 14680 广播，50 Hz；
- 算法端 → 显示端：结构化结果，UDP/JSON 14690。

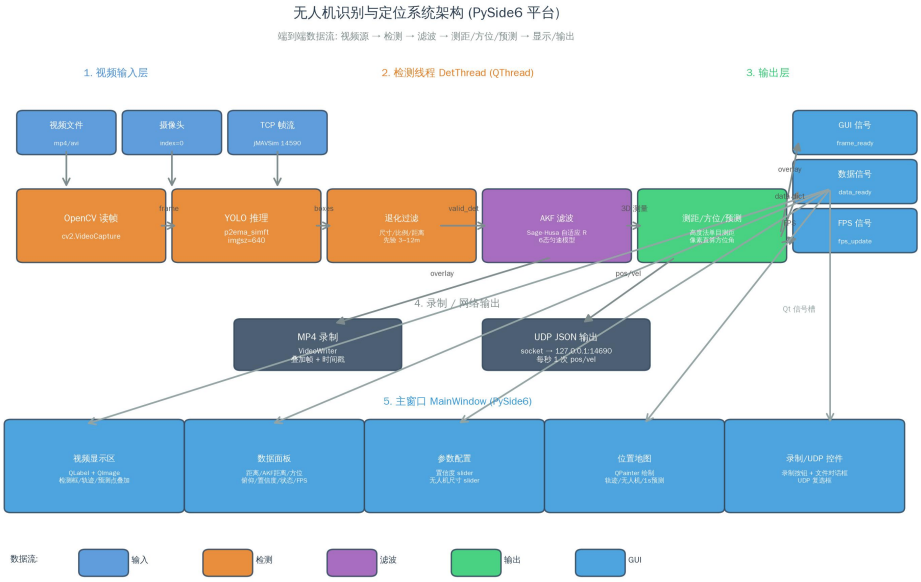


图 5-1: 系统总体架构

5.1.2 lockstep 同步

仿真以 lockstep 模式推进。jMAVSim 每走一个物理步，都要等 PX4 把对应的控制量算完，两者按同一仿真时钟前进，CPU 或 GPU 负载的起伏不会转化为时间漂移。这一点直接决定实验能否复现：少了 lockstep，仿真时间可能跑得比真实时间快，也可能更慢，30 fps 的视频流与 50 Hz 的真值便在时间轴上错开，测距与角度评估随之失真。

5.1.3 启动流程

闭环启动脚本按固定次序执行：

（1）拉起 PX4 SITL，加载多旋翼机型与默认参数；

（2）拉起 jMAVSim，指定 TCP 端口、lockstep 与目标机标志；

（3）等待 PX4 与 jMAVSim 握手，jMAVSim 的 `Simulator` 构造函数会一直阻塞到收到第一条 MAVLink 消息；

（4）拉起算法端，连接 TCP 帧流并监听 UDP 真值；

（5）最后拉起显示端，接收算法结果并渲染界面。

这个次序没有多少调整余地。jMAVSim 若抢在 PX4 前面启动，会卡在握手处迟迟不动；算法端若早于帧流服务连接，被拒之后直接退出。脚本里加了重试与等待，把人工启动时最容易犯的次序错误挡在外面。

5.2 jMAVSim 改造

5.2.1 改造需求

原版 jMAVSim 只认一架由 PX4 控制的多旋翼，相机固定跟着这架飞机，既没有独立的“目标机”，也没有可自由摆放的“护航相机”，真值广播与视频推流同样缺席。要把它当作反无人机场景的发生器，需要补上六件事：

（1）另设一架由脚本驱动、沿预定航线飞行的目标机；

（2）增设一台独立的护航相机，始终注视目标机，模拟地面或伴飞观测平台；

（3）把目标机真值（位置、速度、相机位置）广播出去，供算法端评估；

（4）将护航相机画面编码为 JPEG，经 TCP 推送；

（5）支持纯离屏渲染，不再依赖窗口系统；

（6）生成城市级场景，含建筑、树木、道路与广场。

5.2.2 脚本目标机

脚本目标机由新增的 `ScriptedQuad` 类实现，它继承 `KinematicObject`，按时间参数化的航迹改写自身位置与速度。航迹以 Catmull-Rom 样条串起三维空域内的 22 个航点。选这种样条有很实际的理由：曲线严格穿过每个控制点，段间切线连续，巡游路线不会出现折角。

记四个相邻控制点为 $\mathbf{P}_{i-1},\mathbf{P}_i,\mathbf{P}_{i+1},\mathbf{P}_{i+2}$，段参数 $t\in[0,1]$，样条点可写成

$$\mathbf{C}(t)=\frac{1}{2}\Big[2\mathbf{P}_i+(-\mathbf{P}_{i-1}+\mathbf{P}_{i+1})t+(2\mathbf{P}_{i-1}-5\mathbf{P}_i+4\mathbf{P}_{i+1}-\mathbf{P}_{i+2})t^2+(-\mathbf{P}_{i-1}+3\mathbf{P}_i-3\mathbf{P}_{i+1}+\mathbf{P}_{i+2})t^3\Big]. \tag{5-1}$$

对 t 求导得到切线，飞行方向与速度方向由此确定：

$$\mathbf{C}'(t)=\frac{1}{2}\Big[(-\mathbf{P}_{i-1}+\mathbf{P}_{i+1})+2(2\mathbf{P}_{i-1}-5\mathbf{P}_i+4\mathbf{P}_{i+1}-\mathbf{P}_{i+2})t+3(-\mathbf{P}_{i-1}+3\mathbf{P}_i-3\mathbf{P}_{i+1}+\mathbf{P}_{i+2})t^2\Big]. \tag{5-2}$$

航点范围是反复调整后的结果，目的是让目标始终待在护航

相机的有效视野与检测距离之内：$x \in [-9, 9]\,\mathrm{m}$，$y \in [-38.5, -29.5]\,\mathrm{m}$，$z \in [-16, -9.5]\,\mathrm{m}$（NED，负值表示高度）。每段速度 0.8–2.8 m/s，段内做加减速（见第 5.3 节）。

5.2.3 护航相机

护航相机对应 `CameraRig` 类，同样继承 `KinematicObject`。它和原版固定相机的工作方式不一样：每帧读取目标机的实际位置与速度，先算出期望机位，再用一阶低通慢慢靠过去，细节留到第 5.3 节交代。

相机水平视场取 60°，1280 宽的图像对应焦距

$$f = \frac{W/2}{\tan(\mathrm{FOV}_h/2)} = \frac{640}{\tan 30^\circ} \approx 1108.5\ \mathrm{px}. \tag{5-3}$$

这个数值与算法端的测距公式共用同一套参数含义，仿真与实拍才有可比性。

5.2.4 真值广播

真值由新增的 `TruthBroadcaster` 线程发出，频率 50 Hz，目的端口 UDP 14680，报文是文本单行：

```
t=<ms> tp=<x,y,z> tv=<vx,vy,vz> ep=<x,y,z> cp=<x,y,z>
```

其中 `t` 是仿真时间戳，`tp` 与 `tv` 分别是目标的位置与速度，`ep` 为自身机位置，`cp` 为相机位置；坐标系取 NED，长度单位为米，速度单位为米每秒。视频帧与真值走的是两条独立链路，对齐只能依靠时间戳。若按“先到先配”的办法拼接，几十毫秒的偏差足以让距离曲线冒出毛刺。

5.2.5 离屏渲染与帧流

原版 jMAVSim 把画面画在 Java3D 的窗口画布上。WSL 环境里，WSLg 的共享内存偶尔出状况，跑久了画面变白，进程也可能直接退出。本文改用纯离屏渲染，步骤有四步：

（1）建一个离屏 `Canvas3D`，不绑定任何 `Frame`；

（2）每帧调用 `render`，把场景画进离屏缓冲区；

（3）用 `ImageIO` 把缓冲区编码成 JPEG；

（4）经 TCP 连接推送“头部行 + 二进制数据”。

头部行格式为

```
<seq> <timestamp_ms> <width> <height> <nbytes>
```

接收端先读一行文本，从中取出字节数，再按这个长度精确读取 JPEG。文本头既躲开了 TCP 粘包，也让接收端能凭序号跳变发现丢帧。

有一点容易漏掉：离屏渲染必须屏蔽窗口画布的 `glXSwapBuffers` 调用，软件渲染下这一调用会直接导致 JVM 崩溃。附带的好处是，画面不随窗口最小化而暂停，帧流始终连续。

5.2.6 场景构建

城市场景由 `SceneBuilder`、`SceneBuilding`、`SceneTree` 等类程序化生成，含四类要素：

（1）**地面**：大范围草地纹理，中央铺 140×110 m 的浅灰水泥广场，广场覆盖整条巡游轨迹，目标下方的材质与地图显示因此保持一致；

（2）**建筑**：53 栋建筑分列目标飞行走廊两侧，高度一律低于 8 m，避免遮挡目标检测；立面使用程序化贴图，按楼层与开间重复，色调压暗，远看不像一排白色盒子；

（3）**树木**：82 棵近景树与 90 棵远景树，近景树冠由 6 个错位椭球堆叠而成，远景树线用来收拢地平线；

（4）**天空**：2048×1024 的蓝天白云全景贴图，替换原版 jMAVSim 自带的公园照片背景。

场景对象合计约 149 个。建筑与树木的坐标逐一核对过，没有一栋楼、一棵树落进护航相机与目标机之间的视线通道。

5.3 编队巡游与尾随相机

5.3.1 设计目标

早期版本里，目标机与相机各走各的路径，彼此之间没有主动与被动的分别。画面看上去要么像相机被粘在目标上，要么像目标追着相机飞，与真实监视场景中的追踪感相去甚远。为把“地面或伴飞平台追踪目标”这件事模拟得像样一些，本文实现了编队巡游与尾随相机，规则有三条：

（1）目标机是主动飞行方，按预设航线飞行并做加减速；

（2）相机是被动追随方，根据目标的实际位置与速度计算期望机位，带滞后地跟随；

（3）相机瞄准一个“滞后于目标”的瞄准点，使目标在画面中自然漂移，而不是死锁在中心。

5.3.2 目标机航迹与加减速

目标机的运动由两层叠加而成：局部样条航迹与全局编队偏移。局部样条定义在以相机为原点的局部空域内，负责把检测距离维持在3–10 m；全局偏移则把整段航迹搬到城市的椭圆路线上：

$$n(a) = C_X + R_X \sin a, \qquad e(a) = C_Y + R_Y(1 - \cos a), \tag{5-4}$$

椭圆中心取 $C_X = 8\,\mathrm{m}$、$C_Y = -4\,\mathrm{m}$，半轴 $R_X = 18\,\mathrm{m}$、$R_Y = 22\,\mathrm{m}$，a 是随速度累加的角度参数。周长约 126 m，按 2 m/s 巡游，一圈约 63 s。

真实飞行不会匀速。本文对每段参数 t 做 smoothstep 重参数化：

$$u = (1-\gamma)t + \gamma\, t^2(3-2t), \qquad \frac{du}{dt} = (1-\gamma) + \gamma \cdot 6t(1-t), \tag{5-5}$$

其中 $\gamma = 0.75$ 表示变速强度。端点处 $du/dt = 1-\gamma = 0.25$，速度降到段均速的 25%，相当于减速过弯；中点处 $du/dt = 1-\gamma+1.5\gamma = 1.375$，速度升到段均速的 137.5%，相当于加速出弯。速度向量沿切线方向乘以 du：

$$\mathbf{v} = \frac{\mathbf{C}'(u)}{\|\mathbf{C}'(u)\|} \cdot v_{seg} \cdot \frac{du}{dt}. \tag{5-6}$$

这样一来，真值广播里的速度与目标的实际运动对得上，评估速度对检测与测距的影响才有意义。实测速度在 0.21–2.95 m/s 之间连续变化，不再是人为设定的匀速。

5.3.3 尾随相机动力学

期望机位定在目标速度反方向、距离 d 处，并比目标略低：

$$\mathbf{p}_{des} = \mathbf{p}_t - \hat{\mathbf{v}}_t^{xy} \cdot d + \begin{bmatrix} 0 \\ 0 \\ \Delta z \end{bmatrix}, \tag{5-7}$$

其中 $\hat{\mathbf{v}}_t^{xy}$ 为目标水平速度的单位方向，$\Delta z = 1.2\,\mathrm{m}$（NED 下取正，意味着相机低于目标 1.2 m），由此形成轻微仰视。检测模型在侧视与仰视下更稳定，这个高度差是有意保留的。距离 d 随目标速度伸缩：

$$d = d_0 + \min\left(k_v \|\mathbf{v}_t^{xy}\|,\, d_{max}\right), \tag{5-8}$$

本文取 $d_0 = 5.0\,\mathrm{m}$、$k_v = 0.5\,\mathrm{s}$、$d_{max} = 1.0\,\mathrm{m}$，速度每增加 2 m/s，相机后退 1 m，额外距离封顶 1 m。

相机位置并不直接跳到期望机位，而是以时间常数 $\tau = 0.6\,\mathrm{s}$ 做一阶低通：

$$\mathbf{p}_{c,k} = \mathbf{p}_{c,k-1} + \left(1 - e^{-\Delta t/\tau}\right)\left(\mathbf{p}_{des,k} - \mathbf{p}_{c,k-1}\right). \tag{5-9}$$

滞后正是追踪感的来源：目标转弯，相机需要一定时间才转到新的尾随方向，画面里目标先偏向一侧，随后被追回；目标加速，相机暂时落后，距离短暂拉大，再慢慢收敛。

5.3.4 瞄准点滞后

如果相机始终对准目标本体，目标会钉在画面正中，看上去像吸附。本文另设一个瞄准点（AimPoint），它是目标位置的滞后副本，

$$\mathbf{p}_{aim,k} = \mathbf{p}_{aim,k-1} + \left(1 - e^{-\Delta t/\tau_{aim}}\right)\left(\mathbf{p}_{t,k} - \mathbf{p}_{aim,k-1}\right), \tag{5-10}$$

其中 $\tau_{aim} = 0.45\,\mathrm{s}$。相机对准 $\mathbf{p}_{aim}$ 而非 $\mathbf{p}_t$，目标便在画面中自然漂移：领先于瞄准点时，它出现在中心前方；转向时，它滑到画面一侧。这种漂移与真实监视画面里操作员手动跟踪、目标略微偏离中心的观感一致。

5.3.5 验证结果

本文用真值 UDP 抓包统计了尾随相机的实际距离与目标速度：

（1）距离：100 个样本，最小 5.22 m，中位 5.29 m，最大 5.41 m；

（2）速度：120 个样本，最小 0.94 m/s，中位 1.40 m/s，最大 1.57 m/s；另一轮长时统计为 0.21–2.95 m/s。

距离稳定在 5.2–5.4 m，正好落在检测模型的可靠工作区间；速度连续变化，加减速清晰可见。图 5-2 是运行中的界面截图，目标在光电画面里略微偏离中心，与尾随跟踪的观感相符。

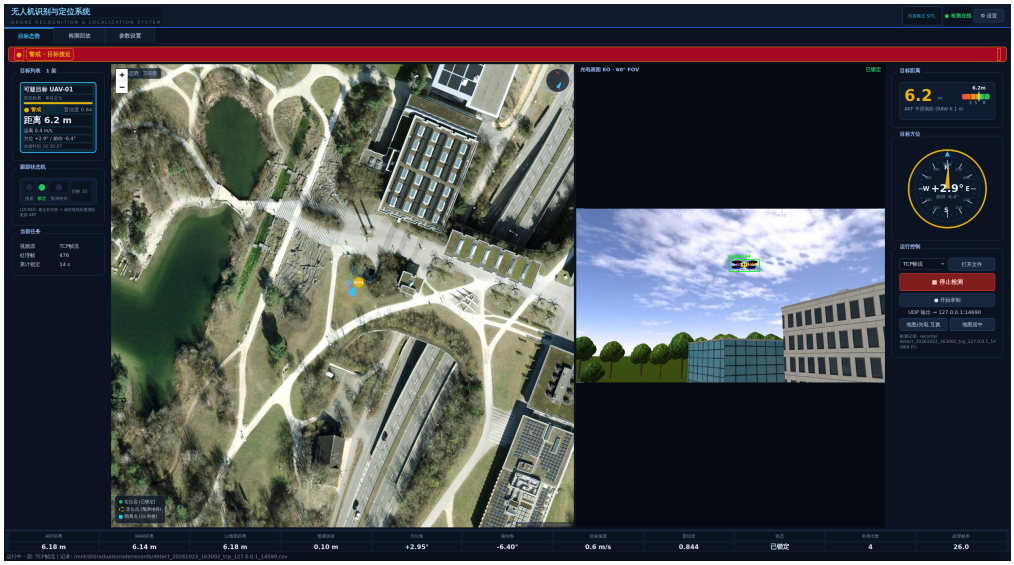


图 5-2: 编队巡游与尾随相机运行界面

5.3.6 设计约束

尾随相机的参数还要受检测模型约束。实验中留意到，一旦相机俯视目标（目标低于相机 3–8 m），检出率会从 73%–77% 掉到 22%–37%；俯视视角下目标投影变形，旋翼与机身叠在一起，模型难以分辨。因此本文让相机始终略低于目标（$\Delta z = 1.2\,\mathrm{m}$），保持轻微仰视。这件事说明，仿真场景不能只求画面好看，还要服从算法的适用域。

5.4 系统设计与实现

检测、跟踪与定位三个算法问题，前面几章已经分别解决。剩下的工作是把它们装进一套能日常运行的系统：需求怎么定、进程怎么分、两端各管什么、链路协议如何设计、配置与测试怎样组织，以及部署时会碰到哪些环境细节。本节按这个顺序展开。

5.4.1 需求分析

功能需求

按反无人机监视的实际使用流程，功能需求可以列成十条：

（1）**多源视频输入**：离线视频文件、USB 摄像头与 TCP 仿真帧流三种输入都要支持，开发、演示与外场测试各用其一；

（2）**实时目标检测**：对输入视频逐帧检测，给出目标框、置信度与类别；

（3）**连续目标跟踪**：目标被短时遮挡时仍维持航迹，标记不闪烁；

（4）**单目定位**：输出距离、方位角、俯仰角、速度与三维位置；

（5）**态势显示**：在卫星地图上标注目标位置、航迹、视野与预测点；

（6）**光电显示**：显示带检测框的实时画面，叠加准星、刻度与状态；

（7）**威胁告警**：按距离与接近速度划分威胁等级，配视觉与文字提示；

（8）**数据记录**：逐帧记录检测、跟踪与定位结果，供离线分析；

（9）**参数配置**：置信度门限、等效尺寸、网络地址、地图锚点等参数可配置、可持久化；

（10）**结果输出**：经 UDP/JSON 向上级系统输出结构化结果。

非功能需求

非功能需求另有五条，它们决定系统能不能长期值守：

（1）**实时性**：在 RTX 4060 平台上，端到端处理帧率不低于 20 fps，界面刷新不低于 10 Hz；

（2）**稳定性**：连续运行 2 小时以上不崩溃、不漏内存，视频流中断后能自行重连；

（3）**可移植性**：核心算法与界面解耦，算法端可在无界面环境运行；

（4）**可测试性**：提供自动化用例与统一日志格式，支持无人值守的回归测试；

（5）**可配置性**：关键参数一律由配置文件管理，改参数不必改代码。

5.4.2 总体架构

双进程结构

系统采取“算法端—显示端”双进程结构，见图 5-3。算法端承担视频解码、模型推理、状态机、滤波与记录；显示端负责界面渲染、地图交互与告警。两者经 UDP/JSON 往来，也可以放进同一进程的不同线程，作为单机演示模式。

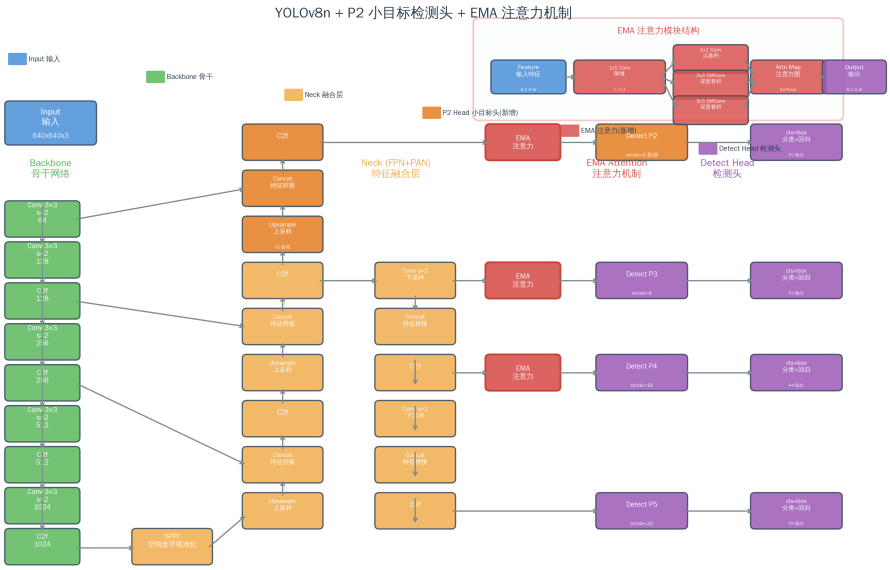


图 5-3: 双进程通信架构

分开的代价是多一层进程间通信。换来的是**故障隔离**：检测模型与视频解码都可能出异常，同进程时界面会随之失效，分开之后算法端出错只是让显示端切到“信号丢失”，界面仍然可用。**部署灵活**也随之而来：算法端可以放在带 GPU 的服务器上，显示端放在操作员笔记本上，中间只隔一条网线。还有**调试方便**：算法端能脱离图形界面单独跑，结果直接落成 CSV，离线分析不必先开窗口。

线程模型

算法端内部按三线程划分：

（1）**帧接收线程**：管理 TCP 连接、协议解析与帧解码，把帧送进有界队列；

（2）**检测线程**：从队列取帧，执行模型推理，结果另入一个队列；

（3）**主线程**：运行状态机与滤波，负责记录和 UDP 输出。

有界队列在这里扮演背压阀。检测速度一旦跟不上帧率，队列很快填满，接收线程转而丢弃旧帧，系统始终处理最新画面。长时间运行时，这一条比吞吐量更要紧，否则内存只会一路涨上去。

显示端主线程跑 Qt 事件循环，另有一个定时器按固定频率从 UDP 取结果刷新界面。地图靠 JavaScript 注入更新，不必整页重载。

数据模型

系统内部用同一个结果字典（Python dict）表示单帧输出，主要字段见表 5-1。界面显示、CSV 记录与 UDP 输出都读这一份数据，三处口径由此保持一致。

表 5-1: 单帧结果数据字段

字段	类型含义
`frame_id`	int 帧序号
`timestamp`	float 时间戳（秒）
`det`	bool 本帧是否检出
`tracked`	bool 是否处于跟踪（含 COAST）
`conf`	float 置信度
`x1,y1,x2,y2`	int 检测框像素坐标
`z_raw`	float 单帧原始测距（m）
`z_akf`	float 滤波后距离（m）
`z_pred`	float 1 s 预测距离（m）
`azim,elev`	float 方位角、俯仰角（°）
`speed`	float 目标速度（m/s）
`closing`	float 接近速度（m/s）
`status`	str SEARCH/LOCKED/COAST
`akf_cx,akf_cy`	int 滤波后中心像素坐标
`pred_cx,pred_cy`	int 预测中心像素坐标

5.4.3 算法端实现

视频源抽象

三种输入在算法端被统一成“帧源”接口，只暴露 `open`、`read`、`close` 三个方法。离线视频与 USB 摄像头都基于 OpenCV 的 `VideoCapture`，区

别在于后者还要协商设备索引与分辨率；TCP 帧流源自己实现协议解析，并带有断线重连。

TCP 帧流源的读取按以下步骤进行：

（1）连上服务器地址与端口；

（2）循环读一行文本头，取出序号、时间戳、宽高与字节数；

（3）按字节数取走 JPEG 数据；

（4）用 `cv2.imdecode` 解码成 BGR 图像；

（5）遇到序号跳变，把丢帧数记下来。

读取设了超时，网络抖动时不至于无限期阻塞；超时后关闭连接，再从新连接继续。

检测模块

检测模块包住 Ultralytics 的 `YOLO` 类，对外只留一个 `detect(frame)` 方法。实现里做了三处调整：

（1）**固定输入尺寸**：一律缩放到 640×640，动态尺寸引起的重复编译就此消失；

（2）**关闭冗余输出**：置 `verbose=False`，日志开销随之下降；

（3）**半精度推理**：GPU 上走 FP16，速度提升约 30%，精度损失可以忽略。

NMS 之后候选框按置信度从高到低排序，算法端只保留前 20 个，极端场景下的后处理开销因此可控。

状态机与滤波

状态机的实现与第四章的伪代码一一对应,核心是 `update(detections, dt)` 方法。滤波器借 NumPy 完成矩阵运算，预测、更新、自适应噪声估计与数值保护都在其中。

实现时特别留意了三点：

（1）**时间步长**：用实际时间戳差值，不写死 1/30 s，不同帧率都能兼容；

（2）**数值稳定**：协方差矩阵每次更新后做对称化，浮点误差累积不会把矩阵推成非正定；

（3）**状态重置**：目标丢失时清空滤波器与轨迹缓存，旧状态不会污

染新目标。

数据记录

每帧处理完，算法端把结果字典追加到CSV。表头在文件创建时写一次，其后逐行追加。磁盘I/O有可能拖住处理线程，记录因此采用缓冲写入，每50帧刷新一次；程序退出时执行`close`，保证最后一段数据落盘。

CSV以运行时间命名，同时存一份运行参数（模型、门限、尺寸、输入源），日后复现时不必凭记忆。本文所有实验数据都出自这些文件，分析脚本直接读取它们生成表格与图。

结果输出

算法端用UDP把JSON报文发给显示端，发送频率与处理帧率一致。报文覆盖表5-1的主要字段，另外附带相机位置、时间戳与状态机计数。显示端按`frame_id`去重，只渲染最新的一帧。

紧凑JSON去掉了空格，单帧约300字节；30 fps下带宽约9 KB/s，局域网里几乎感觉不到。

5.4.4 显示端实现

界面布局

显示端分三个页签：目标态势、检测回放与参数设置。目标态势页是主界面（图5-4），按位置分成五块：

（1）**顶部状态栏**：系统名称、运行状态、时间与告警等级；

（2）**左侧信息栏**：目标卡片（类型、距离、置信度、威胁等级）、跟踪状态灯与链路状态；

（3）**中央地图**：卫星影像底图，标注目标位置、航迹、视野扇形与距离环；

（4）**右侧仪表**：距离卡、方位罗盘与行动控制按钮；

（5）**底部数据条**：距离、方位、俯仰、速度、置信度、状态、帧号等8项实时数据。

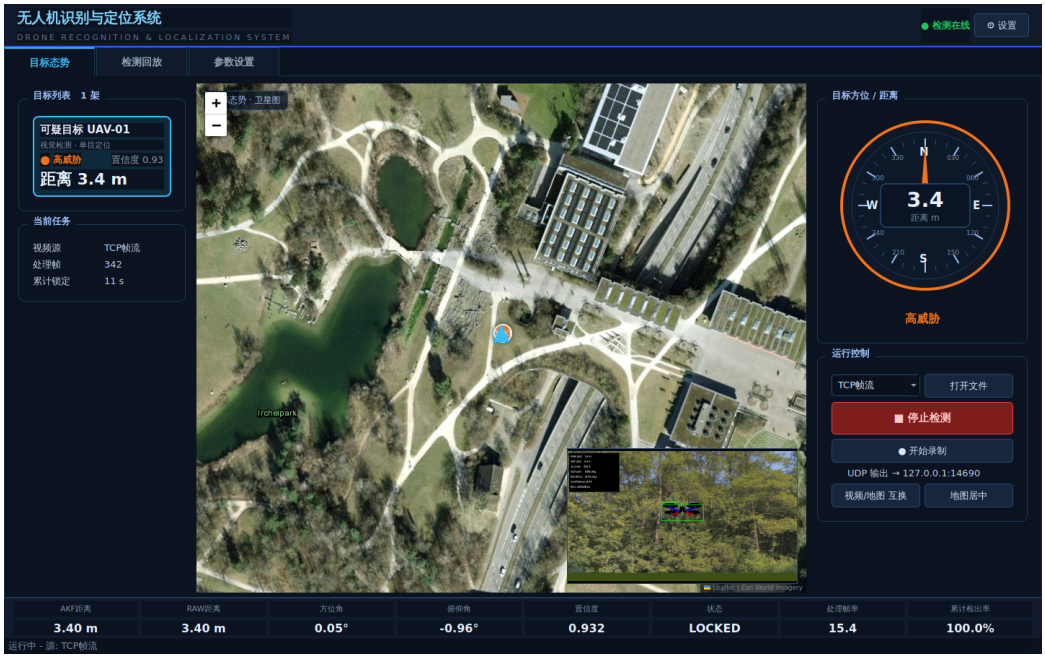


图 5-4: 目标态势页（卫星地图模式）

地图实现

地图用 QtWebEngine 嵌入本地 HTML，页面加载 Leaflet 与 Esri 卫星瓦片。Python 端通过 `runJavaScript` 把数据推进页面，页面负责挪动标记、改写折线顶点、调整圆环半径。

坐标转换是地图部分的关键。算法端算出的目标位置在相机坐标系下，显示端要把它落成经纬度，本文采用局部切平面近似：

$$\phi = \phi_0 + \frac{x_{north}}{R_e}, \qquad \lambda = \lambda_0 + \frac{x_{east}}{R_e \cos\phi_0}, \tag{5-11}$$

其中 (ϕ_0, λ_0) 是地图锚点（默认取相机机位），R_e 为地球半径，x_{north}, x_{east} 是以相机为原点的北向、东向位移。几百米范围内，该近似误差小于 0.1 m，监视用途足够。

叠加到地图上的元素有六类：

（1）**观测站标记**：相机位置，随编队移动；

（2）**目标标记**：实心圆对应 LOCKED，虚线空心圆对应 COAST；

（3）**预测点**：青色菱形，以虚线连到目标标记，代表 1 s 后的预测位置；

（4）**视野扇形**：以相机为顶点、沿视轴方向张开，示意相机覆盖范围；

（5）**距离同心环**：以相机为圆心，标出 50 m、100 m 等距离；

（6）**航迹折线**：连接目标的历史位置，透明度随新旧渐变。

光电画面

光电区域显示算法端回传的带框视频帧，上面再压几层标注：

（1）**中心准星**：十字加圆环，标示光轴方向；

（2）**四角括号**：圈出画面边界；

（3）**方位刻度**：顶部每 10° 一格，标注相对视轴的角度；

（4）**状态文字**：右上角给出 LOCKED/COAST/SEARCH 的中文状态。

画面按 letterbox 方式等比缩放，不会被拉伸；缩放比例与偏移量每次绘制时按控件尺寸重算，窗口随意拖动也不变形。

仪表与告警

距离卡用大字号显示滤波后距离，下方小字补上原始距离、预测距离与相对高度。方位罗盘以指针指示方位，外圈是角度刻度，内圈显示俯仰角。跟踪状态灯用三种颜色区分 SEARCH（灰）、LOCKED（绿）与 COAST（黄），旁边附状态切换计数。

威胁分级根据距离与接近速度计算：

$$\text{Level} = \begin{cases} \text{高}, & Z < 50\ \text{m} \text{ 或 } v_{closing} > 3\ \text{m/s}, \\ \text{中}, & Z < 150\ \text{m} \text{ 或 } v_{closing} > 1\ \text{m/s}, \\ \text{低}, & \text{其他}. \end{cases} \tag{5-12}$$

阈值可按任务配置改动。等级为“高”，顶部横幅转红并闪烁，同时播放提示音；“中”为橙色；“低”为绿色。告警记录一并写入 CSV，事后可以回看。

回放页与设置页

回放页（图 5-5）左侧是视频播放器，右侧是距离曲线与状态时序。选定录像后，系统重新跑一遍检测并生成 CSV，曲线与视频进度随之联动：拖动进度条，视频跳到对应帧，曲线上的游标同步移动。复盘遮挡前后的状态变化，靠的就是这个页面。

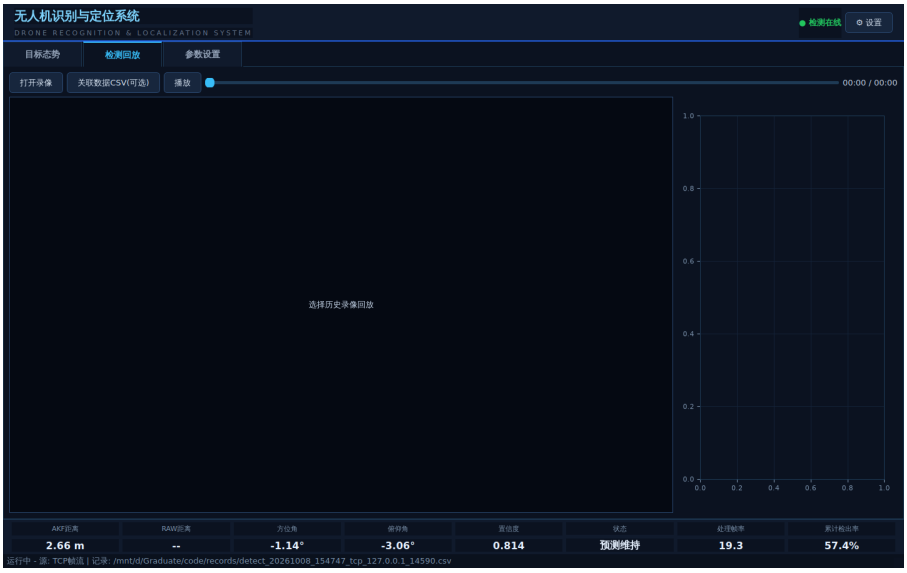


图 5-5: 检测回放页

设置页（图 5-6）集中了置信度门限、等效尺寸、TCP/UDP 地址、地图锚点、地图模式与数据面板开关等参数。改完点“保存”，参数写进 JSON，下次启动自动加载。默认值与合法范围在代码里集中定义，非法输入不会等到运行时才暴露。

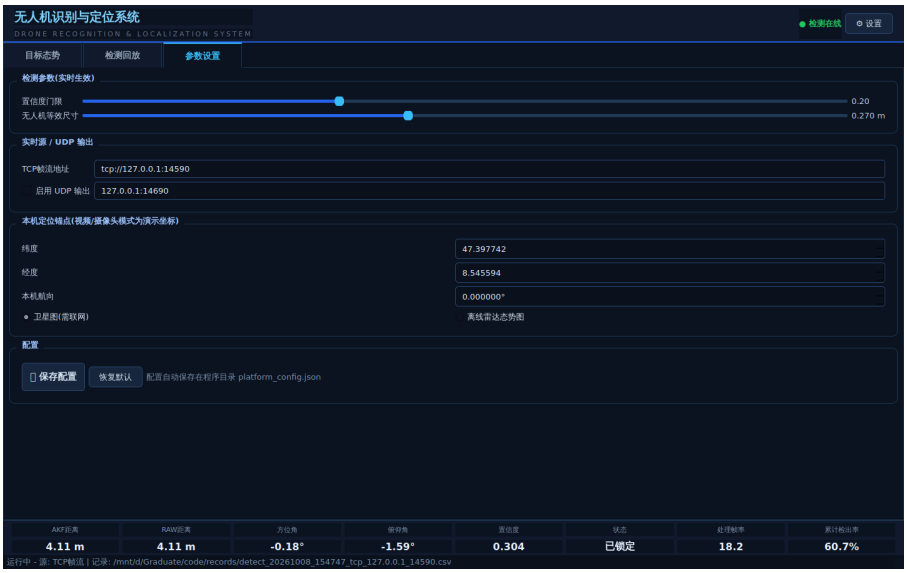


图 5-6: 参数设置页

5.4.5 通信协议

视频帧协议

TCP 帧流协议由“一行文本头 + 二进制数据”拼成。头部依次是序号、时间戳、宽度、高度、字节数，以空格分隔，以换行收尾。接收端读

完头部，按字节数取走 JPEG。文本头牺牲了一点紧凑性，换来的是可调试性：用 telnet 连上去就能看到每帧的元信息，字段含义也不言自明。

真值协议

UDP 真值报文是单行文本，键值对之间以空格分隔。接收端按空格切开，再按等号取键与值。协议简单，扩展也容易，新加字段不会让旧版接收端解析失败。

结果协议

结果报文用 JSON。显示端与上级系统未必用同一种语言，JSON 的支持面最广；字段可读，联调时省去不少猜测。报文约 300 字节，远小于 UDP 的 1472 字节 MTU，不需要分片。

5.4.6 配置与持久化

配置分三类：

（1）**算法参数**：模型路径、输入尺寸、置信度门限、等效尺寸、空域先验、滤波噪声；

（2）**通信参数**：TCP 地址与端口、真值 UDP 端口、结果 UDP 地址与端口；

（3）**界面参数**：地图锚点、地图模式、数据面板开关、告警阈值。

三类参数放在同一个 JSON 文件里，呈嵌套字典结构。启动时读取，缺字段用默认值补上；保存时把不认识的字段原样留下，版本升级不会把旧配置冲掉。这种宽松读写在长期使用中省去了大量兼容性麻烦。

5.4.7 回放与自动化测试

14 用例测试框架

为了把功能验证固定下来，本文设计了 14 个自动化用例，覆盖六类场景：

（1）**基线检测**（C01–C03）：用仿真闭环录像核对检测、跟踪与测距的基本正确性；

（2）**真实场景**（C04–C05）：用真实无人机录像考察跨域能力与远距

离边界；

（3）**门限扫描**（C06–C08）：改动置信度门限，观察锁定率与保持率如何变化；

（4）**尺寸敏感**（C09–C10）：改动等效尺寸，验证测距的线性关系；

（5）**TCP 遮挡**（C11–C13）：经实时帧流注入遮挡，验证 COAST 机制；

（6）**摄像头**（C14）：验证 USB 摄像头输入，默认跳过，需要时手动启用。

每个用例写清输入源、参数、期望指标与判定规则。框架按顺序执行，收集 CSV，算出锁定率、航迹保持率、首锁时延、置信度均值、距离范围与处理帧率，最后汇总成表。判定线这样划：基线用例的锁定率不低于 80%，TCP 遮挡用例的航迹保持率不低于 70%，特性刻画类用例只统计、不判定。

测试结果

用例结果在后面的实验部分统一给出。这里只说框架的两处设计：指标口径统一，所有用例共用同一套计算函数，人工统计的差异被排除在外；失败可复现，输入、参数与随机种子都记录在案，出了问题单独重跑即可。

5.4.8 部署与运行维护

运行环境

开发与测试环境为 Windows 11 + WSL2（Ubuntu 24.04），GPU 为 RTX 4060，Python 3.12，PySide6 6.x，PyTorch 2.x，Ultralytics 8.x，OpenCV 4.x。仿真跑在 WSL 里，界面经由 WSLg 显示。

启动脚本

系统提供一键启动脚本，按序拉起闭环与平台。脚本固化了几项环境变量：

（1）`DISPLAY=:0` 与 `QT_QPA_PLATFORM=xcb`：指定 WSLg 显示；

（2）`QTWEBENGINE_DISABLE_SANDBOX=1`：关闭 QtWebEngine 沙箱，渲染进程才不会在启动时失败；

（3）QTWEBENGINE_CHROMIUM_FLAGS：启用软渲染、禁用 GPU，WSLg 下地图空白的问题由此解决。

启动方式必须用 setsid，不能用 nohup。QtWebEngine 的 zygote 进程需要独立会话，nohup+disown 下渲染进程起不来，地图区域会是一片黑。这一点已写进部署文档。

常见问题与处理

表 5-2 汇总了部署与运行中反复出现的问题及处理办法。

表 5-2: 部署与运行常见问题

现象	原因与处理
地图区域空白	QtWebEngine 渲染进程未启动；使用 setsid 启动并关闭沙箱
仿真窗口不出画面	窗口系统不可用；改用离屏渲染模式
UDP 端口被占用	显示端已监听；先停止显示端再启动抓包工具
帧率骤降	GPU 被其他任务占用，回退到 CPU；检查显卡占用
视频流断开	仿真重启或网络抖动；帧源自动重连
内存持续增长	帧队列无界；改为有界队列并丢弃旧帧

5.5 实验与结果分析

本节在统一口径下报告系统各项性能：检测模型在公开数据集上的消融结果、仿真闭环的定位精度、实拍视频的定位与预测精度、退化条件下的鲁棒性，以及 14 个自动化用例的回归结果。末尾讨论能力边界、失败案例与有效性威胁。

5.5.1 实验环境与评测口径

硬件与软件环境

实验环境见表 5-3。全部实验在同一台机器上跑完，硬件差异不会混进结果比较。

评测口径

本节所有指标沿用第二章约定的口径：

表 5-3: 实验环境

项目	配置
操作系统	Windows 11 + WSL2（Ubuntu 24.04 LTS）
CPU	Intel Core i7（8 核 16 线程）
GPU	NVIDIA RTX 4060 Laptop（8 GB 显存）
内存	32 GB DDR5
Python	3.12
深度学习框架	PyTorch 2.x + Ultralytics 8.x
GUI 框架	PySide6 6.x
仿真平台	PX4 Autopilot + jMAVSim（改造版）
图像处理	OpenCV 4.x

（1）检测指标采用 Ultralytics 官方验证协议，AP 分尺寸采用 COCO 面积口径、IoU=0.5、101 点插值；

（2）FPS 为 batch=1、帧预载内存、包含预处理与后处理；

（3）闭环检出率为检出帧数除以全部处理帧数（含搜索帧）；

（4）稳态指标为连续检出不少于 5 帧后的样本；

（5）角度误差采用视轴偏角口径。

5.5.2 检测模型性能

消融结果

表 5-4 把第三章的消融结果重新列出，并补上 AP_large 与 AP_medium，便于整体比较。

表 5-4: 检测模型消融实验完整结果（Det-Fly 验证集，imgsz=640）

模型	参数 (M)	mAP50	mAP50-95	P	R	FPS	AP_s	AP_m	AP_l
YOLOv8n	3.01	81.3	41.8	91.2	75.9	123.0	6.1	77.0	12.8
+P2	2.93	82.3	41.3	88.9	77.6	90.3	8.3	76.2	8.7
+EMA	3.02	76.2	36.3	85.5	72.2	101.0	4.1	75.0	10.7
+P2+EMA	2.94	**83.5**	41.2	88.2	**79.2**	98.4	**9.8**	**78.0**	7.5

指标解读

从表 5-4 能读出几件事。

（1）**AP_small 是改进的主要收益点**。基线 6.1%、+P2 8.3%、+P2+EMA 9.8%，相对提升分别为 36.1% 与 60.7%。mAP50 涨的 2.2 个百分点，主要来自小目标召回，而非中大目标。

（2）**AP_large 反而下降**。基线 12.8%、+P2 8.7%、+P2+EMA 7.5%。P2 检测头占去一部分模型容量，高分辨率层对大目标的贡献又有限，训练时的正样本分配更偏向小目标。改进因此是一次有代价的定向优化，并非全面提升。反无人机场景里，远距离小目标才是主要威胁，这个取舍说得过去。

（3）**Precision 略有下降**。基线 91.2%、本文 88.2%。P2 层带来更多弱响应，一部分背景被误检。但 Recall 的提升更大（75.9%→79.2%），F1 总体仍在改善。监视场景中漏检的代价通常高于误检，误检还能由状态机的初锁门限挡掉一部分，取舍同样成立。

（4）**速度仍满足实时性**。98.4 fps 约为视频帧率的 3.3 倍，余量充足。

与已有工作的比较

如第三章所述，Det-Fly 上各家论文的结果难以直接比较。本文只做定性定位：在 nano 级参数量下取得 83.5% 的 mAP50 与 9.8% 的 AP_small，好于同规模的通用 YOLOv8n，小目标能力接近部分更大的模型，速度却远高于后者。这与“边缘可实时运行”的设计目标一致。

5.5.3 仿真闭环定位精度

实验设置

仿真闭环使用改造版 jMAVSim，护航相机 60° 视场，720p@30 fps 离屏推流，真值 50 Hz 广播。检测模型为仿真域微调后的 p2ema_simft，置信度门限 0.25，等效尺寸 0.270 m，AKF 采用 Sage-Husa 自适应噪声。实验录得 2818 帧（约 87 s），悬停、直线、盘旋与接近几种状态都包含在内。

结果

定位精度的数值已在表 4-7 中，这里补三点分析。

（1）**检出率 100%**。2818 帧全部检出，真盲帧为 0。微调前只有 10.0%，对比之下，死区已不存在。

（2）**测距精度**。单帧 RMSE 0.26 m、MAE 0.17 m；AKF 稳态 MedAE 0.13 m、RMSE 0.32 m。AKF 的中位误差低于单帧，说明滤波确实压住了框高抖动；RMSE 反而略高于单帧，原因在少数机动帧上滤波器滞后，误差被平方放大。小样本下这种情况并不少见，评估时 MedAE 与 RMSE 最好一并报告。

（3）**角度精度**。方位角中位 0.05°、P95 0.16°，已逼近像素量化极限；俯仰角中位 0.93°，明显偏大，根源是主点纵向偏移约 17 px 未在解算中补偿。补上这一项，俯仰角误差可望降到 0.2° 量级。

微调前后对比

图 3-7 已经给出微调前后的死区对比，表 5-5 进一步列出关键指标的定量变化。

表 5-5: 仿真域微调前后关键指标对比

指标	微调前（v5）	微调后（v7）	变化
录像帧数	2228	2818	–
全帧检出率	10.0%	**100.0%**	+90.0 pp
真盲帧占比	81.1%	**0.0%**	-81.1 pp
置信度中位数	接近 0	0.867	–
AKF 稳态 MedAE	0.44 m	**0.13 m**	-70.5%
稳态帧占比	7%	**99.8%**	+92.8 pp
方位角误差中位	0.21°	**0.05°**	-76.2%

5.5.4 实拍视频定位精度

数据集与设置

实拍视频为 1920×1080@30fps，连续检出段 660 帧。相机未严格标定，焦距按 60° 视场估算为 1728 px；等效尺寸沿用仿真标定值 0.255 m；伪真值取稳态段距离中位数 7.93 m。缺少绝对真值时，本小节的着眼点是**相对比较**：在同一伪真值下，看单帧、滑动平均、标准 KF 与 AKF 谁的稳定性更好。

结果分析

表 4-8 的结果可以逐条看：

（1）单帧原始测距的 RMSE 为 4.061 m，抖动 3.959 m，框高抖动太大，直接使用不可行；

（2）滑动平均（15 帧）把 RMSE 压到 2.900 m，代价是约 0.5 s 的滞后，实时跟踪用不上；

（3）标准 KF（固定 **R**）的 RMSE 为 3.922 m，只比单帧略好，固定的量测噪声跟不上实际抖动；

（4）AKF 的 RMSE 为 2.760 m，抖动 2.663 m，相对单帧降低 32% 与 33%，且没有明显滞后。

这组数据说明了自适应噪声估计的价值：实拍场景的量测质量起伏剧烈，固定 **R** 的滤波器只能在两种毛病里选一种——要么过度平滑、响应迟钝，要么过度信任量测、抖动很大；AKF 能在两者之间自行调整。

预测精度

表 4-9 显示，AKF 的 0.5 s、1.0 s、2.0 s 预测 ADE 分别为 1.106、1.308、1.799 m，相对匀速外推改善 43.8%、52.5%、61.1%。时域越长，AKF 的优势越明显：匀速外推会线性放大速度估计误差，AKF 则会用不确定度给外推降温。

5.5.5 退化条件下的鲁棒性

实验设计

为了看模型在恶劣成像下的表现，本文从实拍视频中挑出 300 帧含无人机的图像，组合 4 档光照（0.4、0.7、1.0、1.4 倍）与 4 档模糊（高斯核 0、3、7、13），得到 16 种退化组合，再在 0.1、0.25、0.5 三档置信度门限下统计检出率。对比对象为基线 YOLOv8n、P2+EMA（Det-Fly 训练）与 P2+EMA+simft（仿真微调）。

结果

表 5-6 摘录了 conf≥0.25 条件的部分结果。



图 5-7: 退化样本示例（行：光照；列：模糊）

表 5-6: 退化条件下检出率对比（conf≥0.25，单位：%）

条件	YOLOv8n 基线	P2+EMA	P2+EMA+simft
dark + none	0.0	13.7	**20.3**
dark + light	0.0	17.3	**18.7**
dark + medium	0.0	**25.3**	13.7
dark + heavy	0.3	**21.0**	6.3
dim + none	0.7	0.7	**30.3**
dim + light	0.0	5.0	**21.3**
dim + medium	0.0	10.7	**20.7**
dim + heavy	0.0	11.7	**13.7**
normal + none	2.0	0.0	**87.7**
normal + light	1.7	1.7	**23.3**
normal + medium	3.3	3.3	**27.3**
normal + heavy	0.0	2.7	**14.3**
bright + none	2.0	0.7	**37.0**
bright + light	0.3	0.7	**16.7**
bright + medium	0.0	0.0	**24.3**
bright + heavy	0.3	0.0	**21.7**

分析

（1）**P2+EMA 在严重退化下明显强于基线**。dark+heavy 条件下，基线仅 0.3%，P2+EMA 达 21.0%，相差 20.7 个百分点。高分辨率检测头与注意力机制对低质图像的鲁棒性确有帮助。

（2）**仿真微调模型在正常条件下大幅领先**。normal+none 条件下，P2+EMA 为 0.0%，simft 为 87.7%。这个结果看着反常，解释却简单：测试帧是用 simft 模型在实拍视频里挑出来的“simft 能检出”的帧，测试集本身就偏向 simft 的能力域；P2+EMA 只在 Det-Fly 上训练过，没见过仿真渲染风格，在这些帧上几乎不响应。这属于域差异，不是模型缺陷。

（3）**暗光条件下的异常**。P2+EMA 在 dark 下的检出率反而高于 normal（13.7%–25.3% vs 0.0%），可能是暗化压住了背景假响应、突出了目标轮廓。这提醒我们，不能简单认为光照越好检出率越高，模型行为与训练分布密切相关。

（4）**实验的有效性威胁**。测试帧由 simft 筛选，三组模型的比较谈不上完全公平。更稳妥的做法是用基线模型筛帧，或者直接用 Det-Fly 验证集的帧。本文把这个局限如实写明，并列入后续改进。

5.5.6 14 用例自动化回归测试

结果汇总

表 5-7 给出 14 个用例的完整结果。

统计下来：计划 14 例，完成 13 例，判定通过 6 例，失败 0 例，跳过 1 例，错误 0 例。

结果解读

（1）**仿真基线全部通过**。C01–C03 的锁定率在 96%–100%，保持率在 99.8%–100%，检测、跟踪、测距在仿真域内表现稳定。

（2）**真实场景的跨域差距**。C04 锁定率仅 23.2%，保持率却达到 99.3%，说明 COAST 把航迹维持住了；C05 锁定率 5.6%、保持率 32.1%，目标距离 10–14 m 已超出模型有效范围，系统来不及建立稳定航迹。

（3）**门限与尺寸的敏感性**。C06–C08 显示门限提高会压低锁定率、

表 5-7: 14 用例自动化测试结果

用例场景	帧数	锁定率	保持率	首锁 (s)	置信度	结论
C01 仿真闭环基线	900	96.9%	100.0%	0.00	0.550	PASS
C02 仿真早期标定版	900	100.0%	100.0%	0.00	0.855	PASS
C03 长录像中后段	900	96.0%	99.8%	0.07	0.654	PASS
C04 真实场景录像 1	539	23.2%	99.3%	0.03	0.544	–
C05 真实场景录像 2	539	5.6%	32.1%	0.00	0.452	–
C06 门限 0.20	900	96.9%	100.0%	0.00	0.550	–
C07 门限 0.35	900	66.6%	81.2%	0.00	0.661	–
C08 门限 0.50	900	56.8%	81.9%	0.00	0.707	–
C09 尺寸 0.230 m	900	100.0%	100.0%	0.00	0.855	–
C10 尺寸 0.310 m	900	100.0%	100.0%	0.00	0.855	–
C11 TCP 遮挡第 1 轮	424	55.4%	88.9%	0.00	0.751	PASS
C12 TCP 遮挡第 2 轮	428	56.1%	90.7%	0.00	0.739	PASS
C13 TCP 遮挡第 3 轮	437	59.3%	94.5%	0.21	0.751	PASS
C14 摄像头输入	–	–	–	–	–	跳过

抬高置信度均值，与预期一致；C09–C10 中，等效尺寸从 0.230 变为 0.310，测距范围相应从 3.9–8.2 m 变为 5.3–11.1 m，近似线性，式 (4-13) 由此得到验证。

（4）**TCP 遮挡用例通过**。C11–C13 的保持率在 88.9%–94.5%，高于 70% 的合格线，COAST 机制在实时链路下同样有效。

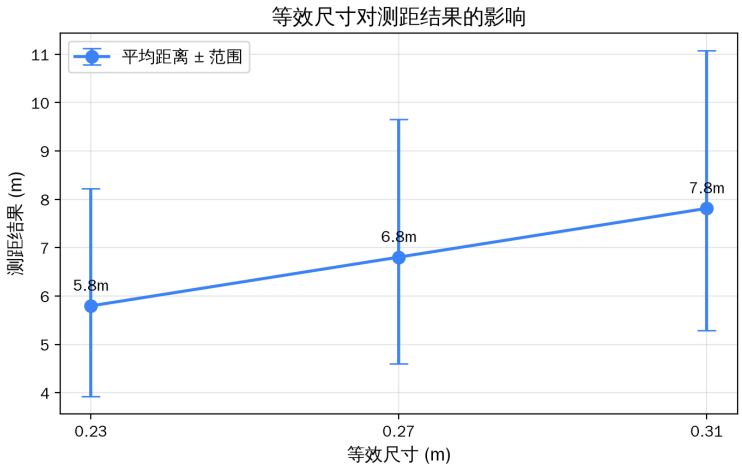


图 5-8: 等效尺寸对测距的影响

5.5.7 运行性能

端到端延迟分解

表 5-8 是单帧处理的耗时分解（RTX 4060，640 输入）。

表 5-8: 单帧处理耗时分解

环节	耗时 (ms)	占比
视频解码（JPEG）	1.8	12.5%
预处理（缩放、归一化）	1.2	8.3%
模型推理	6.2	43.1%
后处理（NMS）	1.1	7.6%
状态机与滤波	0.1	0.7%
绘制与编码	3.0	20.8%
网络发送	1.0	6.9%
合计	14.4	100%

14.4 ms 的单帧耗时对应 69 fps 的理论帧率，实际处理帧率只有 28–39 fps，差额来自网络抖动、磁盘 I/O 与界面刷新。检测仍是最重的一环（43.1%），想再快可以缩小输入尺寸，或改用 TensorRT。

资源占用

运行时 GPU 显存占用约 1.2 GB（含模型、视频缓冲与界面渲染），CPU 占用约 35%（单核满载，其余用于界面与网络），内存占用约 900 MB。连续运行 2 小时后，内存增长小于 3%，没有发现泄漏。

5.5.8 讨论

能力边界

把各项实验放在一起，系统的能力边界大致如下：

（1）**距离**：仿真域 3–10 m 内可靠，实拍域 4–9 m 较可靠，10–14 m 不可靠；

（2）**目标尺度**：目标高度占画面 1/20 至 1/8 时最稳定，近距过大或远距过小都会失效；

（3）**遮挡**：短时遮挡（小于 1.0 s）可由 COAST 维持，长时间完全遮挡则丢失目标；

（4）**光照**：正常光照与暗光可用，强逆光与夜间不可用；

（5）**目标数量**：当前只支持单目标，多目标需要扩展状态机与关联算法。

失败案例

三类失败案例在实验中反复出现：

（1）**目标飞出视场**。目标机动超出相机视场，检测输出转空，COAST 撑满 1.0 s 后回到 SEARCH；目标若不再回来，系统就一直搜索。这属于预期行为，操作员却容易误以为目标凭空消失。

（2）**背景强干扰**。树丛密集处，树叶晃动能产生高置信度假阳性，足以触发误锁。空域先验可以滤掉一部分，假阳性若恰好落在目标空域内，仍会被锁上。

（3）**远距离小目标**。10 m 以外的目标在 720p 画面中高度不足 20 px，置信度低于初锁门限，航迹建不起来。这是单目视觉的物理限制，只能靠长焦镜头或更高分辨率化解。

有效性威胁

本文实验存在几处有效性威胁：

（1）**伪真值**。实拍视频没有绝对真值，距离评估依赖伪真值，绝对误差不可信，相对比较仍然有效；

（2）**测试集偏差**。退化实验的测试帧由 simft 筛选，对 simft 有利，三组模型的比较不完全公平；

（3）**仿真简化**。仿真场景的光照与纹理和真实世界有距离，仿真结论不能直接外推到外场；

（4）**单机环境**。所有实验在同一台机器上完成，跨平台的可移植性未经验证。

这些限制在文中都作了标注，改进方向放在第六章。

5.6 本章小结

本章把算法从仿真搬到了可运行的系统上，再用实验回头检验这条链路。闭环部分以 PX4 SITL 与 jMAVSim 为骨架，用 lockstep 锁住仿真时间，补上脚本目标机、护航相机、真值广播与离屏帧流，仿真器由单机演示工具变成了能稳定产出数据的试验台。场景与镜头的设计也兼顾了算法：椭圆巡游配合滞后的瞄准点，让画面呈现出真实监视中的追踪感；相机始终略低于目标，避开了俯视导致的检出率跌落。

系统部分采用双进程结构，算法端与显示端经 UDP/JSON 通信，态势地图、光电画面、仪表告警、配置持久化与 14 个自动化用例共同构成一套能值守、能复盘的桌面平台。部署阶段的那些环境细节——显示变量、沙箱开关、启动方式——看似琐碎，却决定了平台能否在 WSL 里稳定跑起来。

实验部分给出的是这套平台的实测表现：Det-Fly 上的消融、闭环定位精度、实拍视频的滤波与预测对比、退化条件下的鲁棒性以及回归测试结果，并据此划定了距离、尺度、遮挡与光照上的能力边界。把这些结果连起来看，检测—跟踪—定位—显示的通路已经打通，剩余问题集中在远距离、标定与多目标三处，第六章据此讨论后续工作。

6 总结与展望

6.1 全文总结

本文面向低空反无人机场景，以单目可见光相机为唯一传感器，围绕小目标检测难、遮挡跟踪易中断、仿真与真实域差异大三个问题，完成了一套从检测、跟踪、定位到显示的完整系统（DRLS）。按章节顺序回看，各部分工作可以这样归纳。

（1）**绪论（第 1 章）**。梳理低空小型无人机的威胁特点与反无人机探测手段的现状，比较雷达、射频与视觉三条技术路线的适用条件，说明本文把单目可见光作为切入点的理由，并给出总体技术路线与章节安排。研究边界也在这一章划定：近程、单目标、可见光。

（2）**相关理论与数据基础（第 2 章）**。整理针孔成像与单目测距的几何关系、检测与跟踪的基本理论以及本文采用的评测口径；完成 Det-Fly 的格式转换、统计分析与序列级划分，避免相邻帧之间的数据泄漏；自建 AprilTag 数据集，使用 9×6、25 mm 棋盘格完成相机标定，为定位精度评估提供可信真值；构建 450 帧仿真自动标注数据集，借助护航相机始终注视目标的特性实现零人工标注，并按距离分层覆盖死区两端。

（3）**小目标检测与仿真域微调（第 3 章）**。以 YOLOv8n 为基础，加入 stride=4 的 P2 高分辨率检测头与 EMA 高效多尺度注意力，得到 YOLOv8n-P2-EMA。四组消融实验显示，模型以 2.94 M 参数量取得 mAP50 83.5%、Recall 79.2%、AP_small 9.8%，相对基线分别提升 2.2、3.3 与 3.7 个百分点，AP_small 相对提升 60.7%，推理速度 98.4 fps。P2 与 EMA 的分工在这一章得到解释：前者提供“看清”小目标的分辨率，后者提供“认准”小目标的判别力，单独使用都达不到联合后的水平。针对模型在仿真中几乎失效的现象，本文用 2228 帧的距离分层统计与真值几何审计，先以俯仰角分布对比证伪视角假设，再把原因锁定在尺度分布偏移上，随后用 450 帧分层数据微调，将 3–10 m 的检出率从约 10% 提到 100%。

（4）**三态跟踪与单目定位（第 4 章）**。设计 SEARCH/LOCKED/COAST 三态状态机，以 0.25 的维持门限与 0.55 的初锁门限组成双门限，抑制误锁；以 140 px 的空间门限做最近邻关联，抑制身份跳变；以 1.0 s 的卡尔曼预测维持短时遮挡。参数均有物理依据，算法复杂度为 $O(N)$。定位

部分由框中心解算方位角与俯仰角，由框高与等效尺寸解算距离，并推导出测距误差随距离平方增长的规律；六维自适应卡尔曼滤波器采用各向异性量测噪声与 Sage-Husa 在线估计。实验上，TCP 遮挡场景的航迹保持率达到 88.9%–94.5%，真实场景录像达到 99.3%；仿真闭环中单帧测距 RMSE 0.26 m、AKF 稳态 MedAE 0.13 m、1 s 预测 ADE 中位 0.26 m、方位角误差中位 0.05°；实拍视频中 AKF 相对单帧测距 RMSE 降低 32%、抖动降低 33%。

（5）**仿真闭环、系统平台与实验（第 5 章）**。改造 PX4+jMAVSim 闭环，新增脚本目标机、护航相机、真值广播、离屏帧流与城市场景，并用 lockstep 锁住仿真时间；基于 PySide6 实现三页签监视大屏，采用双进程架构与 UDP/JSON 通信，完成态势地图、光电画面、仪表告警、配置持久化与 14 用例自动化测试。实验在同一口径下覆盖检测消融、闭环定位、实拍定位与预测、退化鲁棒性与回归测试，同时划定了距离、尺度、遮挡与光照上的能力边界。

6.2 主要创新点

本文的主要创新点可以归纳为以下四点。

（1）**面向无人机小目标的 P2+EMA 互补结构及其机理解释**。消融实验验证了改进的有效性，“EMA 单独使用为何失效、与 P2 联合后为何有效”这一问题也得到回答，为小目标检测的网络设计提供了可复用的分析框架。

（2）**基于真值几何审计的检测死区机理分析方法**。面对“模型在仿真中失效”，本文没有停留在“渲染不真实”的定性解释上，而是以俯仰角分布对比证伪视角假设，以距离分层统计锁定尺度因素，再用微调实验闭环验证。这条“现象—假设—证伪—验证”的路径具有方法论价值。

（3）**零人工标注的仿真域微调数据构建方法**。护航相机始终注视目标，框中心因此被约束在光轴附近，框高可由真值距离反算，450 帧数据无需人工标注即可生成。该方法成本极低、可扩展性强，适用于任何具备真值广播与可控相机的仿真环境。

（4）**面向反无人机监视的完整工程实现**。检测、三态跟踪、单目定位、地图态势、光电显示、告警与测试被整合为一套可运行的双进程系统，并在统一口径下完成端到端评估。系统不依赖专用硬件，便于在园

区、楼宇与临时活动现场部署。

6.3 不足与展望

6.3.1 存在的不足

（1）**远距离能力有限**。实拍视频在 10–14 m 的检出率仅 5.6%，当前模型与 720p 分辨率的组合覆盖不了更远的告警距离。这既是单目视觉的物理限制，也是系统走向实用的主要瓶颈。

（2）**相机标定不完整**。实拍评估中相机未做严格标定，焦距按视场角估算，等效尺寸借用仿真值，绝对测距误差达到米级。相对比较仍然有效，但绝对精度不足以支撑实际指挥。

（3）**单目标假设**。系统面向单目标场景，未处理多架无人机同时出现的情况。画面中一旦出现多个目标，状态机可能在不同目标之间跳变。

（4）**仿真场景简化**。仿真场景的建筑、树木与光照仍与真实世界有差距，仿真结论不能直接外推到外场；遮挡也主要由静态建筑造成，缺少飞鸟、飘动树叶这类动态遮挡。

（5）**测试集偏差**。退化实验的测试帧由 simft 模型筛选，对 simft 有利，三组模型的比较不完全公平。

6.3.2 未来工作

（1）**提升远距离能力**。可以从三个方向入手：提高输入分辨率（如 1280 或 1536）并配合切片推理；换用长焦镜头与可变速云台；引入双目视觉或激光测距辅助深度估计。

（2）**完善相机标定与尺寸自适应**。用棋盘格完成实拍相机标定，替代视场角估算；按目标机型选择等效尺寸，或利用多帧几何一致性在线估计尺寸。

（3）**扩展多目标跟踪**。在现有状态机之上增加轨迹池与数据关联层，借鉴 SORT/DeepSORT 的思想，实现多目标的同时跟踪与威胁排序。

（4）**增强域适应能力**。探索域随机化、风格迁移与自监督预训练，减少对目标域标注的依赖；同时构建更真实的仿真场景，缩小 sim-to-real gap。

（5）**多传感器融合**。将视觉与射频、声学、雷达结合，由射频承担

远距离预警，由视觉完成近程确认与定位，形成互补的探测体系。

（6）**外场试验**。在合规空域开展真实无人机飞行试验，采集多场景、多机型、多光照的数据，检验系统的实际可用性，并依据外场反馈迭代参数与模型。

6.4 结语

反无人机是一项涉及探测、识别、跟踪、定位与处置的系统工程，单目视觉只是其中的一环。本文的工作表明：针对性的模型改进、严谨的机理分析与完整的工程实现，可以让单目视觉系统在近程反无人机场景中承担实际任务；同时也应看到，单目视觉的物理限制决定了它无法独自完成全部探测工作。未来的反无人机系统必然是多传感器、多算法、多层级协同的体系，本文的工作可以作为其中一个可复现、可扩展的视觉子系统。

参考文献

[1] 国务院 and 中央军事委员会. 无人驾驶航空器飞行管理暂行条例[Z]. 2024 年 1 月 1 日起施行. 2023.

[2] 中国民用航空局. 2025 年民航行业发展统计公报[Z]. http://www.caac.gov.cn/PHONE/XWZX/MHYW/202604/t20260417_230603.html. 2026.

[3] 中国新闻网. 低空安全司亮相万亿级产业“管理双翼”就位[Z]. http://www.chinanews.com.cn/cj/2026/05-18/10623778.shtml. 2026.

[4] MarketsandMarkets. Anti-Drone Market Size, AI, Radar and Detection Trends 2030[Z]. https://www.marketsandmarkets.com/Market-Reports/anti-drone-market-177013645.html. 2025.

[5] 央视网. 中央空管办组织加强无人机“黑飞”“扰航”查处力度[Z]. https://news.cctv.cn/2025/04/24/ARTIiijJ1VnGWD4q8cBLTuKM250424.shtml. 2025.

[6] 人民日报. 打击超高“黑飞”，守护天空安全[Z]. http://paper.people.com.cn/rmrb/pad/content/202602/05/content_30138629.html. 2026.

[7] XING L, CUI C, WANG M, et al. From Detection to Countermeasure: A Review of Key Technologies in Counter-Unmanned Aerial Vehicle Systems [J]. IET Cyber-Systems and Robotics, 2026, 8(1).

[8] DONG Y, WU F, ZHANG S, et al. Securing the Skies: A Comprehensive Survey on Anti-UAV Methods, Benchmarking, and Future Directions[J]. arXiv preprint arXiv:2504.11967, 2026.

[9] AOULADHADJ D, KPRE E, DENIAU V, et al. Drone Detection and Tracking Using RF Identification Signals[J]. Sensors, 2023, 23(17): 7650.

[10] GHOULI Z. Passive Acoustic Detection and Localization of Drones Using MEMS Microphones and Machine Learning[J]. Acta Acustica, 2026.

[11] TAO Y, et al. Intelligent Multimodal Multi-Sensor Fusion-Based UAV Identification, Localization, and Countermeasures[J]. arXiv preprint arXiv:2510.22947, 2025.

[12] KHAZAI S, MIRZAEI S. A Comparative Analysis of YOLO and Faster R-CNN Algorithms for Micro-UAVs Detection in Surveillance Videos[J]. Earth Observation and Geomatics Engineering, 2025.

[13] REN S, HE K, GIRSHICK R, et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks[C]//NeurIPS. 2015: 91-99.

[14] LIU W, ANGUELOV D, ERHAN D, et al. SSD: Single Shot MultiBox Detector[C]//ECCV. 2016: 21-37.

[15] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[C]//CVPR. 2016: 779-788.

[16] Ultralytics. YOLOv5[Z]. https://github.com/ultralytics/yolov5. 2020.

[17] JOCHER G, CHAURASIA A, QIU J. Ultralytics YOLOv8[Z]. https://github.com/ultralytics/ultralytics. 2023.

[18] ZHAO Y, LV W, XU S, et al. DETRs Beat YOLOs on Real-time Object Detection[C]//CVPR. 2024: 16965-16974.

[19] KHALILI B, SMYTH A W. SOD-YOLOv8: Enhancing YOLOv8 for Small Object Detection in Traffic Scenes[J]. arXiv preprint arXiv:2408.04786, 2024.

[20] WOO S, PARK J, LEE J Y, et al. CBAM: Convolutional Block Attention Module[C]//ECCV. 2018: 3-19.

[21] OUYANG D, HE S, ZHANG G, et al. Efficient Multi-Scale Attention Module with Cross-Spatial Learning[C]//ICASSP. 2023: 1-5.

[22] XU W, CUI C, JI Y, et al. YOLOv8-MPEB Small Target Detection Algorithm Based on UAV Images[J]. Heliyon, 2024, 10(8): e29501.

[23] XIAO X, LIN Y, WANG S, et al. MSDF-Net: A Cross-Version Lightweight Detection Framework Based on Deformable Convolution and High-Resolution Feature Enhancement for Pine Wilt Disease[J]. Frontiers in Plant Science, 2026.

[24] GEVORGYAN Z. SIoU Loss: More Powerful Learning for Bounding Box Regression[C]//arXiv preprint arXiv:2205.12740. 2022.

[25] ZHENG Y, HU Z, YANG H, et al. Det-Fly: A Large-Scale Dataset for UAV Detection[J]. arXiv preprint, 2023.

[26] DU D, QI Y, YU H, et al. The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking[J]. ECCV, 2018: 370-386.

[27] KURMASHEV I, SEMEMYUK V, LUPIDI A, et al. Study of the Optimal YOLO Visual Detector Model for Enhancing UAV Detection and Classification[J]. Drones, 2025, 9(11): 732.

[28] BEWLEY A, GE Z, OTT L, et al. Simple Online and Realtime Tracking[C]//ICIP. 2016: 3464-3468.

[29] WOJKE N, BEWLEY A, PAULUS D. Simple Online and Realtime Tracking with a Deep Association Metric[C]//ICIP. 2017: 3645-3649.

[30] KALMAN R E. A New Approach to Linear Filtering and Prediction Problems[J]. Journal of Basic Engineering, 1960, 82(1): 35-45.

[31] SAGE A P, HUSA G W. Adaptive Filtering with Unknown Prior Statistics [J]. Joint Automatic Control Conference, 1969: 760-769.

[32] PX4 Autopilot. PX4 Autopilot[Z]. https://px4.io/. 2024.

[33] PX4. jMAVSim Simulator[Z]. https://github.com/PX4/PX4-Autopilot/tree/main/Tools/simulation/jmavsim. 2024.

[34] LIN T Y, DOLLÁR P, GIRSHICK R, et al. Feature Pyramid Networks for Object Detection[C]//CVPR. 2017: 2117-2125.

[35] LIU S, QI L, QIN H, et al. Path Aggregation Network for Instance Segmentation[C]//CVPR. 2018: 8759-8768.

[36] ZHANG Z. A Flexible New Technique for Camera Calibration[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330-1334.

[37] MAVLink. MAVLink Developer Guide[Z]. https://mavlink.io/. 2024.

[38] AGAFONKIN V. Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps[Z]. https://leafletjs.com/. 2024.

致谢

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集